

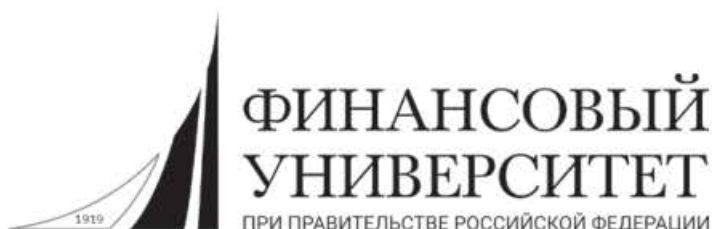


Э.Г. Дадян

ОСНОВЫ ЯЗЫКА ПРОГРАММИРОВАНИЯ 1С 8.3



УЧЕБНОЕ
ПОСОБИЕ



Э.Г. ДАДЯН

ОСНОВЫ ЯЗЫКА ПРОГРАММИРОВАНИЯ 1С 8.3

УЧЕБНОЕ ПОСОБИЕ

*Рекомендовано
Межрегиональным учебно-методическим советом
профессионального образования в качестве учебного пособия
для студентов высших учебных заведений, обучающихся
по укрупненной группе специальностей и направлений
38.00.00 «Экономика и управление»
(протокол № 13 от 16.09.2019)*

**Электронно-
Библиотечная
Система
znanium.com**

Москва
ВУЗОВСКИЙ УЧЕБНИК
ИНФРА-М
2020

УДК 004.43(075.8)
ББК 32.973-018.1я73
Д14

ФЗ
№ 436-ФЗ

Издание не подлежит маркировке
в соответствии с п. 1 ч. 4 ст. 11

Д14 **Дадян Э.Г.**
Основы языка программирования 1С 8.3 : учебное пособие /
Э.Г. Дадян. — Москва : Вузовский учебник : ИНФРА-М, 2020. —
133 с. — (Высшее образование: Бакалавриат).

ISBN 978-5-9558-0164-3 (Вузовский учебник)

ISBN 978-5-16-012380-6 (ИНФРА-М, print)

ISBN 978-5-16-105375-1 (ИНФРА-М, online)

Основы языка программирования 1С 8.3, представленные в данном учебном пособии, предназначены и для начинающих, и для продвинутых пользователей. Изложение материала в учебном пособии разбито на два раздела. В первом разделе даются основы языка программирования 1С 8.3 для начинающих. Во втором разделе применение языка 1С 8.3 дается для продвинутых пользователей — в справочной форме, на конкретных примерах.

Предназначено для студентов высших учебных заведений, обучающихся по всем направлениям подготовки и специальностям, а также для аспирантов и слушателей ИПК.

УДК 004.43(075.8)
ББК 32.973-018.1я73



Данная книга доступна в цветном исполнении
в электронно-библиотечной системе Znanium.com

ISBN 978-5-9558-0164-3 (Вузовский учебник)

ISBN 978-5-16-012380-6 (ИНФРА-М, print)

ISBN 978-5-16-105375-1 (ИНФРА-М, online)

© Дадян Э.Г., 2016

© Вузовский учебник, 2016

Подписано в печать 25.10.2019.

Формат 60×90/16. Бумага офсетная. Гарнитура Newton.
Печать цифровая. Усл. печ. л. 8,31. ППТ50. Заказ № 00000

ТК 641227-1066510-311016

ООО «Издательский Дом «Вузовский учебник»
127247, Москва, ул. С. Ковалевской, д. 1, стр. 52
www.vuzbook.ru

ООО «Научно-издательский центр ИНФРА-М»
127214, Москва, ул. Полярная, д. 31В, стр. 1
Тел.: (495) 280-15-96, 280-33-86. Факс: (495) 280-36-29
E-mail: books@infra-m.ru <http://www.infra-m.ru>

Отпечатано в типографии ООО «Научно-издательский центр ИНФРА-М»
127214, Москва, ул. Полярная, д. 31В, стр. 1
Тел.: (495) 280-15-96, 280-33-86. Факс: (495) 280-36-29

ВВЕДЕНИЕ

Основы языка программирования 1С 8.3, представленные в данном учебном пособии, предназначены и для начинающих, и для продвинутых пользователей. Обычно такие люди заинтересованы в освоении продуктов фирмы 1С. Они пытаются глубже разобраться в интересующих их предметных областях, корректировать имеющиеся приложения и, даже, разрабатывать новые. Это и внедренцы и консультанты, и главные бухгалтеры, студенты и аспиранты, и состоявшиеся специалисты. В своей профессии они достигли уже такого уровня, что начинают осваивать смежные.

И это правильно, потому что знание элементарных основ программирования позволит специалисту в любой предметной области достичь новых высот в своем деле.

Например, бухгалтер, владеющий основами программирования и конфигурирования в среде 1С: Предприятие, сможет не только автоматизировать свою деятельность без помощи программистов, но и найти ошибки и понять, почему программа выдает тот или иной результат. Если ему и понадобится помощь программистов — он сможет разговаривать с ними на одном языке. Грамотно ставить задачи. А это уже 50% успеха.

Итак, вы решили освоить язык программирования 1С 8.3 (или хотя бы его основы) и не знаете с чего начать? Растерянность на этом этапе вполне понятна. И тут бесполезно лезть в конфигуратор и сразу пытаться создавать новые справочники, документы и прочие объекты.

Самое главное, что вам нужно сделать, это усвоить суть программирования. Ухватить сам принцип выполнения программы. А для этого нужно не объекты новые создавать в базе, а изучать основы самого языка программирования 1С 8.3. А все остальное уже потом приложится и будет изучено быстро и легко.

Изложение материала в пособии разбито на два раздела. В первом разделе даются основы языка программирования 1С 8.3 для начинающих. Во втором, в справочной форме, на конкретных примерах, дается применение языка 1С 8.3 для продвинутых пользователей, изучивших материал первой части.

Материал пособия излагается последовательно, по сложности освоения — "от простого к сложному", по методике изложения "key—by—key" (делай как я).

Примечание.

В тексте пособия часто для краткости записи вместо "1С 8.3" используется "1С".

Раздел 1 ОСНОВЫ ЯЗЫКА ПРОГРАММИРОВАНИЯ 1С 8.3 ДЛЯ НАЧИНАЮЩИХ ПОЛЬЗОВАТЕЛЕЙ

ЮНИТ 1. УСТАНОВКА 1С И НАША ПЕРВАЯ ПРОГРАММА

Для обучения программированию нам понадобится 1С 8.3 (не ниже 8.3.4.482).

Если у вас есть установленная 1С версии 8.3 — используйте ее. Если нет — скачайте и установите учебную версию, которую фирма 1С выпускает специально для образовательных целей.

Задание №1. Скачать и установить учебную версию 1С 8.3.

Инструкция для выполнения:

1. Распакуйте архив, скачанный по ссылке в отдельную папку на рабочем столе:

1.1. ссылка на скачивание с Dropbox (вариант №1)

https://www.dropbox.com/s/t69xyrrhsdugfzf/1C_Predprijatje_8.3_uchebnaja_versija.zip?dl=1

1.2. ссылка на скачивание с Mail.ru (вариант №2)

https://cloud.mail.ru/public/59a9a532864d/1C_Predprijatje_8.3_uchebnaja_versija.zip

2. Запустите файл "setup.exe".

3. Нажмите "Далее", "Далее", "Установить".

4. Дождитесь окончания установки.

После выполнения Задания №1 на вашем рабочем столе должен появиться вот такой ярлык (рис. 1):

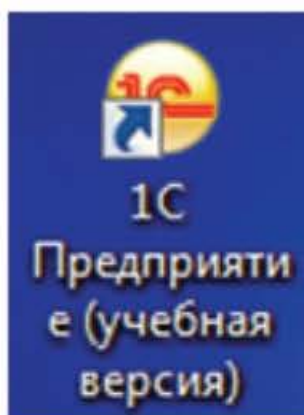


Рис. 1. Ярлык запуска 1С:Предприятие

Первая программа

Это уже стало традицией для программистов — изучая новый язык, первое, что на нем пишут, это программу, печатающую на экране фразу "Hello, World!".

Заставим компьютер вывести на экран эту фразу на русском языке "Привет, Мир!".

Задание №2. Написать программу на языке 1С 8.3, которая при запуске выводит на экран фразу "Привет, Мир!".

Инструкция для выполнения:

1. Запустите ярлык "1С:Предприятие" на вашем рабочем столе.
2. Перед вами открылся список баз 1С (рис. 2). Как видите, он пуст. Давайте создадим учебную базу, в которой и будем программировать. Нажмите кнопку "Добавить".

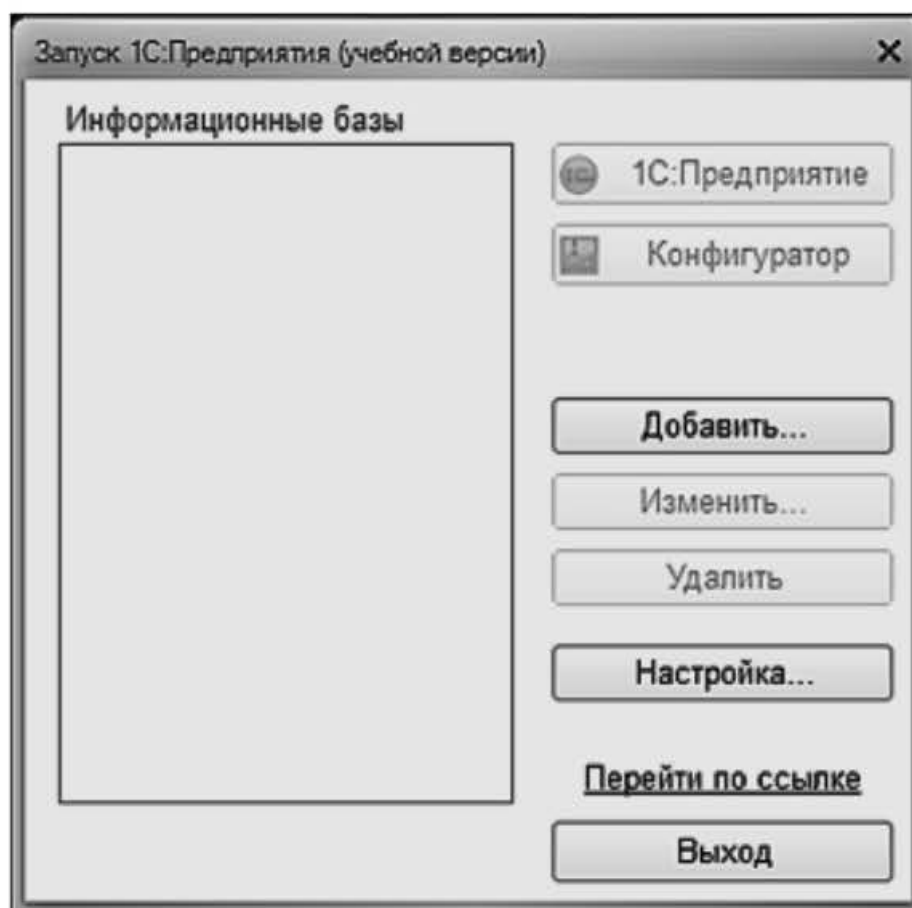


Рис. 2. Окно списка баз 1С

3. Выберите пункт "Создание новой информационной базы" и нажмите кнопку "Далее" (рис. 3).

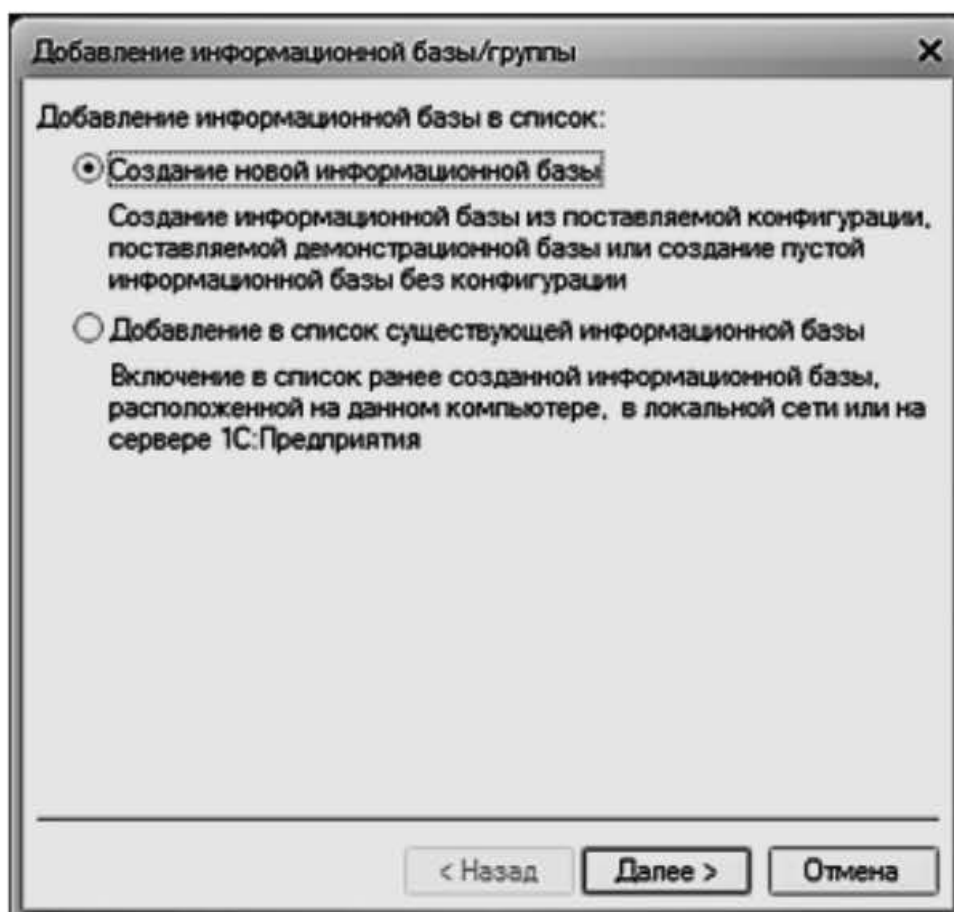


Рис. 3. Создание новой информационной базы

4. Выберите пункт "Создание информационной базы без конфигурации ..." (рис. 4) и нажмите кнопку "Далее".

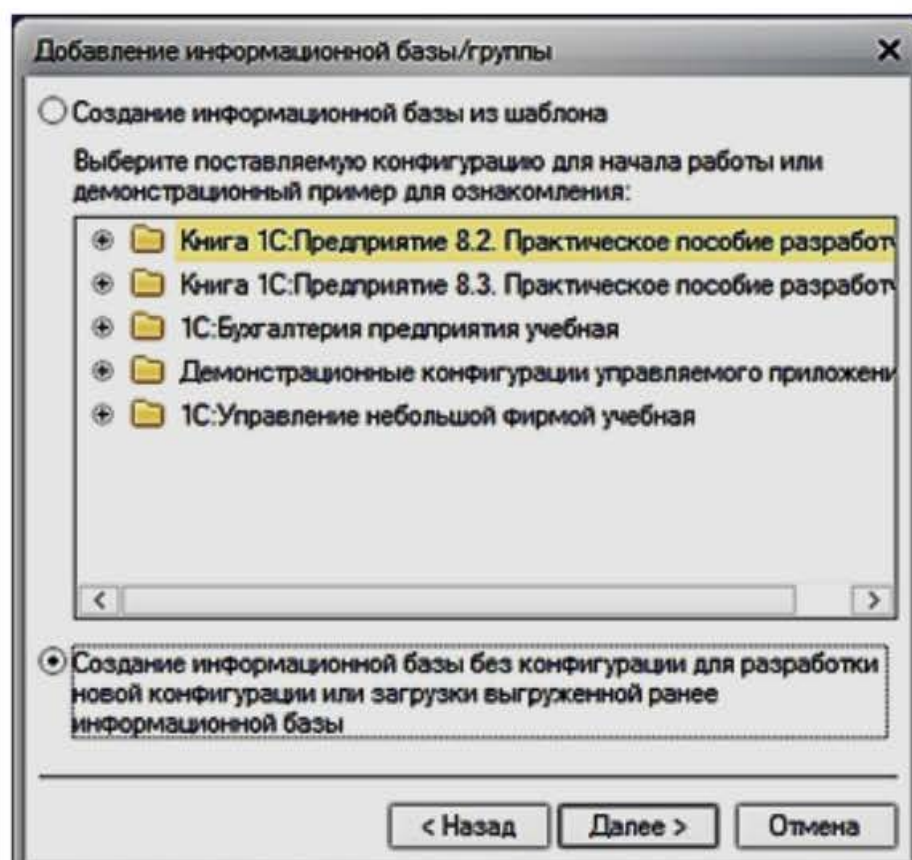


Рис. 4. Создание информационной базы без конфигурации

5. В качестве имени базы укажите "Учебная" (рис. 5) и нажмите кнопку "Далее".

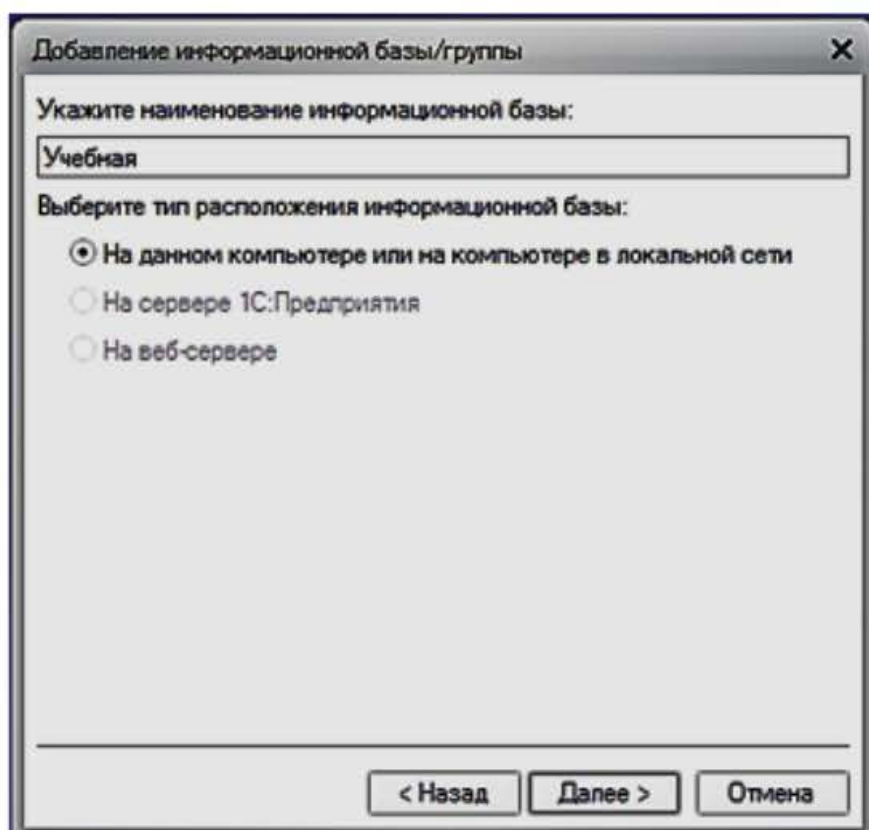


Рис. 5. Указание наименования информационной базы

6. В качестве каталога базы укажите любую пустую папку (в данном случае это папка "Учебная" в моих документах (рис. 6)). Нажмите кнопку "Далее".

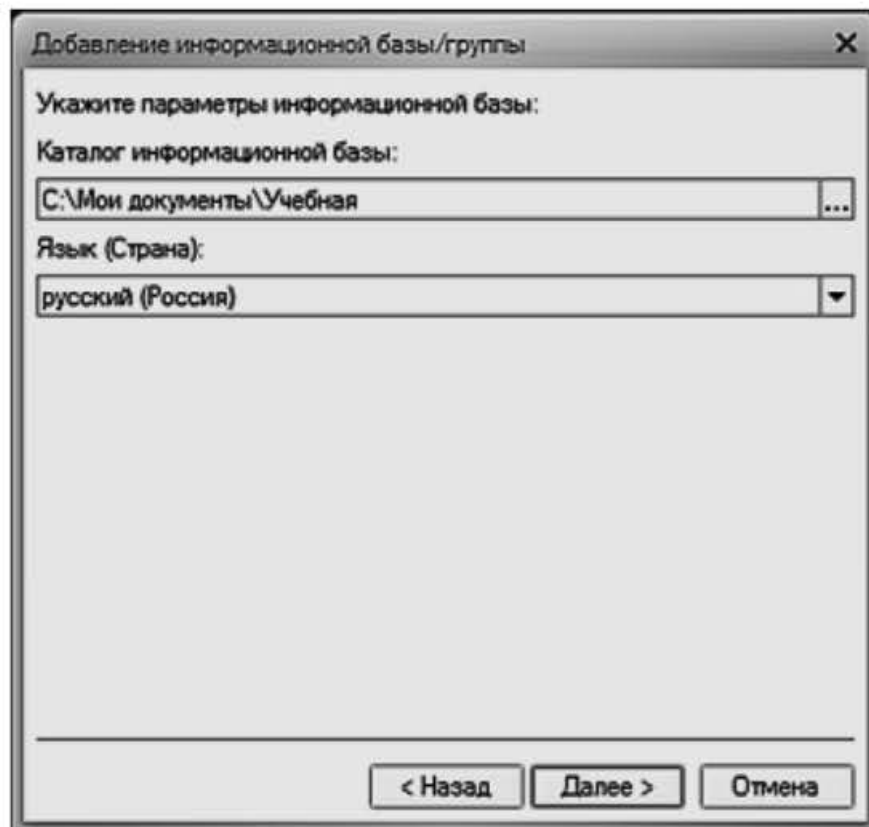


Рис. 6. Указание параметров информационной базы

7. Укажите вариант аутентификации "*Выбирать автоматически*", режим запуска "*Толстый клиент*". В качестве версии 1С:Предприятия ничего не указывайте, нажмите кнопку "Готово" (рис. 7).

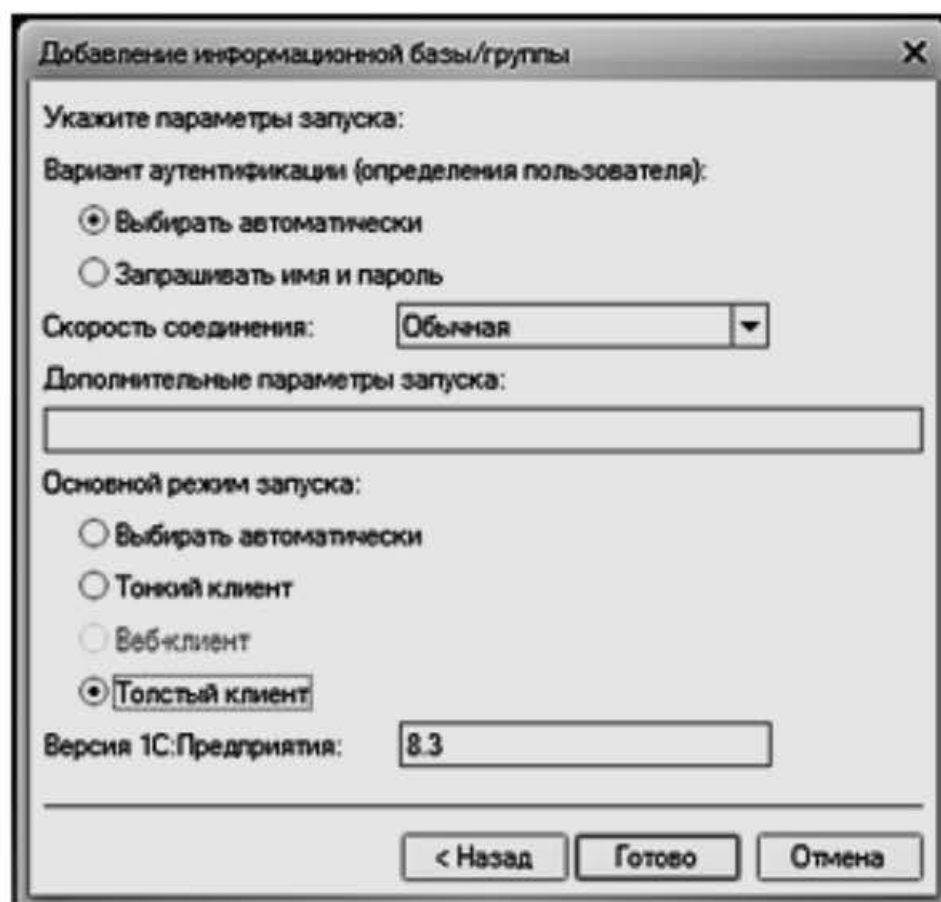


Рис. 7. Указание параметров запуска

8. Как видите, в списке, наконец, появилась первая база, которую мы только что создали сами (рис. 8)! Начнем писать код. Нажмите кнопку "*Конфигуратор*".

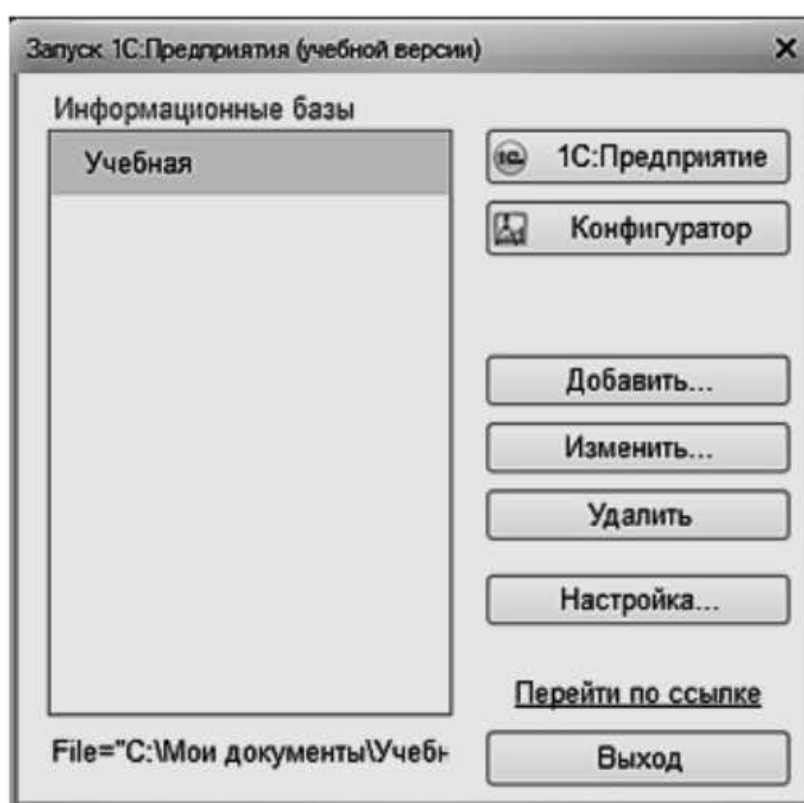


Рис. 8. Запуск Конфигуратора

9. В открывшемся окне, выберите через меню "*Конфигурация*" пункт "*Открыть конфигурацию*" (рис. 9).

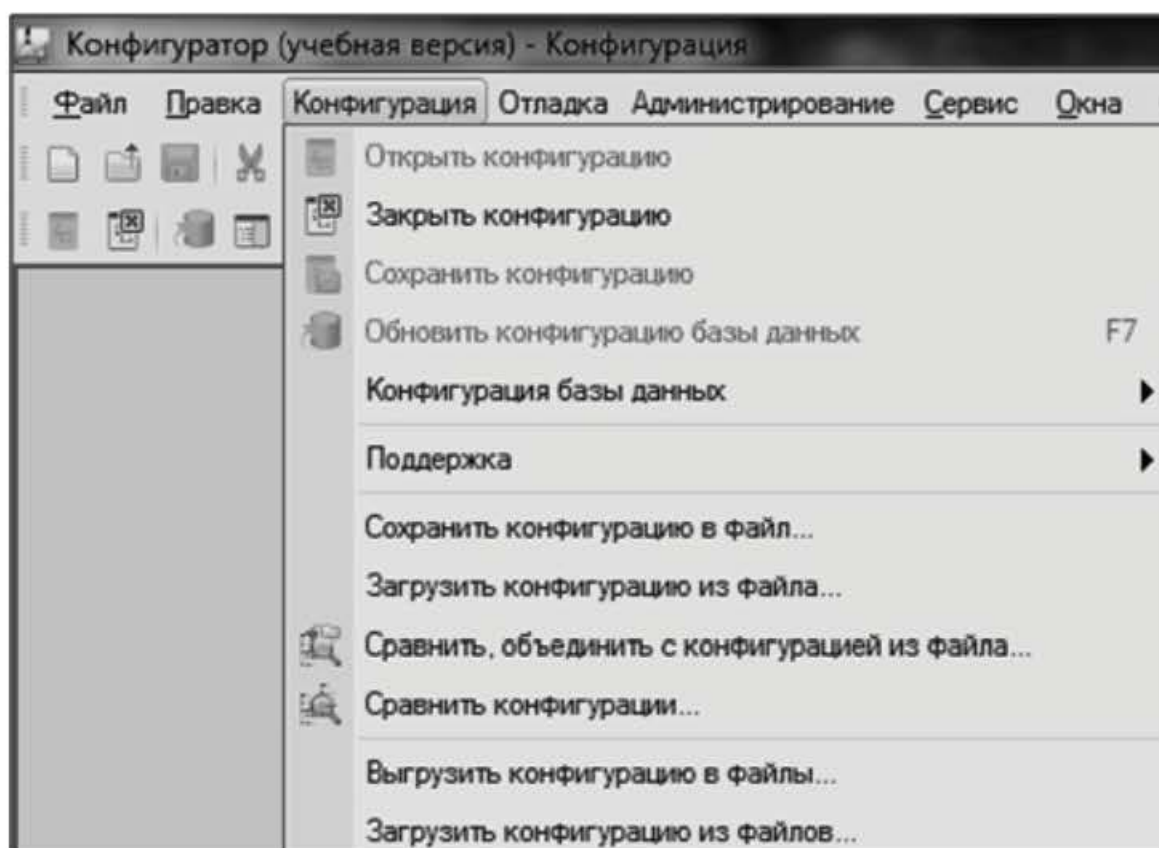


Рис. 9. Выбор пункта меню "Открыть конфигурацию"

10. В открывшейся слева панели нажмите правой кнопкой мыши на "Конфигурация" и выберите из списка "Открыть модуль управляемого приложения" (рис. 10).

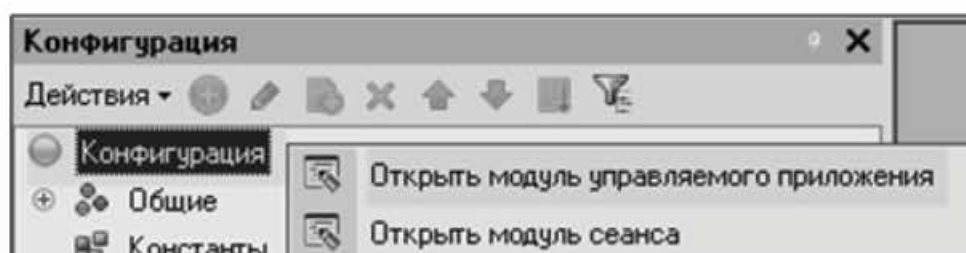


Рис. 10. Выбор пункта "Открыть модуль управляемого приложения"

11. Открылось окно с редактором (рис. 11). Сюда можно писать текст! С ним мы и будем работать в дальнейшем.

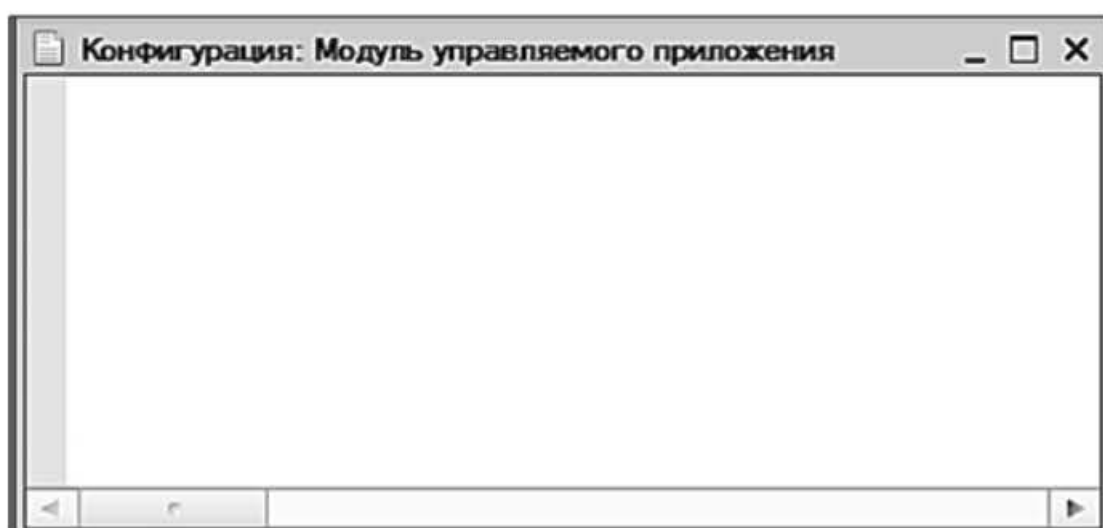


Рис. 11. Модуль управляемого приложения

12. Перепишите в это окно следующий текст (рис. 12):
Сообщить("Привет, Мир!");

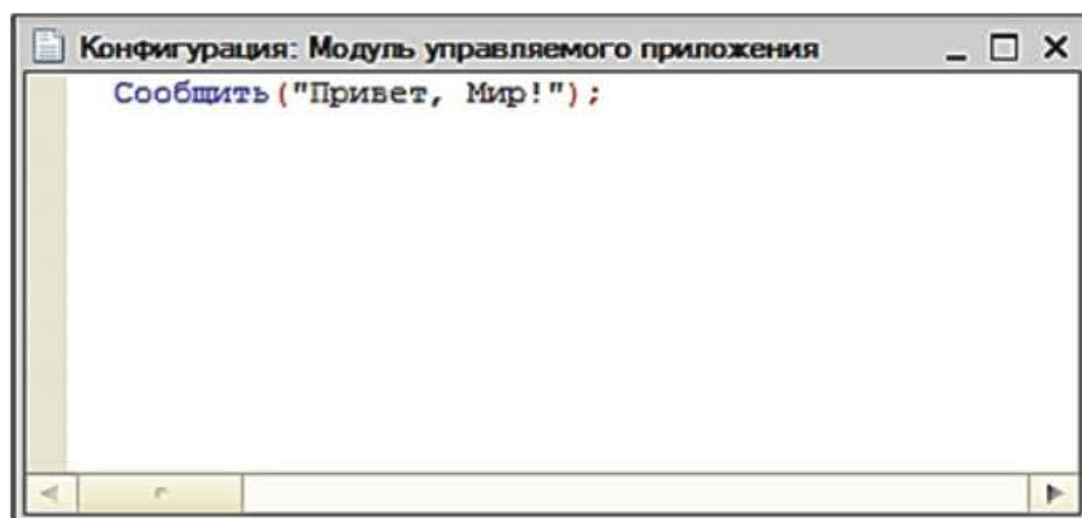
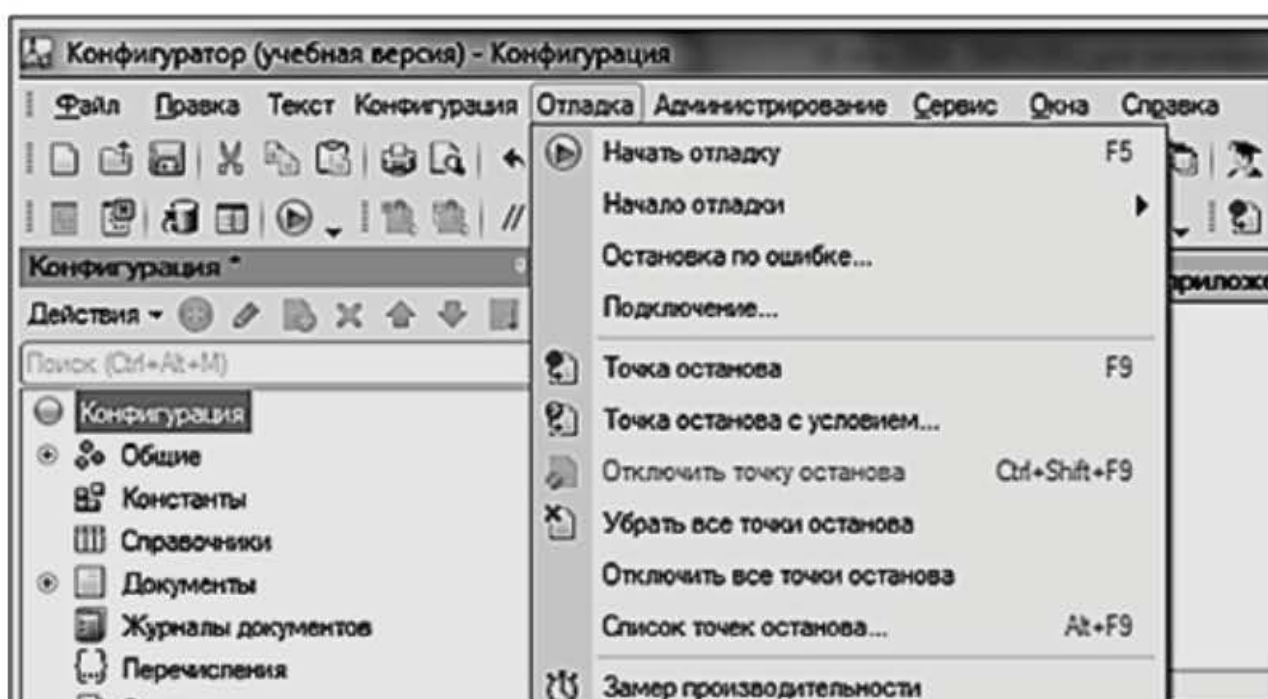


Рис. 12. Модуль управляемого приложения с текстом

13. Код нашей первой программы готов. Запустим его на выполнение. Через меню "Отладка" "Начать отладку" (или клавиша *F5*) (рис. 13).



14. На вопрос ответим утвердительно.

15. Спустя секунду или две запустится 1С и в окне снизу появится надпись "Привет, Мир!". Мы добились от компьютера того, чего хотели, заработала наша первая программа (рис. 14).

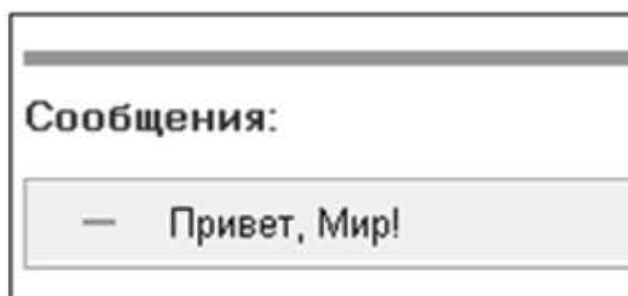


Рис. 14. Результаты работы программы

ЮНИТ 2. ОСНОВНЫЕ ПРАВИЛА ЯЗЫКА

Задание №3. Добейтесь того, чтобы компьютер при запуске вашей программы вывел следующий отрывок из стихотворения Александра Пушкина "Зимний вечер":

*Буря мглою небо кроет,
Вихри снежные крутя;
То, как зверь, она завоет,
То заплачет, как дитя.*

Обратите внимание, что стихотворение нужно вывести в четыре строки. Попробуйте его выполнить сами без подсказки и лишь при необходимости воспользуйтесь инструкцией ниже:

Инструкция

Зайдите в уже созданную на прошлом занятии базу "Учебная" в режиме конфигуратор (заново создавать базу не нужно).

Так же, как и в первом юните, откройте модуль управляемого приложения.

Там вы обнаружите свой старый код:

Сообщить("Привет, Мир!");

Сотрите текст в редакторе и вместо него напишите следующее:

Сообщить("Буря мглою небо кроет,");

Сообщить("Вихри снежные крутя,");

Сообщить("То, как зверь, она завоет,");

Сообщить("То заплачет, как дитя.");

Запустите программу через меню "Отладка" $\Rightarrow$ "Начать отладку" (или клавиша F5).

Первые наблюдения о языке

А теперь давайте внимательно присмотримся к написанной программе:

```
Сообщить("Буря мглою небо кроет,");  
Сообщить("Вихри снежные крутя;");  
Сообщить("То, как зверь, она завоет,");  
Сообщить("То заплачет, как дитя.");
```

Вроде бы обычный текст. Но на самом деле он написан по особым правилам, которые делают его понятным для компьютера.

Компьютер считывает этот текст (еще говорят "код программы") строку за строкой и выполняет задуманные программистом команды.

Что мы можем сказать об этом тексте?

1. Он разделен на строчки.
2. Каждая строчка содержит отдельную *команду*, которая называется "Сообщить" и заканчивается точкой с запятой.
3. Каждая команда "Сообщить" имеет рядом с собой некий текст, заключенный в круглые скобки — этот текст называется *параметром* команды.
4. Текст внутри круглых скобок заключен в двойные кавычки, как мы обычно поступаем при цитатах.

На самом деле в этих четырех наблюдениях, замеченных нами лишь по небольшому фрагменту программы, уже содержатся правила внутреннего языка программирования 1С 8.3. Но сформулируем мы их позже, а пока просто обратите внимание на эти особенности текста.

Задание №4. Сделайте 6 вариантов изменения нашего кода таким образом, чтобы каждый из них отличался хоть чем-нибудь от других, но все они выводили одинаковый результат — а именно те самые 4 строчки из Пушкина.

Если программа после вашего изменения запустилась и вывела стихотворение — значит, ваш вариант подходит. Если появляется сообщение об ошибке — значит, так менять текст нельзя, потому что это противоречит правилам внутреннего языка 1С 8.3.

Рассмотрим эти варианты:

// **вариант №1**

```
Сообщить  
("Буря мглою небо кроет,");  
Сообщить("Вихри снежные крутя;");  
Сообщить("То, как зверь, она завоет,");  
Сообщить("То заплачет, как дитя.");
```

// вариант №2

Сообщить

("Буря мглою небо кроет,")

;

Сообщить("Вихри снежные крутя;");Сообщить("То, как зверь, она завоет,");

Сообщить("То заплачет, как дитя.");

// вариант №3

Сообщить("Буря мглою небо кроет,");

Сообщить(

"Вихри снежные крутя;");

Сообщить("То, как зверь, она завоет,");

Сообщить("То заплачет, как дитя.");

// вариант №4

Сообщить("Буря мглою небо кроет,");

Сообщить("Вихри снежные крутя;");

Сообщить("То, как зверь, она завоет,");

Сообщить("То заплачет, как дитя.");

// вариант №5

Сообщить("Буря мглою небо кроет,");

Сообщить("Вихри снежные крутя;");

Сообщить("То, как зверь, она завоет,");

Сообщить("То заплачет, как дитя.");

// вариант №6

Сообщить("Буря мглою небо кроет,");

Сообщить("Вихри снежные крутя;");

Сообщить("То, как зверь, она завоет,");

Сообщить("То заплачет, как дитя.");

Обратите внимание на пояснения (**Полужирным** шрифтом), которые использованы в решении. Такие пояснения (по-другому **комментарии**) могут встречаться где угодно в коде. **Комментарий начинается с двух косых черт** и продолжается до конца строки. Компьютер комментариев не читает — они предназначены для человека, чтобы легче и быстрее разбираться в написанной программе. Наличие или отсутствие комментариев никак не влияет на результат выполнения программы.

Правила языка 1С

Итак, из наших экспериментов можно сделать следующие выводы о правилах языка 1С 8.3:

- Код программы состоит из команд для компьютера (пока мы познакомились только с командой Сообщить).
- Все команды языка отделяются друг от друга точкой с запятой.
- Чтобы отдать компьютеру команду на выполнение, нужно передать некий параметр (текст в круглых скобках рядом с командой), чтобы команда знала, что мы от нее хотим.
- Одна команда (вместе с параметром и точкой с запятой) может занимать сразу несколько строчек в редакторе.
- Компьютер выполняет команды последовательно — одну за другой.

Проверьте себя

Задание №5. Ответьте на следующие вопросы по уже изученным правилам внутреннего языка IC 8.3:

1. Сколько команд содержится в приведенном ниже фрагменте кода:

```
Сообщить ("Буря мглою небо кроет, ");
Сообщить ("Вихри снежные крутя;");
Сообщить ("То, как зверь, она завоет, ");
Сообщить ("То заплачет, как дитя.");
```

2. Назовите параметр второй по счету команды из приведенного фрагмента кода.

3. Можно ли уместить все эти команды в одну строчку?

4. Назовите третью по счету команду целиком.

5. Переделайте фрагмент кода так, чтобы он вывел указанные строчки стиха в обратном порядке.

Ответы

1. Четыре команды. Мы четыре раза просим компьютер вывести ту или иную строку из стиха.

2. "Вихри снежные крутя;". Параметром команды является все то, что заключено в круглые скобки сразу после имени команды.

3. Да, можно. Главное, чтобы между командами была точка с запятой. Это разделитель для компьютера. На строчки мы их разбиваем для "красоты", чтобы человеку проще было читать.

4. Командой является не только ее имя, но и параметр вместе с точкой запятой в самом конце:

```
Сообщить("То, как зверь, она завоет,");
```

5. Так как компьютер читает и выполняет команды последовательно, просто переставим их в обратном порядке:

Сообщить("То заплачет, как дитя.");
Сообщить("То, как зверь, она завоет,");
Сообщить("Вихри снежные крутя");
Сообщить("Буря мглою небо кроет,");

ЮНИТ 3. ТИПЫ ДАННЫХ В ЯЗЫКЕ 1С

Одна из главных задач программирования — обработка данных, поэтому необходимо разобраться, какие основные типы данных используются в языке 1С:

- **Строка.** Отличный пример таких данных — цитата из стихотворения *"Буря мглою небо кроет,"*. Обратите внимание, что данные типа Строка всегда заключаются в *двойные кавычки*.
- **Число.** Всем нам знакомый тип данных. С ним мы часто имеем дело в реальной жизни: на работе в день зарплаты, а затем в магазине. Числа, как вы, конечно, знаете, могут быть *целые* и *дробные*. Пример целого числа: 1000000, ну, а дробного — 3.14. Дробная часть от целой отделяется *точкой*. Числа также имеют знак: *плюс* или *минус*.
- **Дата.** Дата вашего рождения и есть пример данных такого типа. Она включает в себя год, месяц и день. Например, первый полет человека в космос состоялся 12 апреля 1961 года. На языке компьютера эти данные будут выглядеть так: *'19610412'*. Дата всегда заключается в *одинарные кавычки*.

Задание №6. Запишите в формате компьютера (используя нужный тип данных) ответы на следующие вопросы:

1. Когда был запущен первый искусственный спутник?
2. Как звали первого космонавта?
3. Сколько планет в солнечной системе?

Проверьте себя

'19571004'

"Юрий Гагарин"

8

Задание №7. Напишите программу, которая заставит компьютер вывести ваши ответы на экран. Каждый ответ должен быть выведен как данные своего типа.

Проверьте себя

Сообщить('19571004');

Сообщить("Юрий Гагарин");

Сообщить(8);

По результатам выполнения задания №7 можно сделать интересный вывод: в качестве параметра для команды *Сообщить* можно передавать не только строки, но и данные любого другого типа.

Обработка данных

Какие операции мы можем совершать над данными? Это зависит от их типа.

Например, строки мы можем соединять одну с другой при помощи знака сложения:

Сообщить("Юрий" + " " + "Гагарин");

Над числами можно выполнять различные арифметические операции:

Сообщить(2 + 2);

Сообщить(9 / 3);

Сообщить(100 - 50);

Сообщить(3 * 3);

Для более сложных арифметических выражений использовать скобки:

Сообщить((2 + 2) * (9 / 3) + (10 - 5) * 2);

К дате можно прибавлять и отнимать секунды:

Сообщить('19561025' + 1);

Задание №8. Выполните указанные примеры на компьютере.

Задание №9. Запишите в формате компьютера следующую дату: 19 июля 1980 года. Напишите программу, которая прибавит к этой дате столько секунд, чтобы в результате получилось 20 июля 1980 года.

Проверьте себя

Сообщить('19800719' + 60 * 60 * 24);

Почему прибавляются именно столько секунд? В сутках — 24 часа, в часе — 60 минут, в минуте — 60 секунд. Все просто.

Преобразование данных

А что будет, если мы попытаемся обработать различные типы данных в одной команде? Напишем, например, вот так:

Сообщить("3 * 4 = " + 3 * 4);

Какой ответ компьютер выведет на экран?

Подсказка

В результате мы увидим строку: 3 * 4 = 12

Компьютер видит, что мы пытаемся прибавить к строке $3 * 4 =$ результат выполнения арифметической операции, а так как к строке можно прибавить только строку — он преобразует число 12 к строковому типу данных.

Часто возникают вопросы по определению типов данных у выражений. Рассмотрим эти примеры подробнее.

Какой тип будет иметь результат следующего выражения $"123" + 456$?

Перед нами выражение, в котором складываются два значения: левое (строка $"123"$) и правое (число 456).

Компьютер смотрит сначала на левое значение и видит там строку $"123"$. Из этого он делает вывод, что результатом всего выражения тоже будет строка.

Поэтому он и правое значение (число 456) пытается превратить в строку. И ему это без труда удастся, ведь перевести любое число в строку очень просто. В данном случае получается строка $"456"$. Таким образом, результатом этого выражения является значение $"123456"$, то есть строка.

Какой тип будет иметь результат следующего выражения $100 + "001"$?

Перед нами выражение, в котором складываются два значения: левое (число 100) и правое (строка $"001"$). Компьютер смотрит сначала на левое значение и видит там число 100. Из этого он делает вывод, что результатом всего выражения тоже будет число. Поэтому он и правое значение (строка $"001"$) пытается превратить в число. И ему это удастся, ведь строка $"001"$ без ошибок переводится в число 1. Таким образом, результатом этого выражения является значение 101, то есть число.

Задание №10. Напишите программу, которая выводит на компьютер таблицу умножения для числа 7.

Проверьте себя(решение)

Сообщить($"7 * 1 = " + 7 * 1$);

Сообщить($"7 * 2 = " + 7 * 2$);

Сообщить($"7 * 3 = " + 7 * 3$);

Сообщить($"7 * 4 = " + 7 * 4$);

Сообщить($"7 * 5 = " + 7 * 5$);

Сообщить($"7 * 6 = " + 7 * 6$);

Сообщить($"7 * 7 = " + 7 * 7$);

Сообщить($"7 * 8 = " + 7 * 8$);

Сообщить($"7 * 9 = " + 7 * 9$);

Сообщить($"7 * 10 = " + 7 * 10$);

ЮНИТ 4. ИМЕНА ПЕРЕМЕННЫХ В ЯЗЫКЕ 1С

В предыдущем юните мы узнали, какие бывают данные, и научились совершать с ними простейшие операции.

Назначение имен для данных

Задание имен для программы — очень увлекательное занятие, но требуется соблюдать ряд правил:

- Имя может состоять только из букв русского и английского алфавитов, цифр и знаков подчеркивания.
- Имя не должно начинаться с цифры.

Пример. Количество планет нашей системы — 8. Это числовой тип данных.

Подходящим именем в данном случае будет *КоличествоПланет*. Чтобы связать наше имя и цифру 8, используем знак *равно*. Не забудем поставить в конце точку с запятой, так как связывание цифры 8 и имени — отдельная команда для компьютера.

КоличествоПланет = 8;

Теперь, после выполнения связывания имени *КоличествоПланет* с восьмеркой, мы можем обращаться к цифре 8, используя это имя.

КоличествоПланет = 8;
Сообщить(КоличествоПланет);

Код программы стал более наглядным. Это одно из назначений имен — делать код программы более понятным.

Сравните два примера одного и того же кода:

Без использования имен

Сообщить('20001231' + 24 * 60 * 60);

С использованием имен

СекундВМинуте = 60;
СекундВЧасе = 60 * СекундВМинуте;
СекундВСутках = 24 * СекундВЧасе;
ПоследнийДень20Века = '20001231';
ПервыйДень21Века = ПоследнийДень20Века + СекундВСутках;
Сообщить(ПервыйДень21Века);

Второй пример получился длиннее, но понятнее. А понятность — очень важный критерий при написании кода.

Нужно всегда помнить, что программа пишется не только для компьютера, но и для других программистов. Они будут ее дорабатывать, исправлять наши ошибки.

И нет ничего хуже, если программа написана плохо, неряшливо, подобраны неудачные имена для данных, а вам требуется не только разобраться, как она работает, но еще и добавить в нее что-то новое.

Итак, мы можем и даже должны придумывать для наших данных понятные и красивые имена. Это отличает хорошего программиста от плохого.

Проверьте себя

Задание №11.Пройдите следующий тест, используя правила для составления имен, описанные выше.

Являются ли допустимыми следующие имена для данных:

1. ПервыйКосмонавт
2. 7ЦветовРадуги
3. СемьЦветовРадуги
4. ЦветовРадуги7
5. Гора"Эверест"
6. Цветов_радуги_7
7. _НазваниеНашейПланеты
8. Рассвет_в космосе
9. Северный<полюс>

Ответы

ПервыйКосмонавт — верное имя.

7ЦветовРадуги — так называть данные нельзя, так как имя не должно начинаться с цифры.

СемьЦветовРадуги — верное имя.

ЦветовРадуги7 — верное имя.

Гора"Эверест" — нельзя использовать кавычки в имени.

Цветов_радуги_7 — верное имя.

_НазваниеНашейПланеты — верное имя.

Рассвет_в космосе — нельзя использовать пробелы в имени.

Северный<полюс> — нельзя использовать угловые скобки в имени.

Дополнительная настройка

Для выполнения последующих заданий необходима дополнительная настройка базы.

1. Нажмите правой кнопкой мыши на элемент "Конфигурация" и выберите пункт "Свойства" (рис. 15).

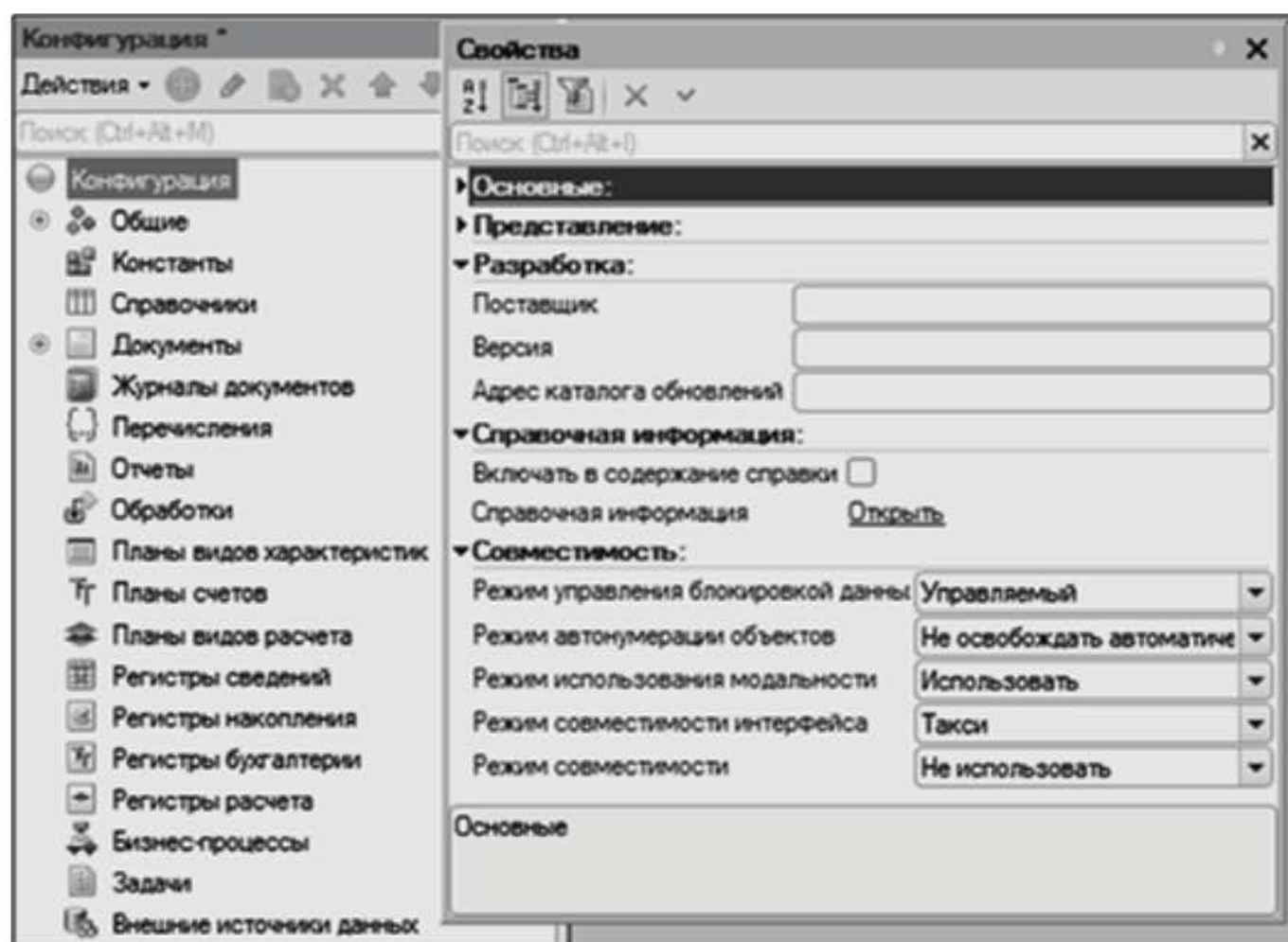


Рис. 15. Таблица свойств "Конфигуратора"

2. Внизу открывшейся справа панели найдите пункт "Режим использования модальности". Установите его значение в "Использовать" (рис. 16).

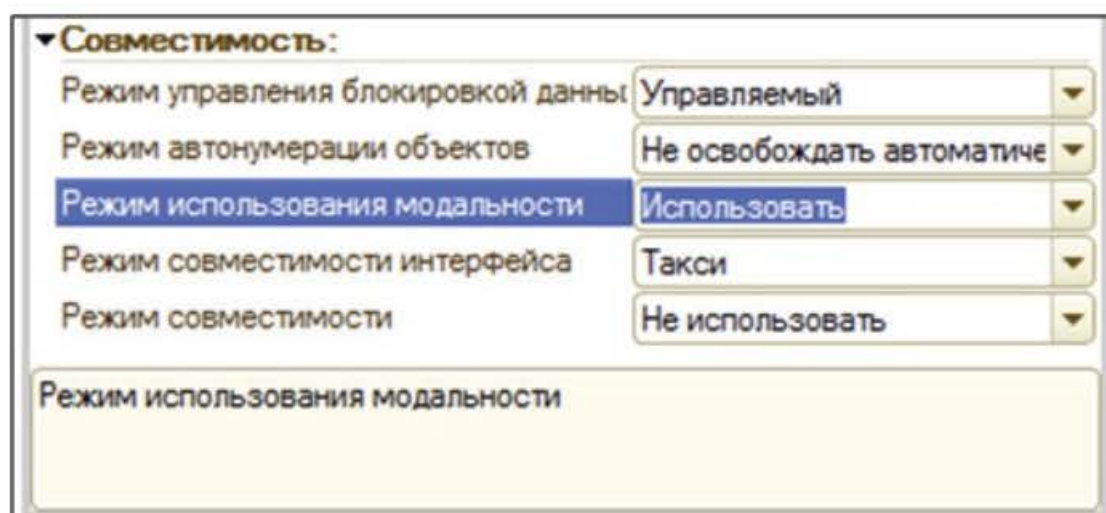


Рис. 16. Режим использования модальности

3. Нажмите на значок дискеты, чтобы сохранить изменения (рис. 17).

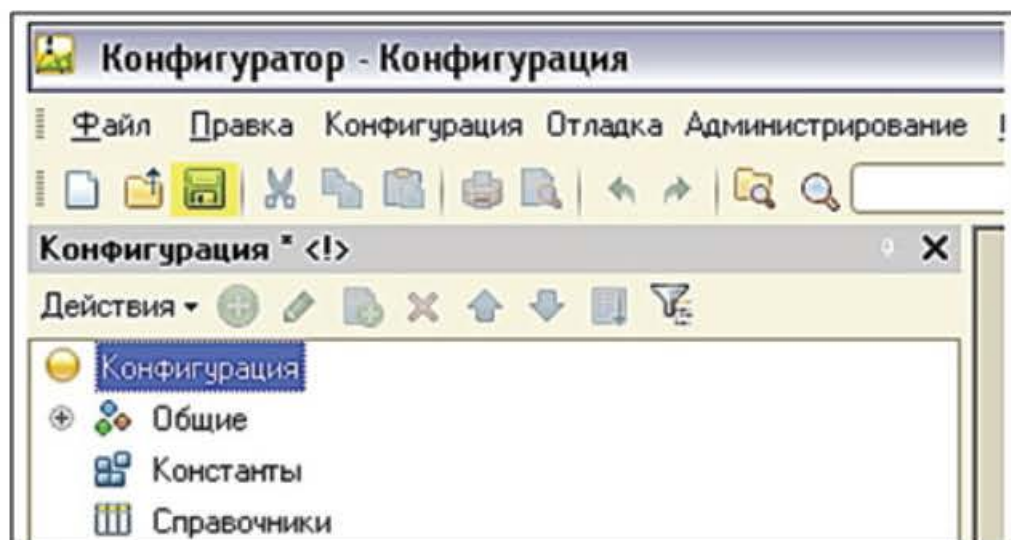


Рис. 17. Сохранение изменений

Ввод данных от пользователя

Задание 12. Требуется, чтобы пользователь ввел два числа, а компьютер посчитал и вывел их сумму.

Задача простая — складывать числа мы умеем, выводить тоже. Но как получить их от пользователя?

Для этого существует специальная команда **ВвестиЧисло**. В качестве параметра она принимает имя, с которым будет связано значение, введенное пользователем.

Введем первое число:

ЧислоА = 0;

ВвестиЧисло(ЧислоА);

Обратите внимание, что в первой строчке мы связываем имя со значением ноль. Проводим первоначальную инициализацию имени. Это нужно сделать обязательно перед первым использованием имени в качестве параметра для команды.

Введем второе число:

ЧислоБ = 0;

ВвестиЧисло(ЧислоБ);

Теперь вычислим сумму введенных чисел и свяжем ее с именем СуммаЧисел:

СуммаЧисел = ЧислоА + ЧислоБ;

Чтобы показать получившуюся сумму пользователю, используем еще одну новую команду **ОткрытьЗначение**:

ОткрытьЗначение(СуммаЧисел);

```

Весь код целиком
ЧислоА = 0; // объявили первое число и указали, что оно равно
нулю
ВвестиЧисло(ЧислоА); // ввели его новое значение от пользова-
теля
// сделали то же со вторым числом
ЧислоБ = 0;
ВвестиЧисло(ЧислоБ);
// сложили оба введенных числа
СуммаЧисел = ЧислоА + ЧислоБ;
// и вывели их значение пользователю
ОткрытьЗначение(СуммаЧисел);

```

Задание 13. Разработать программу, которая вводит от пользова-
теля радиус круга, а затем вычисляет и выводит длину его окружно-
сти. Формула для вычисления длины окружности:

$2 * \text{Пи} * \text{Радиус}$, где $\text{Пи} = 3.14$.

Решение.

```

РадиусОкружности = 0; // объявили имя РадиусОкружности
и // присвоили ему значение ноль
ВвестиЧисло(РадиусОкружности); // ввели число от пользователя
Пи = 3.14; // объявили новое имя со значением 3.14
ДлинаОкружности = 2 * Пи * РадиусОкружности; // по формуле
// вычислили длину окружности
// вывели полученное по формуле значение пользователю
ОткрытьЗначение(
    "Длина окружности радиусом " +
    РадиусОкружности + " равна" + ДлинаОкружности);

```

ЮНИТ 5. ПРОСТЫЕ ЛОГИЧЕСКИЕ ВЫРАЖЕНИЯ В ЯЗЫКЕ 1С

К уже изученным типам данных (строка, число и дата) добавим
еще один — **логический** тип. Он может принимать всего два значения:
Истина или *Ложь*.

Значение логического типа (*Истина* или *Ложь*) является резуль-
татом вычисления некоторого логического выражения.

Логическое выражение можно определить как совокупность опе-
рандов, связанных между собой знаками логических операций.

Под операндами понимаются любые объекты, над которыми вы-
полняются те или иные действия. В логических выражениях в качес-
тве операндов могут использоваться отношения, представляющие
собой выражение, составленное при помощи операций сравнения.
Сравнивать можно числа, даты, строки и другие данные. О логиче-

ском выражении можно сказать, верно оно (*Истина*) или неверно (*Ложь*).

Операции сравнения бывают следующие:

Операция	Обозначение
Равно	=
Не равно	<>
Больше	>
Меньше	<
Больше или равно	>=
Меньше или равно	<=

Пример логического выражения	Результат вычисления	Объяснение
$1 = 2$	Ложь	неверно, единица на самом деле не равна двойке
$1 = 1$	Истина	верно, единица равна единице
"Земля" <> "Луна"	Истина	верно, строка "Земля" не равна строке "Луна"
'18610101' <> '18610101'	Ложь	неверно, дата 01.01.1861 на самом деле равна дате 01.01.1861
$100 > 50$	Истина	верно, сто действительно больше пятидесяти
$10 < 0$	Ложь	неверно, на самом деле десять больше нуля

Задание №14. Укажите для каждого логического выражения его результат — *Истина* или *Ложь*.

- "Венера" = "Юпитер"
- "Венера" <> "Юпитер"
- $123 = 321$
- $123 < 321$
- $123 <= 321$
- $123 <= 123$
- '20000101' <> '20140101'
- '20000101' = '20000101'
- '20140101' > '20120101'
- $25 + 25 = 50$
- $2 * (10 + 10) > 50$
- "Юрий" + "Гагарин" = "Юрий Гагарин"

Попытайтесь ответить на все вопросы самостоятельно. Затем посмотрите ответы ниже и попытайтесь разобраться, если где-то ошиблись.

Ответы

1. "Венера" = "Юпитер" (Ложь, строки не равны между собой, а в выражении утверждается, что равны).
2. "Венера" <> "Юпитер" (Истина, строки не равны между собой, как и утверждается в выражении).
3. $123 = 321$ (Ложь, так как числа не равны между собой, а в выражении утверждается, что равны).
4. $123 < 321$ (Истина, так как 123 меньше 321, как и утверждается в выражении).
5. $123 \leq 321$ (Истина, так как 123 меньше 321, а данное утверждение истинно, если левое число меньше или равно правому).
6. $123 \leq 123$ (Истина, так как 123 равно 123, а данное утверждение истинно, если левое число меньше или равно правому).
7. '20000101' <> '20140101' (Истина, так как левая дата 01.01.2000 не равна 01.01.2014, как и утверждается в выражении).
8. '20000101' = '20000101' (Истина, так как левая дата 01.01.2000 равна 01.01.2000, как и утверждается в выражении).
9. '20140101' > '20120101' (Истина, так как первое января 2014 года больше первого января 2012 года, как и утверждается в выражении).
10. $25 + 25 = 50$ (Истина, так как 25 плюс 25 действительно равно пятидесяти, как и утверждается в выражении).
11. $2 * (10 + 10) > 50$ (Ложь, так как результат левого выражения равен 40, а 40 меньше 50, хотя в утверждении говорится обратное).
12. "Юрий" + " Гагарин" = "Юрий Гагарин" (Истина, так как сумма строк "Юрий" и " Гагарин" образует одну строку "Юрий Гагарин", которая равна строке справа, как и утверждается в выражении).

Заставим считать компьютер

Вычислим выражение $2 * (10 + 10) > 50$ на компьютере.

Для этого напишем такую программу:

Сообщить($2 * (10 + 10) > 50$);

Если мы запустим ее, то компьютер выдаст "**Нет**", что означает — результат равен **Ложь**.

Как он посчитал это выражение?

1. Компьютер увидел команду **Сообщить**.
2. Посмотрел, что за параметр мы передаем этой команде.
3. Увидел, что в качестве параметра указано логическое выражение $2 * (10 + 10) > 50$.
4. Стал вычислять результат логического выражения.
5. Увидел, что выражение состоит из левой части $2 * (10 + 10)$, правой части 50 и знака больше между ними.
6. Посчитал результат левой части и выяснил, что он равен сорока.
7. Снова взглянул на выражение, только в уже упрощенном виде $40 > 50$, а так как 40, конечно же, меньше 50, то он вывел "Нет".

Задание №15. Вычислите все примеры логических выражений из предыдущего задания на компьютере.

Например, выражение "*Венера*" = "*Юпитер*", значение которого, как мы выяснили, равно *Ложь*.

Сообщить("Венера" = "Юпитер");

Компьютер при запуске выводит значение "Нет", что означает: результат действительно равен *Ложь* и мы совершенно верно вычислили его сами.

Решение

Сообщить("Венера" = "Юпитер"); // Нет

Сообщить("Венера" <> "Юпитер"); // Да

Сообщить(123 = 321); // Нет

Сообщить(123 < 321); // Да

Сообщить(123 <= 321); // Да

Сообщить(123 <= 123); // Да

Сообщить('20000101' <> '20140101'); // Да

Сообщить('20000101' = '20000101'); // Да

Сообщить('20140101' > '20120101'); // Да

Сообщить(25 + 25 = 50); // Да

Сообщить(2 * (10 + 10) > 50); // Нет

Сообщить("Юрий" + "Гагарин" = "Юрий Гагарин"); // Да

ЮНИТ 6. СЛОЖНЫЕ ЛОГИЧЕСКИЕ ВЫРАЖЕНИЯ В ЯЗЫКЕ 1С

В пятом юните мы познакомились с *логическим* типом данных, который может принимать одно из двух значений *Истина* или *Ложь*. Также мы узнали о существовании простейших логических выражений. Они состоят из двух данных (число, строка или дата) и операции сравнения между ними. Результатом логического выражения является *Истина* или *Ложь*.

Из простых логических выражений можно составлять сложные логические выражения.

Для этого существуют специальные логические операции: *И*, *Или*, *Не*.

Правила работы логической операции И

А (левая часть)	Б (правая часть)	А И Б	Пример
Истина	Истина	Истина	(2 < 3) И (10 = 10)
Истина	Ложь	Ложь	(2 < 3) И (10 <> 10)
Ложь	Истина	Ложь	(10 <> 10) И (2 < 3)
Ложь	Ложь	Ложь	(1 = 2) И (3 = 5)

Таким образом, логическое выражение, составленное при помощи операции *И*, верно тогда и только тогда, когда *верны оба выражения*, стоящие слева и справа от этой операции.

Для лучшего понимания представьте, что значение **Истина** это **1**, **Ложь** это **0**, а логическая операция **И** это **умножение**:

А (левая часть)	Б (правая часть)	А И Б
1	1	1
1	0	0
0	1	0
0	0	0

Правила работы логической операции *Или*

А (левая часть)	Б (правая часть)	А Или Б	Пример
Истина	Истина	Истина	$(2 < 3)$ Или $(10 = 10)$
Истина	Ложь	Истина	$(2 < 3)$ Или $(10 <> 10)$
Ложь	Истина	Истина	$(10 <> 10)$ Или $(2 < 3)$
Ложь	Ложь	Ложь	$(1 = 2)$ Или $(3 = 5)$

Таким образом, логическое выражение, составленное при помощи операции *Или*, истинно, если *верно хотя бы одно из выражений*, стоящих слева и справа от этой операции.

Для лучшего понимания представьте, что значение **Истина** это **1**, **Ложь** это **0**, а логическая операция **Или** это **сложение**:

А (левая часть)	Б (правая часть)	А Или Б
1	1	1
1	0	1
0	1	1
0	0	0

Правила работы логической операции *Не*

А	Не А	Пример
Истина	Ложь	Не $(2 < 3)$
Ложь	Истина	Не $(10 = 5)$

Таким образом, логическое выражение, составленное при помощи операции *Не*, верно тогда и только тогда, когда неверно выражение, стоящее справа от этой операции. Операцию *Не* еще называют *логическим отрицанием*.

Для лучшего понимания представьте, что значение **Истина** это **1**, а **Ложь** это **0**:

A	Не A
1	0
0	1

Задание №16. Вычислите (пока без помощи компьютера) результат следующих логических выражений.

1. $(1 = 1)$ И $(2 = 2)$
2. Не $(10 > 5)$
3. ("Солнце" = "Луна") Или ("Юпитер" = "Юпитер")
4. Не ("Солнце" = "Луна")
5. $(10 <> 5)$ Или $(5 = 5)$
6. (Истина) Или (Ложь)
7. Не (Истина)
8. (Ложь) И (Истина)
9. (Ложь) Или (Истина)

Ответы

1. $(1 = 1)$ И $(2 = 2)$

Давайте упростим:

$1 = 1$, это Истина

$2 = 2$, это Истина

значит $(1 = 1)$ И $(2 = 2)$ равнозначно (Истина) И (Истина),

а "Истина И Истина", как мы знаем из правил для операции И, равно Истина.

2. Не $(10 > 5)$

Упрощаем:

$10 > 5$, это Истина

значит Не $(10 > 5)$ равнозначно Не (Истина),

а "Не Истина", как мы знаем из правил для операции Не, равно Ложь.

3. ("Солнце" = "Луна") Или ("Юпитер" = "Юпитер")

Упрощаем:

"Солнце" = "Луна", это Ложь

"Юпитер" = "Юпитер", это Истина

значит ("Солнце" = "Луна") Или ("Юпитер" = "Юпитер") равнозначно (Ложь) Или (Истина)

а "Ложь Или Истина", как мы знаем из правил для операции Или, равно Истина.

4. Не ("Солнце" = "Луна")

Упрощаем:

"Солнце" = "Луна", это Ложь

значит Не ("Солнце" = "Луна") равнозначно Не (Ложь)

а "Не Ложь", как мы знаем из правил для операции Не, равно Истина.

5. $(10 <> 5)$ Или $(5 = 5)$

Упрощаем:

$10 <> 5$, это Истина

$5 = 5$, это Истина

значит $(10 <> 5)$ Или $(5 = 5)$ равнозначно (Истина) Или (Истина)

а "Истина Или Истина", как мы знаем из правил для операции Или, равно Истина.

6. (Истина) Или (Ложь)

Выражение уже упрощено.

"Истина Или Ложь" дают, в соответствии с правилами для операции Или, Истину.

7. Не (Истина)

Выражение уже упрощено.

"Не Истина" дают, в соответствии с правилами для операции Не, Ложь.

8. (Ложь) И (Истина)

Выражение уже упрощено.

"Ложь И Истина" дают, в соответствии с правилами для операции И, Ложь.

9. (Ложь) Или (Истина)

Выражение уже упрощено.

"Ложь Или Истина" дают, в соответствии с правилами для операции Или, Истина.

Задание №17. Вычислите все примеры логических выражений из предыдущего задания на компьютере.

Решение.

Сообщить($(1 = 1)$ И $(2 = 2)$); // Да

Сообщить(Не $(10 > 5)$); // Нет

Сообщить(("Солнце" = "Луна") Или ("Юпитер" = "Юпитер")); // Да

Сообщить(Не ("Солнце" = "Луна")); // Да

Сообщить((10 <> 5) Или (5 = 5)); // Да
Сообщить((Истина) Или (Ложь)); // Да
Сообщить(Не (Истина)); // Нет
Сообщить((Ложь) И (Истина)); // Нет
Сообщить((Ложь) Или (Истина)); // Да

ЮНИТ 7. УСЛОВНАЯ КОМАНДА В ЯЗЫКЕ 1С

В шестом юните мы познакомились со сложными логическими выражениями, которые образуются из простых при помощи логических операций *И*, *Или*, *Не*.

В этом юните мы разберемся, зачем эти логические выражения вообще нужны и научимся применять их на практике.

А понадобятся они нам при изучении новой команды языка программирования 1С, которая называется *Условная команда*.

Простая условная команда

Если коротко, то правила и суть этой команды можно представить так:

Если(логическое выражение)Тогда

Различные команды;

КонецЕсли

Более подробно:

1. Частью условной команды являются следующие три слова: *Если*, *Тогда* и *КонецЕсли*.

2. Между словами *Если* и *Тогда* всегда находится некоторое *логическое выражение*, которое принимает значение *Истина* или *Ложь*.

3. Между словами *Тогда* и *КонецЕсли* может находиться любое количество команд компьютеру (например, *Сообщить*, *ВвестиЧисло*, *ОткрытьЗначение* и другие).

4. И, наконец, **главное**: команды компьютеру, заключенные между словами *Тогда* и *КонецЕсли* выполняются только в том случае, если *логическое выражение*, заключенное между *Если* и *Тогда* принимает значение *Истина*.

Примеры использования условной команды с подробным объяснением.

Пример №1.

ЧислоОтПользователя = 0;

ВвестиЧисло(ЧислоОтПользователя);

Если (ЧислоОтПользователя < 0) Тогда

ОткрытьЗначение("Вы ввели отрицательное число.");

КонецЕсли;

Что делает этот код?

Первые две строчки вам должны быть знакомы по прошлым юнитам.

Дальше следует условная команда, условием которой является логическое выражение ($\text{ЧислоОтПользователя} < 0$). Если число, введенное пользователем, действительно меньше нуля, то это выражение принимает значение Истина, иначе Ложь.

Между словами Тогда и КонецЕсли находится всего одна команда *ОткрытьЗначение*, которая, как следует из правила для условной команды, выполнится только в том случае, если логическое выражение окажется верным. А значит, сообщение для пользователя "Вы ввели отрицательное число" выведется только если значение, введенное пользователем, окажется отрицательным. Иначе ничего не выведется. Что и требуется логикой программы.

Пример №2.

ИмяПользователя = "";

ВвестиСтроку(ИмяПользователя);

Если (ИмяПользователя = "Александр") Тогда

ОткрытьЗначение("Вас зовут Александр!");

КонецЕсли;

В этом примере мы вводим имя пользователя и, если оно равно "Александр", то выводим сообщение "Вас зовут Александр!".

Пример №3.

ИмяПользователя = "";

ВвестиСтроку(ИмяПользователя);

Если (ИмяПользователя <> "Александр") Тогда

ОткрытьЗначение("Ваше имя точно не Александр!");

ОткрытьЗначение("На самом деле, вас зовут " +

ИмяПользователя);

КонецЕсли;

В этом примере мы вводим имя пользователя и, если оно не равно "Александр", то выводим сообщение "Ваше имя точно не Александр!" и следом выводим настоящее имя пользователя.

Пример №4.

ЧислоОтПользователя = 0;

ВвестиЧисло(ЧислоОтПользователя);

Если (ЧислоОтПользователя > 0) И

(ЧислоОтПользователя < 10) Тогда

ОткрытьЗначение("Вы ввели число между 0 и 10.");

КонецЕсли;

Отличие этого примера лишь в том, что тут используется сложное логическое выражение, составленное при помощи логической команды *И*.

Таким образом, сообщение "Вы ввели число между 0 и 10." будет выведено пользователю тогда и только тогда, когда введенное число одновременно и больше нуля и меньше десяти.

Расширенная форма условной команды

Если(логическое выражение)**Тогда**

Различные команды; // Эти команды выполняются,
 // если (логическое выражение)=Истина

Иначе

Различные команды; // Эти команды выполняются,
 // если (логическое выражение)=Ложь

КонецЕсли

Разберем расширенную форму условной команды на примере.

Пример №5.

ЧислоОтПользователя = 0;

ВвестиЧисло(ЧислоОтПользователя);

Если (ЧислоОтПользователя >= 0) Тогда

 ОткрытьЗначение("Вы ввели положительное число.");

Иначе

 ОткрытьЗначение("Вы ввели отрицательное число.");

КонецЕсли;

Пользователь вводит число. И в случае, если введенное значение больше или равно нулю, программа выводит строку *"Вы ввели положительное число"*, иначе *"Вы ввели отрицательное число"*.

Задание №18. Напишите программу, которая вводит от пользователя два числа. Если первое число больше или равно второму, то выводит *"Первое число больше или равно второму"*, иначе *"Второе число больше первого"*.

Решение.

ПервоеЧисло = 0;

ВвестиЧисло(ПервоеЧисло);

ВтороеЧисло = 0;

ВвестиЧисло(ВтороеЧисло);

Если ПервоеЧисло >= ВтороеЧисло Тогда

 ОткрытьЗначение("Первое число больше или равно второму");

Иначе

 ОткрытьЗначение("Второе число больше первого");

КонецЕсли;

Задание №19 Напишите программу, которая вводит от пользователя два числа. Если оба числа положительные, то выводит сумму этих чисел, иначе выводит произведение этих чисел.

Решение.

ПервоеЧисло = 0;

ВвестиЧисло(ПервоеЧисло);

```
ВтороеЧисло = 0;  
ВвестиЧисло(ВтороеЧисло);  
Если ПервоеЧисло > 0 И ВтороеЧисло > 0 Тогда  
    ОткрытьЗначение(ПервоеЧисло + ВтороеЧисло);  
Иначе  
    ОткрытьЗначение(ПервоеЧисло * ВтороеЧисло);  
КонецЕсли;
```

Более сложные конструкции условной команды представлены ниже в виде следующих программ:

Пример сложной конструкции "Если" в 1с

```
Если А>Б Тогда  
    // блок операторов  
ИначеЕсли Б>С Тогда  
    // блок операторов  
Иначе  
    // блок операторов  
КонецЕсли;
```

Конструкция тернарной команды "?" в 1с

?(<Логическое выражение>, <выражение 1>, <выражение 2>);

Тернарная команда содержит три параметра. Первый параметр — логическое выражение. На втором и третьем местах — выражения любых типов. Выполнение данной команды начинается с вычисления логического выражения. Если результат вычисления — "Истина", то далее вычисляется выражение 1, в противном случае — выражение 2).

ЮНИТ 8. ЦИКЛ "ДЛЯ" В ЯЗЫКЕ 1С

Цикл — это специальная команда компьютеру, которая позволяет повторять выполнение других команд нужное количество раз.

К примеру, пусть нам требуется вывести на экран числа от 1 до 10.

Можно, конечно, решить задачу уже известным нам способом — без применения циклов. Решение будет таким:

```
Сообщить(1);  
Сообщить(2);  
Сообщить(3);  
Сообщить(4);  
Сообщить(5);
```



```
Сообщить(6);  
Сообщить(7);  
Сообщить(8);  
Сообщить(9);  
Сообщить(10);
```

Но, согласитесь, это громоздко и неудобно. Чтобы решить такую простую задачу, нам понадобилось написать целых 10 строчек кода! К тому же почти одинаковых. Они отличаются лишь параметрами для команды *Сообщить*.

Вот как раз для таких случаев и были придуманы циклы, которых в языке 1С бывает два вида: *Цикл Для* и *Цикл Пока*.

Цикл Для

Конструкция цикла выглядит вот так:

```
Для Имя = НачальноеЧисло По КонечноеЧисло Цикл  
  Команда1();  
  Команда2();  
  ...  
КонецЦикла;
```

Команды, заключенные между словами *Цикл* и *КонецЦикла*, выполняются столько раз, сколько нужно шагов, чтобы *НачальноеЧисло* стало больше *КонечногоЧисла*.

При этом *Имя* программист придумывает сам, и с этим именем связывается текущее значение шага. Разберем эти действия на примере.

Решение задачи вывода чисел от 1 до 10 с применением только что изученного цикла будет выглядеть так:

```
Для Шаг = 1 По 10 Цикл  
  Сообщить(Шаг);  
КонецЦикла;  
Всего три строчки вместо десяти.
```

Компьютер, встретив такую конструкцию, выполняет ее в следующей последовательности:

Компьютер инициализирует имя Шаг значением равным 1.

Затем сравнивает это значение с 10. Значение 1 меньше 10.

Выполняет Сообщить(1);

Увеличивает Шаг на единицу. Сравнивает с 10. Значение 2 меньше 10. Выполняет Сообщить(2);

Увеличивает Шаг на единицу. Сравнивает с 10. Значение 3 меньше 10. Выполняет Сообщить(3);

Увеличивает Шаг на единицу. Сравнивает с 10. Значение 4 меньше 10. Выполняет Сообщить(4);

Увеличивает Шаг на единицу. Сравнивает с 10. Значение 5 меньше 10. Выполняет Сообщить(5);

Увеличивает Шаг на единицу. Сравнивает с 10. Значение 6 меньше 10. Выполняет Сообщить(6);

Увеличивает Шаг на единицу. Сравнивает с 10. Значение 7 меньше 10. Выполняет Сообщить(7);

Увеличивает Шаг на единицу. Сравнивает с 10. Значение 8 меньше 10. Выполняет Сообщить(8);

Увеличивает Шаг на единицу. Сравнивает с 10. Значение 9 меньше 10. Выполняет Сообщить(9);

Увеличивает Шаг на единицу. Сравнивает с 10. Значение равно 10. Выполняет Сообщить(10);

Увеличивает Шаг на единицу. Сравнивает с 10.

Стоп! 11 больше 10. Прекращает выполнение цикла.

Задание №20. Измените приведенный выше пример, чтобы выводились числа от 10 до 100.

Ответ

Для Шаг = 10 По 100 Цикл

Сообщить(Шаг);

КонецЦикла;

А как вы думаете, сколько раз выполнится следующий цикл:

Для Шаг = 10 По 1 Цикл

Сообщить("Привет, Мир!");

КонецЦикла;

Ответ

Конечно же, этот цикл не выполнится ни разу. Ведь 10 уже больше 1.

А как насчет такого примера:

Для Шаг = 1 По 1 Цикл

Сообщить("Привет, Мир!");

КонецЦикла;

Сколько раз выполнится *Сообщить("Привет, Мир!")*?

Ответ

Цикл (вернее команда Сообщить, которая находится внутри цикла) выполнится один раз. Ведь начальное значение (единица) равно конечному значению (единица) и компьютеру понадобится ровно один шаг, чтобы начальное значение цикла стало больше конечного.

Задание №21. Напишите программу, которая выводит на экран числа от 10 до 1, используя *Цикл Для*. Программа должна занимать ровно три строчки.

Решение

```
Для Шаг = 1 По 10 Цикл
  Сообщить(11 - Шаг);
КонецЦикла;
```

Задание №22. Помните, что мы в третье юните выводили таблицу умножения для числа 7? Она заняла у нас целых 10 строчек. Теперь, когда вы знаете, что такое циклы, перепишите решение этой задачи с использованием *Цикла Для*. Решение должно уместиться в три строчки.

Решение

```
Для Шаг = 1 По 10 Цикл
  Сообщить("7 * " + Шаг + " = " + 7 * Шаг);
КонецЦикла;
```

ЮНИТ 9. ЦИКЛ "ПОКА" В ЯЗЫКЕ 1С

Конструкция цикла выглядит так:

```
Пока ЛогическоеВыражение Цикл
  Команда1();
  Команда2();
  ...
КонецЦикла;
```

Команды, заключенные между словами *Цикл* и *КонецЦикла* выполняются пока *ЛогическоеВыражение* равно *Истине*.

В качестве логического выражения может быть любое условие. Пока это условие верно (то есть равно *Истина*), выполнение команд в цикле повторяется снова и снова, как только условие перестало быть верным, команды перестают выполняться.

Решение задачи вывода чисел от 1 до 10 с применением только что изученного цикла будет выглядеть так:

```
Шаг = 1;
Пока Шаг <= 10 Цикл
  Сообщить(Шаг);
  Шаг = Шаг + 1;
КонецЦикла;
```

Компьютер, встретив такую конструкцию, выполняет ее так:

Первой командой компьютер сопоставит имени *Шаг* значение равное 1. Так как имя *Шаг* на данном этапе равно единице — условие цикла верно ($Шаг \leq 10$) — компьютер выводит единицу и увеличивает имя *Шаг* до 2. Имя *Шаг* равно 2, но условие цикла по-прежнему верно, поэтому компьютер выводит 2 и увеличивает имя *Шаг* до 3.

...

Имя *Шаг* равно 10, условие цикла все еще верно — компьютер выводит 10 и увеличивает имя *Шаг* до 11. Стоп! Условие цикла перестало выполняться, теперь *Шаг* (вернее его значение 11) больше 10. Команды цикла больше не выполняются.

Задание №23. Измените приведенный выше пример, чтобы выводились числа от 10 до 100.

Ответ

Шаг = 10;

Пока Шаг <= 100 Цикл

Сообщить(Шаг);

Шаг = Шаг + 1;

КонецЦикла;

Все примеры из заданий нужно обязательно выполнять на компьютере.

А как вы думаете сколько раз выполнятся следующие циклы:

Пока Истина Цикл

Сообщить("Привет!");

КонецЦикла;

Пока 1 = 1 Цикл

Сообщить("Привет!");

КонецЦикла;

Пока "Мир" = "Мир" Цикл

Сообщить("Привет!");

КонецЦикла;

Это так называемые бесконечные циклы, которые, начав выполняться, никогда не закончатся, потому что их логическое условие всегда равно *Истина*.

А вот следующие примеры циклов:

Пока Ложь Цикл

Сообщить("Прощай!");

КонецЦикла;

Пока 1 = 2 Цикл

Сообщить("Прощай!");

КонецЦикла;

Пока "Мир" = "Земля" Цикл

Сообщить("Прощай!");

КонецЦикла;

являются примерами бесполезных. Они никогда не выполнятся, потому что их логическое условие всегда равно *Ложь*.

Задание №24. Напишите программу, которая выводит на экран числа от 10 до 1, используя *Цикл Пока*. Программа должна занимать ровно пять строк.

Решение

```
Шаг = 10;  
Пока Шаг >= 1 Цикл  
    Сообщить(Шаг);  
    Шаг = Шаг - 1;  
КонецЦикла;
```

Задание №25. Напишите вывод таблицы умножения для числа 7, используя *Цикл Пока*.

Решение

```
Шаг = 1;  
Пока Шаг <= 10 Цикл  
    Сообщить("7 * " + Шаг + " = " + 7 * Шаг);  
    Шаг = Шаг + 1;  
КонецЦикла;
```

Задание №26. Перепишите предыдущее задание так, чтобы вывод таблицы умножения происходил в обратном порядке. То есть сначала $7 * 10$, потом $7 * 9$ и так далее.

Решение

```
Шаг = 10;  
Пока Шаг >= 1 Цикл  
    Сообщить("7 * " + Шаг + " = " + 7 * Шаг);  
    Шаг = Шаг - 1;  
КонецЦикла;
```

ЮНИТ 10. КОМБИНИРОВАНИЕ ПРОСТЫХ КОНСТРУКЦИЙ В ЯЗЫКЕ 1С

Мы научились использовать имена, изучили условную команду, узнали два вида циклов.

Все это были достаточно простые программы, в каждой из которых использовалась какая-то одна основная конструкция языка (или условная команда или цикл). В реальных программах, конечно же, так не бывает. В них обычно используется все сразу и по несколько раз. Необходимо научиться комбинировать простые конструкции языка 1С вместе.

Комбинирование простых конструкций языка 1С

К примеру, пусть у нас есть два цикла:

Для Шаг1 = 1 По 2 Цикл

Сообщить(Шаг1);

КонецЦикла;

Для Шаг2 = 1 По 3 Цикл

Сообщить(Шаг2);

КонецЦикла;

Эти циклы хорошо нам знакомы и мы понимаем, что в результате первого цикла выведутся числа от 1 до 2, а в результате второго — от 1 до 3.

А что если мы вложим выполнение одного цикла внутрь другого? Вот так:

Для Шаг1 = 1 По 2 Цикл

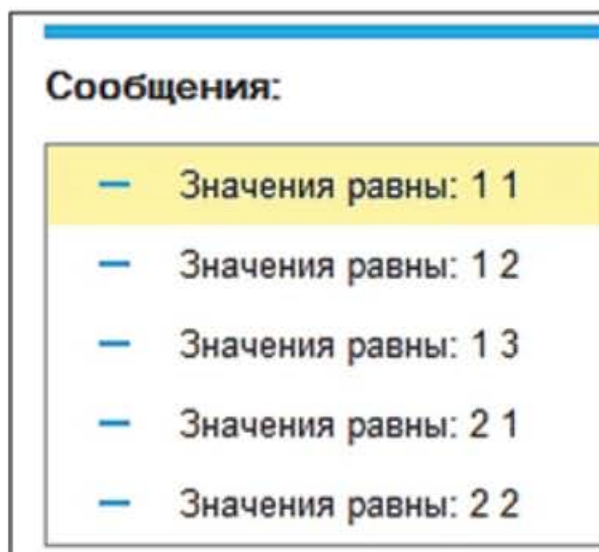
Для Шаг2 = 1 По 3 Цикл

Сообщить("Значения равны: " + Шаг1 + " " + Шаг2);

КонецЦикла;

КонецЦикла;

Если запустить этот пример на компьютере, то получатся следующие результаты:



Как видим, результаты отличаются от выполнения данных циклов по отдельности. И вот почему.

У нас есть внешний цикл от 1 до 2. Это значит, что все, что находится у него между словами *Цикл* и *КонецЦикла* выполнится 2 раза. А у него между этими словами находится другой цикл от 1 до 3, который внутри себя содержит команду *Сообщить*.

Итого такая вложенность приводит к тому, что команда *Сообщить* выполняется 6 раз. Шесть, потому что $2 * 3 = 6$.

Теперь обратите внимание, как меняется значение *Шаг1* и *Шаг2*. В начале *Шаг1* равен 1, а *Шаг2* меняется от 1 до 3. Затем *Шаг1* уже равен 2 (следующее значение цикла) и снова *Шаг2* меняется от 1 до 3.

Задание №27. По аналогии с предыдущим примером напишите тройной вложенный цикл с выводом *Шаг1*, *Шаг2* и *Шаг3* соответственно от 1 до 10, от 10 до 20 и от 20 до 30.

Решение

Для Шаг1 = 1 По 10 Цикл

Для Шаг2 = 10 По 20 Цикл

Для Шаг3 = 20 По 30 Цикл

Сообщить("Значения равны: " + Шаг1 + " " + Шаг2 + " " + Шаг3);

КонецЦикла;

КонецЦикла;

КонецЦикла;

Задание №28. Найдите все пары целых (в том числе одинаковых) чисел величиной от 1 до 9 каждое, сумма которых равна 10.

Решение

Для Шаг1 = 1 По 9 Цикл

Для Шаг2 = 1 По 9 Цикл

Если Шаг1 + Шаг2 = 10 Тогда

Сообщить(" " + Шаг1 + " + " + Шаг2 + " = 10");

КонецЕсли;

КонецЦикла;

КонецЦикла;

Мы перебираем одно число в цикле от 1 до 9 (*Шаг1*). И второе число в другом цикле от 1 до 9 (*Шаг2*). А так как числа для сложения нам нужны одновременно, мы делаем второй цикл вложенным в первый. Осталось воспользоваться условной конструкцией, чтобы отобрать только такие числа, сумма которых равна 10.

Задание №29. Напишите программу, которая найдет все варианты двух чисел от 1 до 30, произведение которых равно 30.

Как всегда, выполните программу на компьютере и убедитесь, что она выдает верные результаты.

Ответ

Для Шаг1 = 1 По 30 Цикл

Для Шаг2 = 1 По 30 Цикл

Если Шаг1 * Шаг2 = 30 Тогда

Сообщить(" " + Шаг1 + " * " + Шаг2 + " = " + 30);

КонецЕсли;

КонецЦикла;

КонецЦикла;

ЮНИТ 11. МАССИВЫ В ЯЗЫКЕ 1С

В этом юните мы познакомимся с новым типом данных языка 1С, который называется *Массив*.

Что такое массивы и зачем они вообще нужны программисту?

Давайте представим, что у нас есть 4 разных числа. К примеру: 25, 84, 31 и 96. Если бы мы захотели использовать их в своей программе, то нужно было бы дать имя каждому из чисел. А что, если хранить их все вместе, под одним общим именем, к примеру, *СлучайныеЧисла*. И обращаться к ним как *СлучайныеЧисла0*, *СлучайныеЧисла1*, *СлучайныеЧисла2* и так далее.

Значений много, а имя одно. И чтобы получить конкретное значение мы бы указывали имя и порядковый номер (**начиная с нуля**). Это и был бы массив.

Еще полезно представлять себе массив как шкаф, у которого множество нумерованных ящиков (**начиная с нуля**). Шкаф — это массив, а ящики — это номера. Содержимое ящиков — элементы массива. Доступ к содержимому конкретного ящика осуществляется по имени шкафа и номеру ящика.

В языке 1С это будет выглядеть так:

```
СлучайныеЧисла = Новый Массив(4); // завели массив для  
// хранения 4 чисел
```

```
СлучайныеЧисла[0] = 25; // поместили в ящик №0 первое  
// число
```

```
СлучайныеЧисла[1] = 84; // поместили в ящик №1 второе  
// число
```

```
СлучайныеЧисла[2] = 31; // и так далее
```

```
СлучайныеЧисла[3] = 96;
```

В этом примере в первый раз использованы пояснения прямо в коде. Такие пояснения называются **комментариями**. Они отделяются от кода программы двумя косыми чертами и нужны только для программиста — компьютер их игнорирует.

В массивах можно хранить значения разных типов, а не только числа:

```
РазличныеЗначения = Новый Массив(3); // массив для  
// хранения 3 значений
```

```
РазличныеЗначения[0] = 100; // поместили в ящик №0  
// число
```

```
РазличныеЗначения[1] = "Солнечный день."; // поместили  
// в ящик №1 строку
```

```
РазличныеЗначения[2] = '19991231'; // поместили в ящик  
// №2 дату
```

В ящики массива значения можно не только класть, но и доставать:


```

Сообщить(РазличныеЗначения[0]); // выведем содержи-
// мое ящика №0
Сообщить(РазличныеЗначения[1]); // выведем содержи
// мое ящика №1
Сообщить(РазличныеЗначения[2]); // выведем содержи
// мое ящика №2

```

Преимущество использования массивов состоит именно в том, что доступ к их значениям осуществляется через числовой номер, через индекс. А значит, мы легко можем перебрать все значения массива в цикле:

```

Для Шаг = 0 По 2 Цикл // делаем цикл от 0 до 2
    Сообщить(РазличныеЗначения[Шаг]); //обращаемся
// к ящику массива по номеру
КонецЦикла;

```

Задание №30. Необходимо ввести от пользователя 5 чисел, найти их сумму и вывести результат.

Давайте решим эту задачу двумя способами.

Сначала без использования массива:

```

Число1 = 0;
Число2 = 0;
Число3 = 0;
Число4 = 0;
Число5 = 0;
ВвестиЧисло(Число1);
ВвестиЧисло(Число2);
ВвестиЧисло(Число3);
ВвестиЧисло(Число4);
ВвестиЧисло(Число5);
СуммаЧисел = Число1 + Число2 + Число3 + Число4 + Число5;
ОткрытьЗначение("Сумма чисел равна " + СуммаЧисел);

```

А теперь с использованием массива:

```

Числа = Новый Массив(5); // объявили массив с 5 ящиками
Для Шаг = 0 По 4 Цикл
    ВвестиЧисло(Числа[Шаг]); //в каждый ящик вводим число
КонецЦикла
СуммаЧисел = 0; // в этом имени будем накапливать
// сумму чисел
Для Шаг = 0 По 4 Цикл // пробегаемся циклом от 0 до 4
    СуммаЧисел = СуммаЧисел + Числа[Шаг]; // суммируем числа
// из массива
КонецЦикла;
ОткрытьЗначение("Сумма чисел равна " + СуммаЧисел);

```

Вроде бы сэкономили всего три строки. А если бы нужно было ввести не 5, а 100 чисел?

Задание №31. Переделайте приведенный выше пример, чтобы вводилось 10 чисел и находилось произведение.

Решение

```
Числа = Новый Массив(10);
Для Шаг = 0 По 9 Цикл
    ВвестиЧисло(Числа[Шаг]);
КонецЦикла;
ПроизведениеЧисел = 1;
Для Шаг = 0 По 9 Цикл
    ПроизведениеЧисел = ПроизведениеЧисел * Числа[Шаг];
КонецЦикла;
ОткрытьЗначение("Произведение чисел равно " + ПроизведениеЧисел);
```

При помощи массивов можно решать достаточно сложные задачи.

Задание №32. Необходимо ввести от пользователя 5 чисел, найти наименьшее из них и вывести его.

Решение

```
МассивЧисел = Новый Массив(5); // объявим массив на 5
                                // ящиков

Для Шаг = 0 По 4 Цикл // пробежимся от 0 до 4
    ВвестиЧисло(МассивЧисел[Шаг]); // в каждый ящик
                                // положим число
КонецЦикла;

// изначально будем считать, что первый элемент самый
                                // маленький
НаименьшееЧисло = МассивЧисел[0];

Для Шаг = 0 По 4 Цикл // пробегаемся по всем значениям
                                // массива
// если в массиве есть число меньше, чем НаименьшееЧисло
    Если МассивЧисел[Шаг] < НаименьшееЧисло Тогда
        // тогда его и делаем новым наименьшим числом
        НаименьшееЧисло = МассивЧисел[Шаг];
    КонецЕсли;
КонецЦикла;
ОткрытьЗначение("Наименьшее из введенных чисел равно " +
НаименьшееЧисло);
```

Разберитесь с этим примером и выполните его на компьютере для различных значений.

Обратите внимание на то, что во всех примерах выше мы создаем массив сразу с нужным количеством ящиков. Например,

```
МассивНа5Ящиков = Новый Массив(5);
```

Это означает, что у этого массива есть ящики под номерами 0, 1, 2, 3, 4.

Если необходимо объявить пустой массив:

```
ДругойМассив = Новый Массив; //мы не указали никакого  
// числа в скобках
```

Если мы сразу после объявления пустого массива попытаемся обратиться к одному из его ящиков, то получим ошибку:

```
Сообщить(ДругойМассив[0]); // ОШИБКА!!! Массив пока  
// не содержит ящиков!
```

В пустой массив ящики нужно сначала добавить, причем сразу указывая их значения:

```
ДругойМассив.Добавить(123); // первый ящик  
ДругойМассив.Добавить(456); // второй ящик  
ДругойМассив.Добавить(789); // третий ящик
```

Только теперь мы можем обращаться к этим ящикам по их номерам (помните, что в 1С массивы нумеруются с нуля):

```
Сообщить(ДругойМассив[0]); // выведет 123  
Сообщить(ДругойМассив[1]); // выведет 456  
Сообщить(ДругойМассив[2]); // выведет 789
```

Для того, чтобы в любой момент узнать количество ящиков в массиве, воспользуемся командой *Количество*. Ее нужно вызывать через точку после имени массива:

```
Сообщить(ДругойМассив.Количество()); // выведет 3
```

Задание №33. Необходимо ввести от пользователя 7 чисел, найти наибольшее из них и вывести его. Задание решается по аналогии с предыдущим.

Решение

```
// объявим массив на 7 элементов
МассивЧисел = Новый Массив(7);

// пробежимся по каждому элементу массива и введем туда
// число от пользователя
Для Шаг = 0 По 6 Цикл
    ВвестиЧисло(МассивЧисел[Шаг]);
КонецЦикла;
// изначально положим, что самым большим является
// первое число в массиве
НаибольшееЧисло = МассивЧисел[0];
// снова пробежимся по всем числам массива
Для Шаг = 0 По 6 Цикл
// если текущее число больше уже известного нам самого
// большого числа
    Если МассивЧисел[Шаг] > НаибольшееЧисло Тогда
        // тогда объявим текущее число как самое большое
        НаибольшееЧисло = МассивЧисел[Шаг];
    КонецЕсли;
КонецЦикла;

// выведем пользователю самое большое из всех чисел массива
ОткрытьЗначение("Наибольшее из введенных чисел равно " +
НаибольшееЧисло);
```

ЮНИТ 12. СТРУКТУРЫ В ЯЗЫКЕ 1С

Следующий тип данных, с которым нам предстоит познакомиться – это *Структура*.

Пусть нам требуется хранить в программе информацию о некоем человеке, которая включает в себя фамилию, имя и отчество. Как лучше представить эти данные?

Можно, конечно, так:

Фамилия = "Пупкин";

Имя = "Василий";

Отчество = "Алибабаевич";

В общем, хорошо, но это три разных объекта, которые никак не связаны между собой. А хочется, чтобы было ясно, что и фамилия, и имя, и отчество указаны не сами по себе, а принадлежат одному человеку. Возникает потребность связать эти три параметра (фамилия, имя и отчество) с одним объектом, который давайте назовем *Человек*.

Можно поместить все три параметра в массив:

Человек = Новый Массив(3);


```
Человек[0] = "Пупкин";  
Человек[1] = "Василий";  
Человек[2] = "Алибабаевич";
```

Вроде бы цель достигнута — связали все три параметра с одним объектом. Но работать с таким объектом неудобно, потому что обращаться к значениям массива придется по числовому индексу, то есть, чтобы нам вывести отчество, придется написать:

```
Сообщить(Человек[2]); // вывод отчества человека
```

Что, конечно же, очень неудобно! Хотелось бы обращаться к параметрам объекта по их именам. Вот для этого и были придуманы *Структуры*.

Структура — это составной объект, в который входят элементы любых типов, обращаться к которым можно по имени этих элементов.

С использованием структур предыдущий пример будет выглядеть так:

```
Человек = Новый Структура("Фамилия, Имя, Отчество");  
Человек.Фамилия = "Пупкин";  
Человек.Имя = "Василий";  
Человек.Отчество = "Алибабаевич";  
Сообщить(Человек.Отчество);
```

По приведенному выше примеру пройдите следующий тест:

Задание 34. Необходимо ввести имена и возраст трех людей. Затем найти, кто из них самый старший и вывести его имя.

Для хранения информации о человеке (имя и возраст) будем использовать структуру. Информацию о каждом человеке будем помещать в отдельный ящик массива. Затем пробежимся по ящикам массива и найдем имя самого старшего человека.

```
// объявляем массив с тремя ящиками  
Люди = Новый Массив(3);  
Для Шаг = 0 По 2 Цикл  
    // вводим информацию по каждому из трех людей  
    // и сохраняем ее в структуру, которую  
    // кладем в соответствующий ящик массива  
    Люди[Шаг] = Новый Структура("Имя, Возраст");  
    ВвестиСтроку(Люди[Шаг].Имя); // вводим имя  
    ВвестиЧисло(Люди[Шаг].Возраст); // вводим возраст  
КонецЦикла;  
// изначально положим, что самым старшим  
// является человек из первой ячейки массива  
ИмяСамогоСтаршегоЧеловека = Люди[0].Имя;
```

```

НаибольшийВозраст = Люди[0].Возраст;
Для Шаг = 0 По 2 Цикл // пробежимся по всем людям
    // и сравним их возраста с наибольшим
    // возрастом на текущий момент
    Если Люди[Шаг].Возраст > НаибольшийВозраст Тогда
        // нашли человека, который старше предыдущего
        // сохраним его возраст
        НаибольшийВозраст = Люди[Шаг].Возраст;
        // и имя
        ИмяСамогоСтаршегоЧеловека = Люди[Шаг].Имя;
    КонецЕсли;
КонецЦикла;
ОткрытьЗначение(
    "Самый старший человек - " +
    ИмяСамогоСтаршегоЧеловека + ". Ему " +
    НаибольшийВозраст + " лет.");

```

По аналогии с этим примером решите следующую задачу.

Задание №35. Необходимо ввести имена и вес четырех человек, найти самого легкого из них и вывести его имя.

Решение

```

// объявляем массив с четырьмя ящиками
Люди = Новый Массив(4);

Для Шаг = 0 По 3 Цикл
    // вводим информацию по каждому из четырех людей
    // и сохраняем ее в структуру, которую
    // кладем в соответствующую ячейку массива
    Люди[Шаг] = Новый Структура("Имя, Вес");
    ВвестиСтроку(Люди[Шаг].Имя); // вводим имя
    ВвестиЧисло(Люди[Шаг].Вес); // вводим вес
КонецЦикла;

// изначально положим, что самым легким
// является человек из первой ячейки массива
ИмяСамогоЛегкогоЧеловека = Люди[0].Имя;
НаименьшийВес = Люди[0].Вес;

Для Шаг = 0 По 3 Цикл // пробежимся по всем людям
    // и сравним их вес с наименьшим
    // весом на текущий момент
    Если Люди[Шаг].Вес < НаименьшийВес Тогда
        // нашли человека, который легче предыдущего
        // сохраним его вес

```

```

    НаименьшийВес = Люди[Шаг].Вес;
    // и имя
    ИмяСамогоЛегкогоЧеловека = Люди[Шаг].Имя;
    КонецЕсли;
    КонецЦикла;

    ОткрытьЗначение(
        "Самый легкий человек - " +
        ИмяСамогоЛегкогоЧеловека + ". Его вес " +
        НаименьшийВес + " кг.");

```

ЮНИТ 13. ФУНКЦИИ И ПРОЦЕДУРЫ В ЯЗЫКЕ 1С

В данном юните мы приступаем изучению того, без чего не может обойтись ни одна серьезная программа — функций и процедур.

Почему функции и процедуры столь полезны для программистов? Попробуем разобраться.

Функции

Пусть нам требуется написать программу, которая вычисляет произведение суммы и разности двух введенных чисел. Выглядеть она будет так:

```

А = 0;
ВвестиЧисло(А);
Б = 0;
ВвестиЧисло(Б);
Результат = (А + Б) * (А - Б);
ОткрытьЗначение(Результат);

```

В данном случае формула вычисления результата достаточно проста, но она могла бы быть гораздо сложнее. А что если нам нужно вычислять ее не один раз, а несколько? Причем в разных местах программы. В этом случае нам пришлось бы снова и снова писать код для ее вычисления:

```
Результат = (А + Б) * (А - Б);
```

Вот бы придумать такое имя, которое будет связано с этой формулой, и при обращении к нему мы будем обращаться ко всей этой формуле целиком.

Пусть этим именем будет *ПроизведениеСуммыИРазности*.

Получается, что теперь мы можем написать:

```
Результат = ПроизведениеСуммыИРазности;
```

И все? Нет, конечно! Ведь непонятно, произведение суммы и разности каких именно чисел нужно считать. Гораздо лучше будет передать эти числа нашему имени в качестве параметров, как мы обычно делаем при вызове команды:

```
Результат = ПроизведениеСуммыИРазности(А, Б);
```


Это так называемый вызов функции. Он выглядит точно так же, как и вызов многих других команд компьютера, которые мы уже неоднократно делали. Только это наша собственная команда, работу которой определяем мы, а не компьютер.

Давайте, наконец, определим нашу функцию, чтобы компьютер, встретив ее вызов, выполнил то, что мы хотим:

Функция ПроизведениеСуммыИРазности(А, В)

Результат = (А + В) * (А – В);

Возврат Результат;

КонецФункции

Что включает в себя определение этой функции?

Прежде всего ключевое слово *Функция*, следом за которым идет имя, которое мы придумали сами.

Затем следуют имена *параметров*, заключенные в круглые скобки. Параметры — это данные, которые мы передадим в нашу функцию при ее вызове. Она с ними что-то сделает и возвратит результат. Каждый параметр имеет свое имя, которое мы также придумываем сами. Это имя можно использовать только внутри функции.

Дальше идет *тело*. Это команды компьютеру, которые будут выполняться в тот момент, когда мы сделаем вызов нашей функции. Тело заканчивается ключевым словом *КонецФункции*.

Внутри функции могут выполняться абсолютно любые знакомые нам команды: условные операторы, циклы и прочее. Но хотя бы один раз внутри каждой функции должна присутствовать команда:

Возврат Результат;

Где вместо *Результат* может быть любое выражение, которое вернется из функции в качестве ее результата.

Мы можем вызывать функцию в программе столько раз, сколько нам потребуется (рис.18).

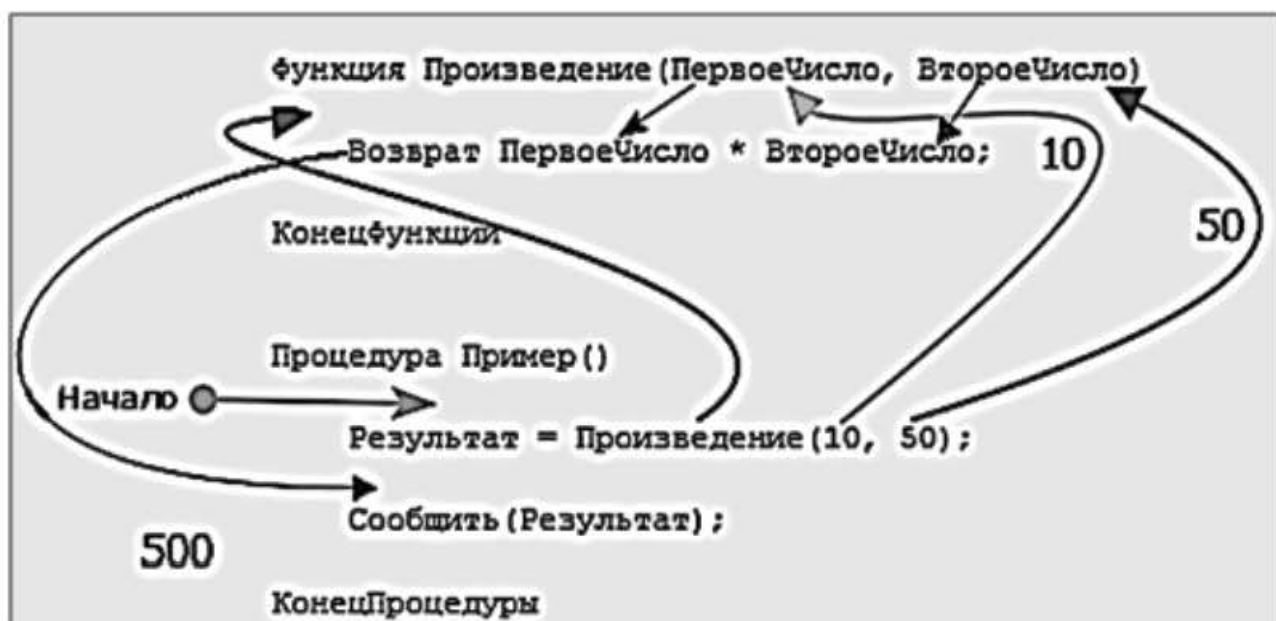


Рис. 18. Формирование и вызов функции

На рис. 19 показан пример формирования функции как объекта программы.

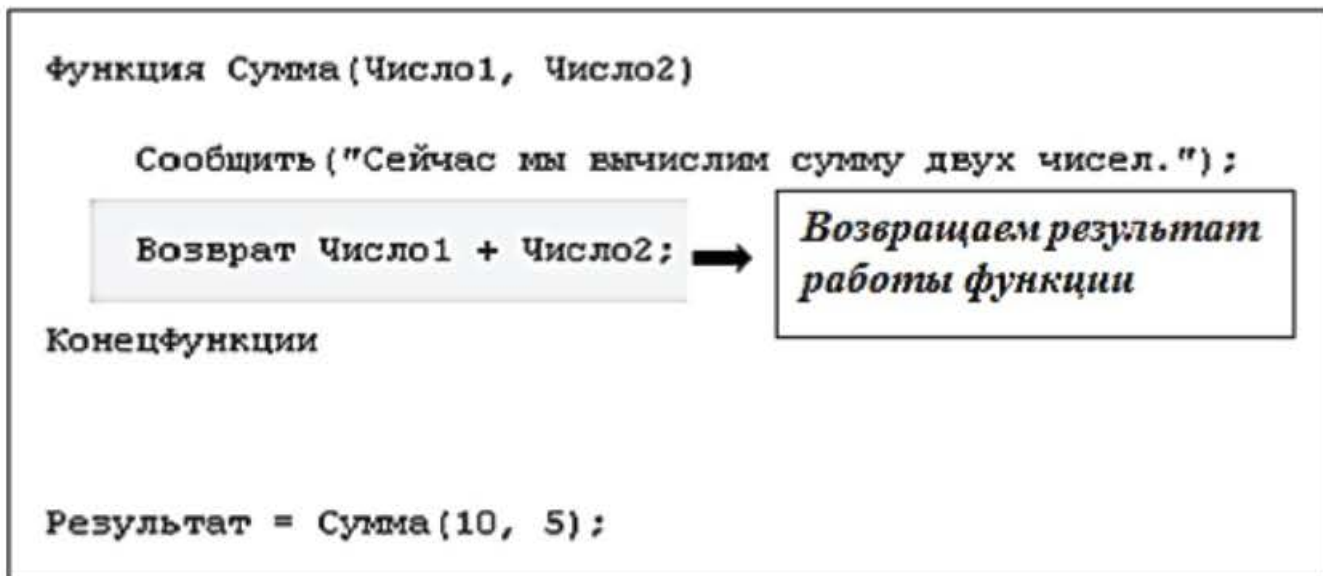


Рис. 19. Пример формирования и вызова функции как объекта программы.

Процедуры

Процедуры это те же самые функции, но они не возвращают результат и объявляются при помощи других ключевых слов: *Процедура* и *КонецПроцедуры* (рис. 20).

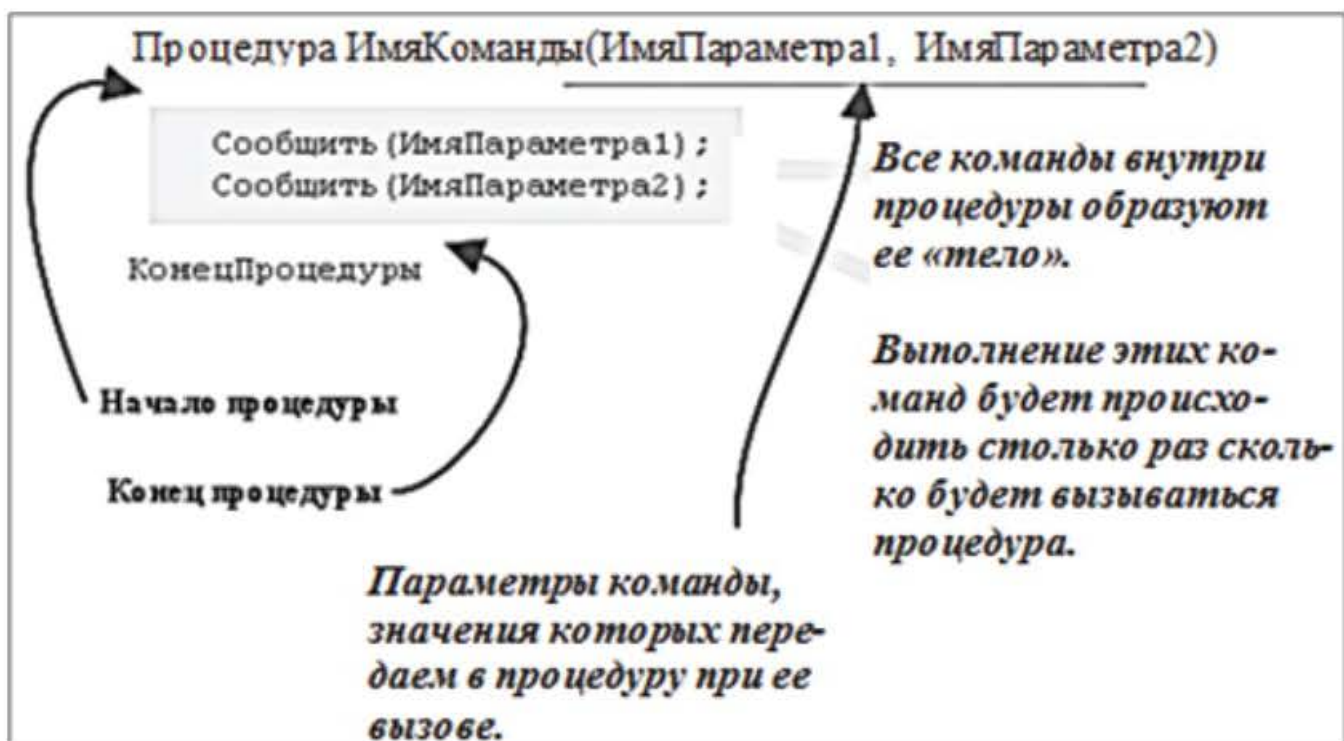


Рис. 20. Формирование процедуры

На рис. 21 показан пример формирования и запуска процедуры как объекта программы.

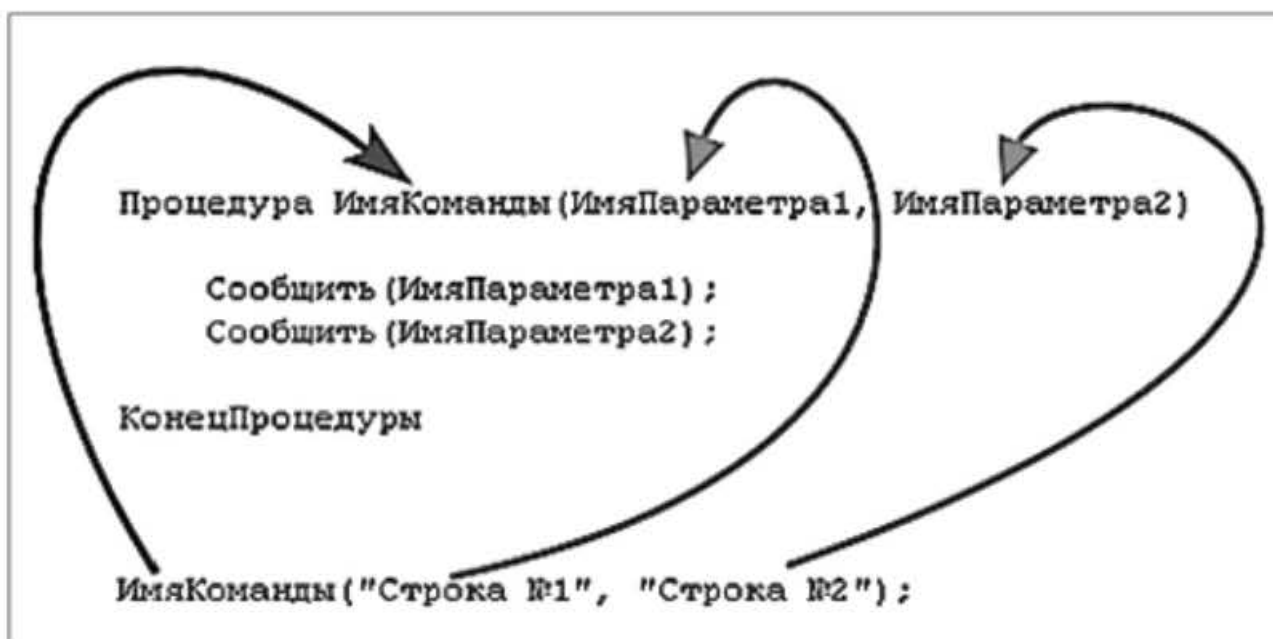


Рис. 21. Пример формирования и запуска процедуры, как объекта программы.

На рис. 22 представлена процедуры *Команда1* и *Команда2*. Схематически показаны вызов процедуры *Команда2* из процедуры *Команда1* и возврат в процедуру *Команда1* после выполнения процедуры *Команда2*.

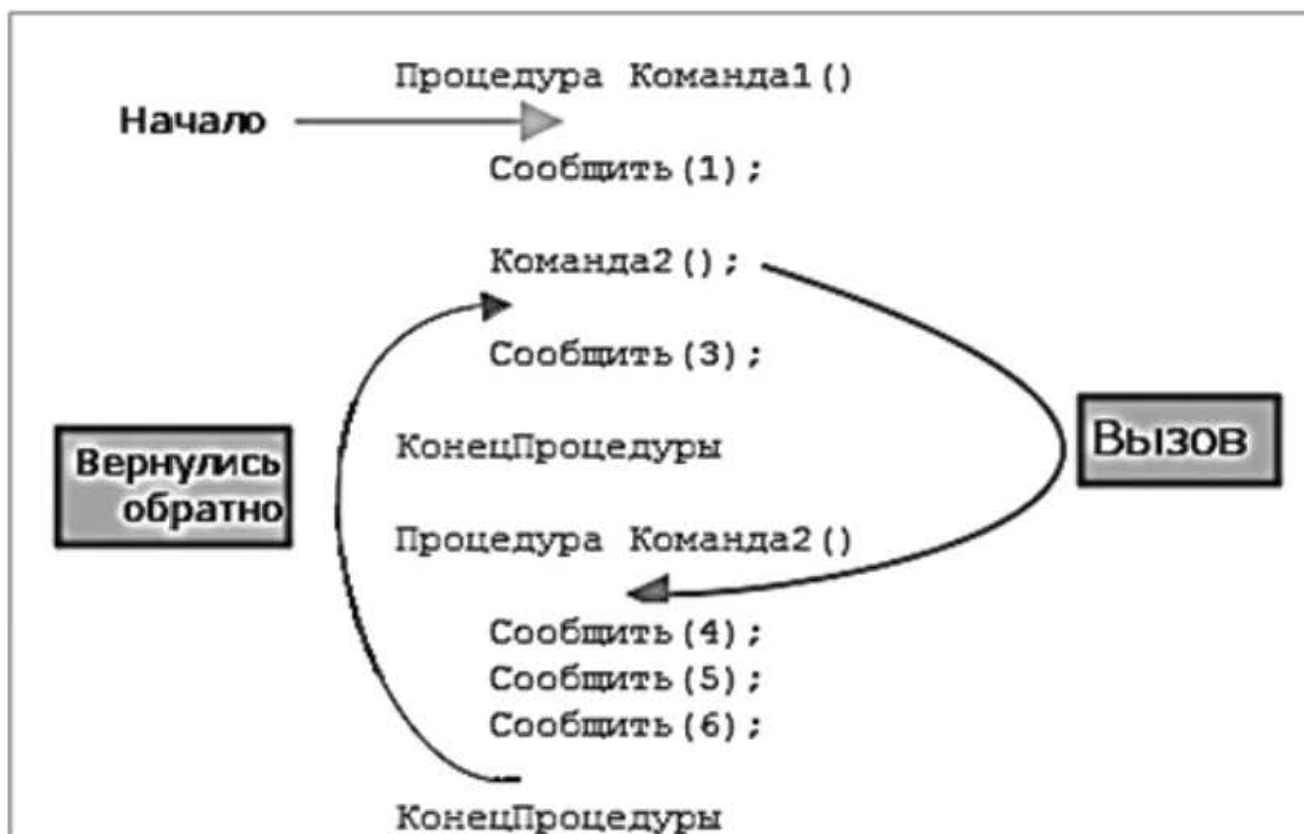
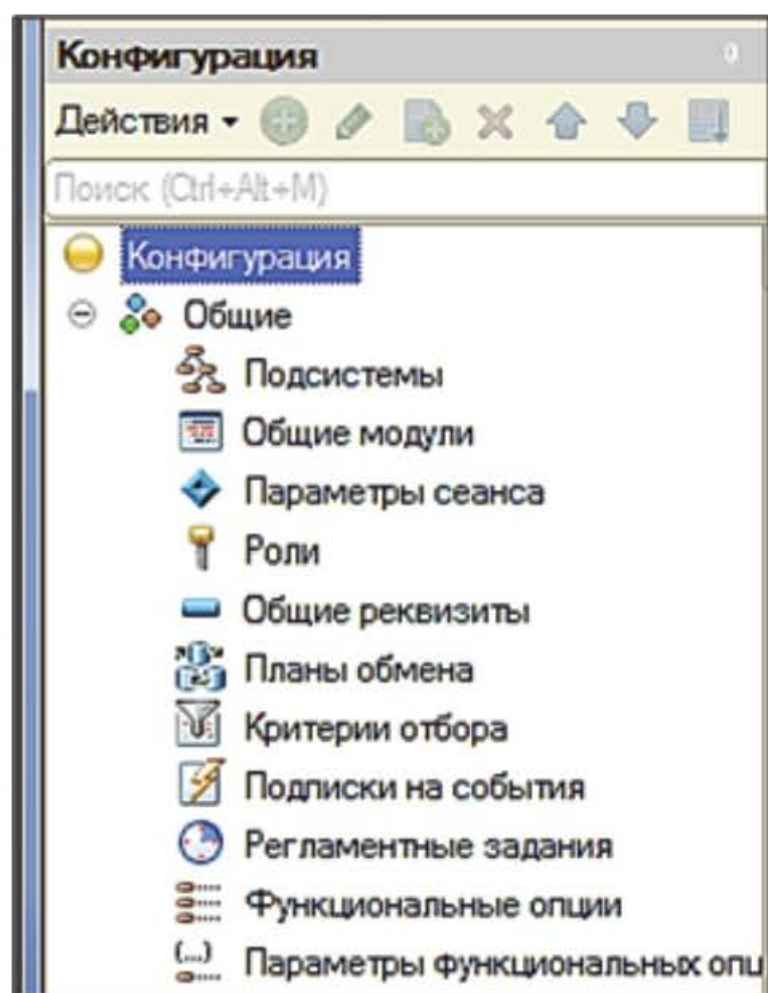


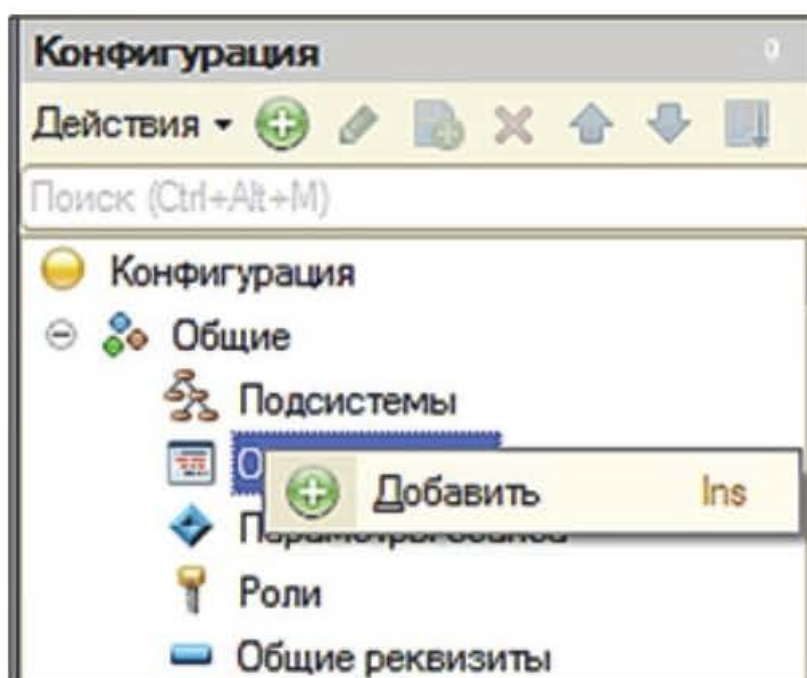
Рис. 22. Пример выполнения процедур

Для определения наших функций мы будем использовать новый модуль. Чтобы его добавить, следуйте инструкциям ниже.

1. Раскройте список "Общие" в дереве конфигурации.



2. Найдите в нем пункт "Общие модули" и нажмите на нем правой кнопкой мыши. Выберите "Добавить".

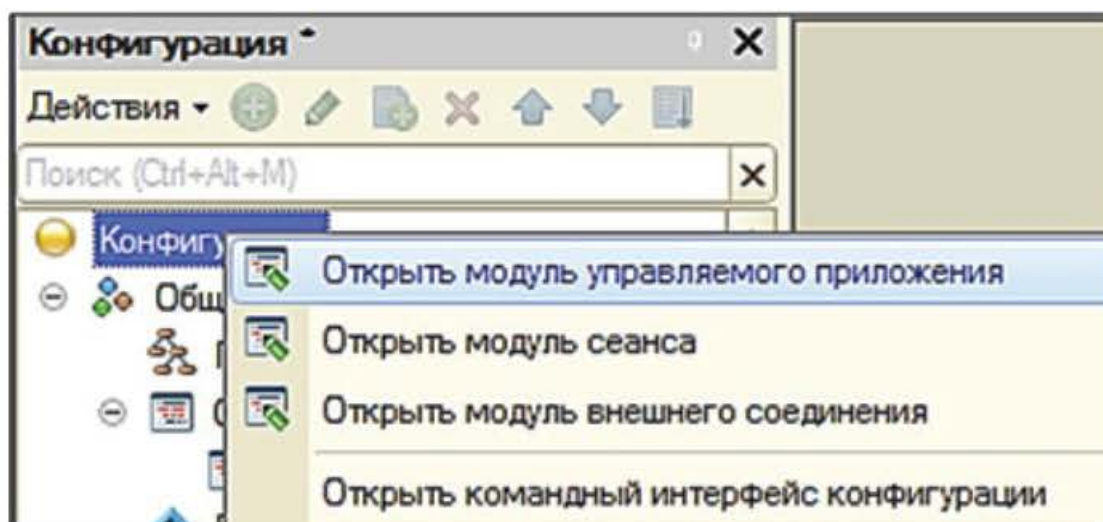


3. Добавился модуль. В правой части окна конфигуратора задайте его имя и свойства, как показано ниже.

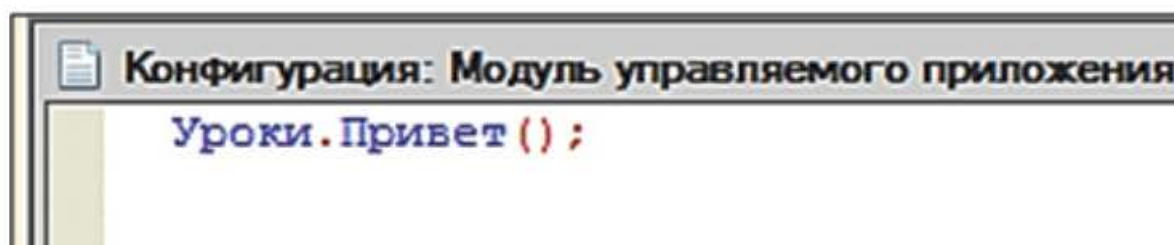
4. Перейдите в этот модуль. Здесь можно писать наши функции и процедуры. Напишите процедуру с именем *Привет*, без параметров, после вызова которой компьютер будет просто здороваться с нами.

Обратите внимание на ключевое слово *Экспорт*, которое идет следом за круглыми скобками. Его наличие обязательно, так как мы определяем функцию в одном модуле (*Уроки*), а использовать будем совсем в другом (*модуль управляемого приложения*).

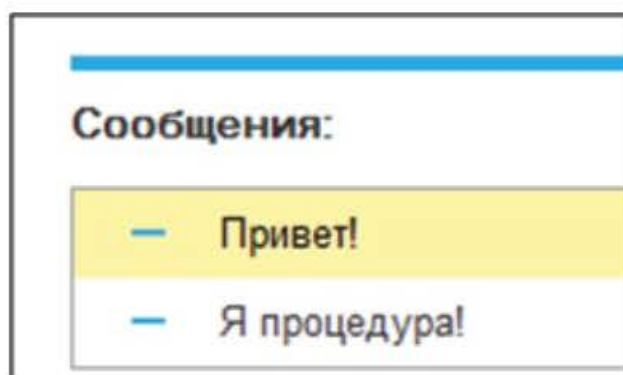
5. Теперь вернитесь в модуль управляемого приложения.



6. И сделайте вызов нашей процедуры. Так как она находится в другом модуле, к ее имени добавляется "Уроки.".



7. Запустите 1С и убедитесь, что все работает!



Примеры функций

// ПРИМЕР №1

//*** Расположить этот код в общем модуле Уроки. ***

Функция СуммаДвухЧисел(ЧислоА, ЧислоБ) Экспорт

// в функцию передаются при вызове два числа

// ЧислоА и ЧислоБ

// находим их сумму и возвращаем наружу

Возврат ЧислоА + ЧислоБ;

КонецФункции

```

//*** Расположить этот код в модуле управляемого
//приложения. ***
// вызвали функцию СуммаДвухЧисел
// передали ей в качестве параметров два числа
Сумма = Уроки.СуммаДвухЧисел(2, 5);
// теперь Сумма равна тому, что вернула нам
// функция т.е.  $2 + 5 = 7$ 
Сообщить(Сумма);

```

// ПРИМЕР №2

```

//*** Расположить этот код в общем модуле Уроки. ***
Функция ОписаниеЗнакаЧисла(ЛюбоеЧисло) Экспорт
    // в функцию передается число ЛюбоеЧисло
    // сравниваем его с нулем и в случае
    // если число больше или равно нулю
    // возвращаем наружу одну фразу
    // иначе другую.
    Если ЛюбоеЧисло >= 0 Тогда
        Возврат "Положительное число.";
    Иначе
        Возврат "Отрицательное число.";
    КонецЕсли;
КонецФункции

```

```

//*** Расположить этот код в модуле управляемого
//приложения. ***
// вызвали функцию ОписаниеЗнакаЧисла
// передали ей в качестве параметра одно число
Описание = Уроки.ОписаниеЗнакаЧисла(-3);
// теперь Описание равно строке "Отрицательное число."
// эту строку нам вернула функция
Сообщить(Описание);

```

// ПРИМЕР №3

```

//*** Расположить этот код в общем модуле Уроки. ***
Функция ДлинаОкружности(Радиус) Экспорт
    // в функцию передается число Радиус
    Пи = 3.14; // объявляем Пи
    // вычисляем результат по формуле
    Результат = 2 * Пи * Радиус;
    // возвращаем этот результат
    // наружу в место откуда была вызвана функция
    Возврат Результат;
КонецФункции

```

```

//*** Расположить этот код в модуле управляемого
//приложения. ***
// вызвали функцию, которая вычисляет длину
// окружности по переданному радиусу
Длина = Уроки.ДлинаОкружности(25);
// теперь Длина равна 157
// именно это число вернула нам функция в качестве
// результата работы
Сообщить(Длина);

```

В следующих уроках мы продолжим изучение функций и процедур на практике, а пока хорошо уясните уже изученный материал и выполните следующее задание.

Задание №36. Напишите функцию, которая принимает два числа и возвращает меньшее из них. Покажите пример использования этой функции.

Решение

```

//*** Модуль уроки ***//
Функция МеньшееИзЧисел(А, Б) Экспорт
    Если А < Б Тогда
        Возврат А;
    Иначе
        Возврат Б;
    КонецЕсли;
КонецФункции

//*** Модуль управляемого приложения ***//
ПервоеЧисло = 0;
ВвестиЧисло(ПервоеЧисло);
ВтороеЧисло = 0;
ВвестиЧисло(ВтороеЧисло);
// пример использования функции МеньшееИзЧисел
ОткрытьЗначение(
    "Меньшее из введенных вами чисел равно " +
    Уроки.МеньшееИзЧисел(ПервоеЧисло, ВтороеЧисло)
);

```

ЮНИТ 14. ФУНКЦИИ И ПРОЦЕДУРЫ В ЯЗЫКЕ 1С (ПРОДОЛЖЕНИЕ)

В юните 13 мы начали знакомство с функциями и процедурами в языке 1С 8.3. Так как это действительно важная тема, продолжим их изучение.

Функции — продолжение...

Итак, мы уже уяснили, что все программы состоят из команд компьютера.

А функции — это способ группировки этих команд точно так же, как, например, папки на компьютере — способ группировки для файлов.

Еще функцию можно рассматривать как черный ящик, внутри которого лежат какие-то команды для компьютера. Черный ящик имеет имя. И чтобы заставить компьютер выполнить команды из этого ящика, нам нужно указать его имя, например, так:

ЧерныйЯщик();

Как видите, вызов "черного ящика" сам по себе является командой. При вызове черного ящика выполнятся все команды, которые находятся внутри него.

Более того, один ящик может содержать в себе не только команды, но и другие ящики, точно так же, как папки могут быть вложенными в другие папки. Вполне понятная аналогия.

Но черные ящики (функции) имеют такую особенность:

1. Они могут содержать входные отверстия (одно или несколько) для того, чтобы мы в них могли что-то положить (параметры).

2. И они обязательно содержат одно выходное отверстие, из которого мы сможем забрать результат работы ящика.

3. Ящик-функция просто обязан вернуть нам хотя бы какой-то результат в выходное отверстие.

Пусть нам нужен ящик, в который мы будем класть два числа, а он будет возвращать их сумму.

1. Придумываем имя для этого ящика — "СуммаЧисел".

2. Придумываем имена для входных отверстий этого ящика (параметров). Их два (числа), поэтому пусть будет — "ЧислоА" и "ЧислоБ".

3. Пишем объявление самого ящика (функции):

Функция СуммаЧисел(ЧислоА, ЧислоБ)

КонецФункции

4. Теперь нам осталось поместить внутри ящика команды, которые посчитают сумму входных параметров (ЧислоА и ЧислоБ) и вернут их в выходное отверстие (команда *Возврат*):

Функция СуммаЧисел(ЧислоА, ЧислоБ)

Результат = ЧислоА + ЧислоБ;

Возврат Результат;

КонецФункции

5. Отлично! Теперь мы можем использовать этот черный ящик в своей работе. Вызывается он так:


```
ЧислоС = СуммаЧисел(10, 25);  
Сообщить(ЧислоС); // выведет 35
```

6. Обращаю ваше внимание, что при вызове мы указали имя ящика (*СуммаЧисел*), а затем его входные параметры в круглых скобках. Ящик выполнил свою работу и вернул результат в переменную *ЧислоС*, значение которой мы вывели командой *Сообщить*.

А вот как выглядит ящик, который принимает всего одно число и возвращает квадрат этого числа:

```
Функция КвадратЧисла(ЧислоА)  
    Результат = ЧислоА * ЧислоА;  
    Возврат Результат;  
КонецФункции
```

Вызывается он так:

```
Сообщить(КвадратЧисла(10)); // выведет 100
```

Функции (ящики) могут принимать и один, и два, и три и гораздо больше входных параметров. А могут и совсем не принимать.

Для примера давайте напишем функцию, которая всегда возвращает в качестве результата работы строку "Привет":

```
Функция Привет()  
    Возврат "Привет";  
КонецФункции
```

Так как функция (ящик) совсем не содержит входных параметров (отверстий), то внутри круглых скобок при вызове писать ничего не нужно:

```
Строка = Привет();  
Сообщить(Строка);
```

Процедуры — продолжение ...

Что же такое процедуры? Да те же черные ящики:

И входные отверстия (параметры) у них имеются.

И команды они в себе также могут содержать.

И другие ящики они также в себя могут включать.

А главное отличие процедур от функций в том, что они не содержат выходного отверстия. То есть процедура не возвращает нам обратный результат.

Таким образом, если нам нужно объединить группу команд, которая что-то сделает и вернет нам результат (например, сумму чисел), то используем функции.

А если обратного результата не будет — то процедуры. Процедуры являются как бы укороченными функциями. Более простыми. Они могут принимать входные параметры и что-то с ними делать, но они не возвращают нам обратный результат после своей работы.

Давайте в качестве примера напишем такой ящик-процедуру, который будет принимать на вход число и, если это число больше или равно нулю, то прямо внутри ящика будет выводиться сообщение "положительное число", иначе "отрицательное число".

1. Определим имя нашего ящика — "ОписаниеЧисла".
2. Определим имя входного параметра — "ЧислоА";
3. Определим описание нашего ящика:

Процедура ОписаниеЧисла(ЧислоА)
КонецПроцедуры

4. Теперь поместим внутрь ящика команды, которые выведут описание числа в зависимости от его знака:

Процедура ОписаниеЧисла(ЧислоА)
Если ЧислоА ≥ 0 Тогда
 Сообщить("положительное число");
Иначе
 Сообщить("отрицательное число");
КонецЕсли;
КонецПроцедуры

5. А вызывать наш ящик будем так:

ОписаниеЧисла(-3);

Задание №37. Напишите ящик, который принимает радиус круга, а возвращает его площадь. Продемонстрируйте его использование.

Решение

```
//*** Модуль уроки ***//  
Функция ПлощадьКруга(Радиус) Экспорт  
    Пи = 3.14;  
    Возврат Пи * Радиус * Радиус;  
КонецФункции  
  
//*** Модуль управляемого приложения ***//  
Радиус = 0;  
ВвестиЧисло(Радиус);
```

```
ОткрытьЗначение(  
    "Площадь круга радиусом " + Радиус +  
    " равна " + Уроки.ПлощадьКруга(Радиус)  
);
```

Задание №38. Напишите ящик, который принимает две строки, а возвращает их объединение (или, как говорят, конкатенацию). То есть передали "Иван" и "Петров", а вернул "Иван Петров". Продемонстрируйте его использование.

Решение

```
//*** Модуль уроки ***//  
Функция ФИО(Имя, Фамилия) Экспорт  
    Возврат Имя + " " + Фамилия;  
КонецФункции  
//*** Модуль управляемого приложения ***//  
ВашеИмя = "";  
ВвестиСтроку(ВашеИмя, "Введите ваше имя");  
ВашаФамилия = "";  
ВвестиСтроку(ВашаФамилия, "Введите вашу фамилию");  
ОткрытьЗначение("Здравствуйте, " +  
    Уроки.ФИО(ВашеИмя, ВашаФамилия));
```

Задание №39. Напишите ящик, который принимает два числа, а при своей работе выводит на экран все целые числа в промежутке от первого до второго числа. Например, передали 1 и 5, а он вывел: 1 2 3 4 5. Продемонстрируйте его использование.

Решение

```
//*** Модуль уроки ***//  
Процедура ВывестиЧисла(ПервоеЧисло, ПоследнееЧисло) Экспорт  
    Строка = "";  
    Для Шаг = ПервоеЧисло По ПоследнееЧисло Цикл  
        Строка = Строка + Шаг + " ";  
    КонецЦикла;  
    ОткрытьЗначение(Строка);  
КонецПроцедуры  
//*** Модуль управляемого приложения ***//  
ПервоеЧисло = 0;  
ВвестиЧисло(ПервоеЧисло, "Введите первое число");
```

```
ВтороеЧисло = 0;  
ВвестиЧисло(ВтороеЧисло, "Введите второе число");  
Уроки.ВывестиЧисла(ПервоеЧисло, ВтороеЧисло);
```

ЮНИТ 15. ОТЛАДКА В ЯЗЫКЕ 1С

Как вы знаете, выполнение программы 1С компьютером - это последовательное исполнение команд, написанных программистом. В данном юните мы научимся останавливать компьютер во время выполнения определенной команды и просматривать значения переменных в этот самый момент.

Этот процесс в 1С называется отладкой и он бывает очень полезен для того, чтобы разобраться в том, как программа работает. Благодаря отладке мы также можем легко находить ошибки, которые допустили на этапе написания кода.

Итак, давайте начнем наш эксперимент.

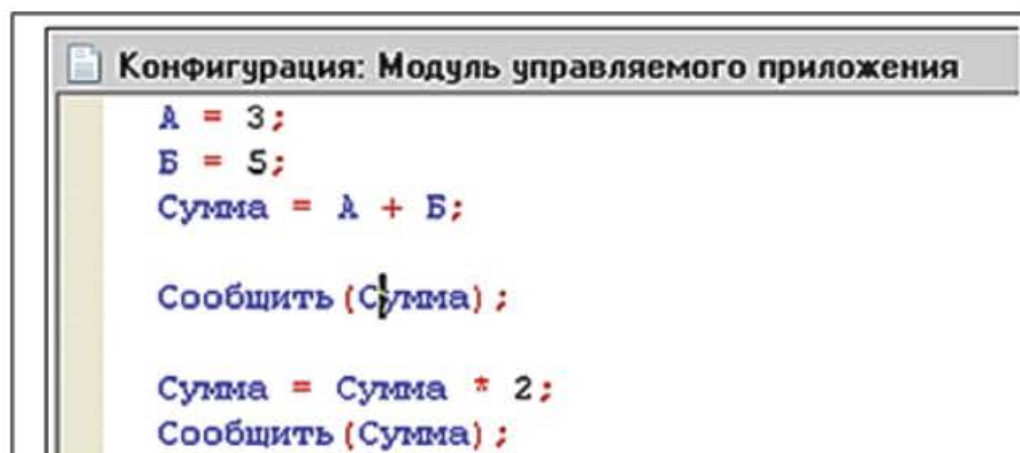
Напишите в модуле управляемого приложения следующий код:

```
А = 3;  
Б = 5;  
Сумма = А + Б;  
Сообщить(Сумма);  
Сумма = Сумма * 2;  
Сообщить(Сумма);
```

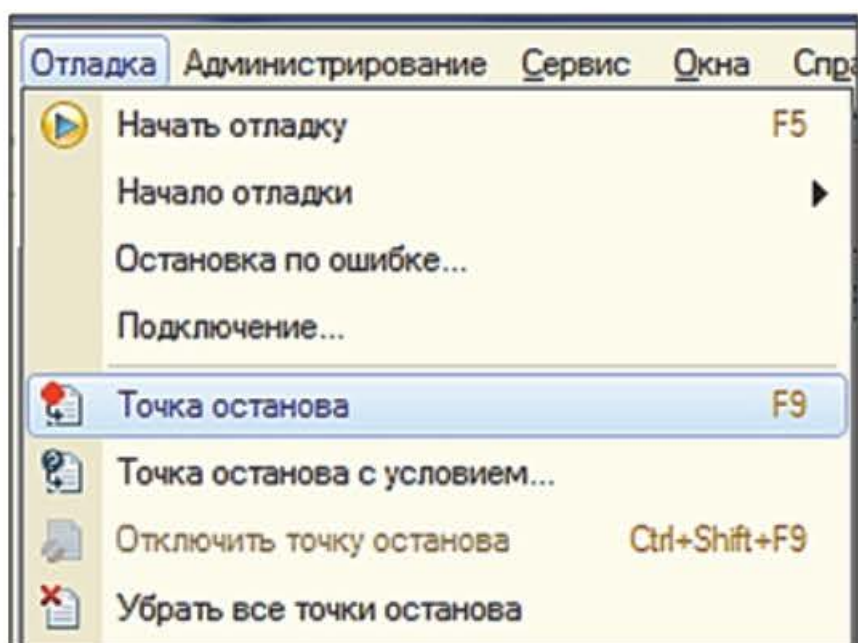
Код очень простой. Сначала мы определяем две переменные с именами А и Б и значениями 3 и 5 соответственно. Затем складываем эти числа и кладем результат сложения в переменную с именем *Сумма*. Затем выводим это значение на экран. Затем удваиваем это значение. И снова выводим.

Как уже было сказано, компьютер выполняет все эти команды последовательно, одну за другой. Давайте заставим его остановиться на пятой строчке, в которой мы выводим результат сложения двух чисел.

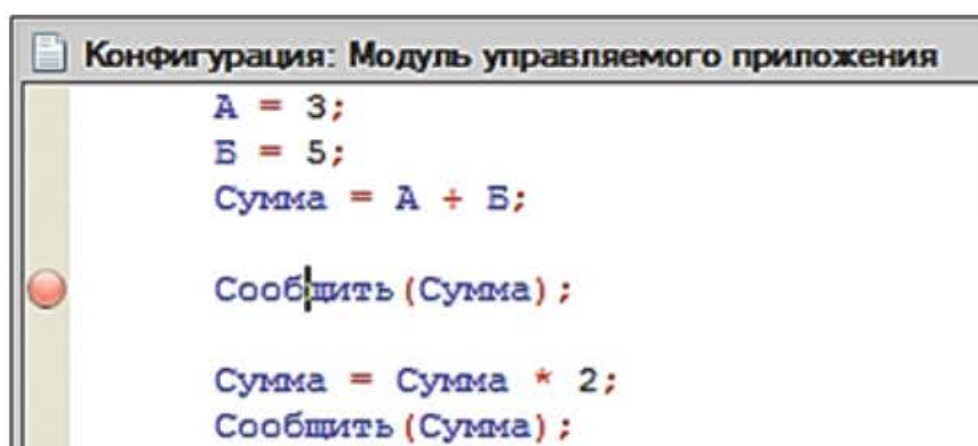
Для этого установим курсор мыши в пятую строчку:



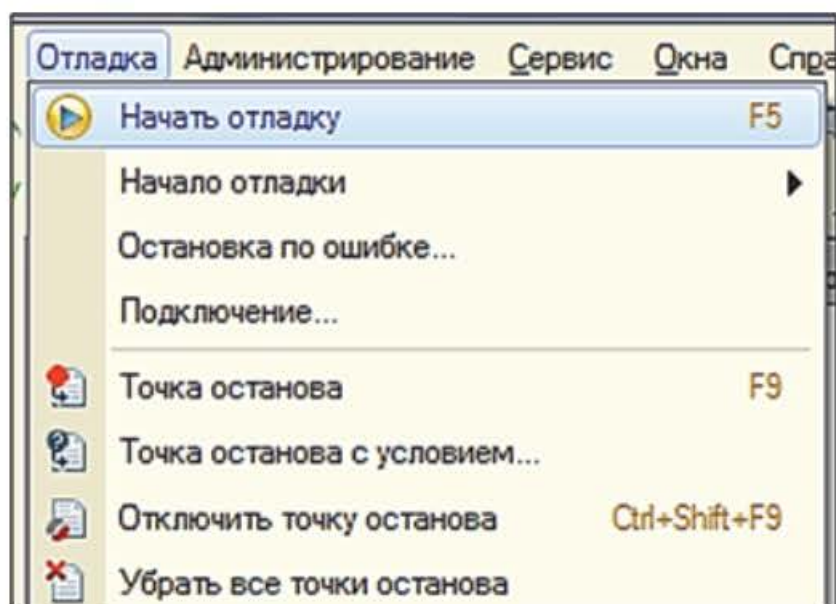
И в главном меню выберем команду "Отладка"-"Точка останова".



Тогда слева от пятой строки появится красный кружок. Этот кружок и означает, что компьютер при переходе к этой строке сделает остановку. В момент остановки у нас будет возможность более детально исследовать состояние нашей программы.



Теперь запустим отладку программы. Для этого через главное меню выполним команду "Отладка"-"Начать отладку".



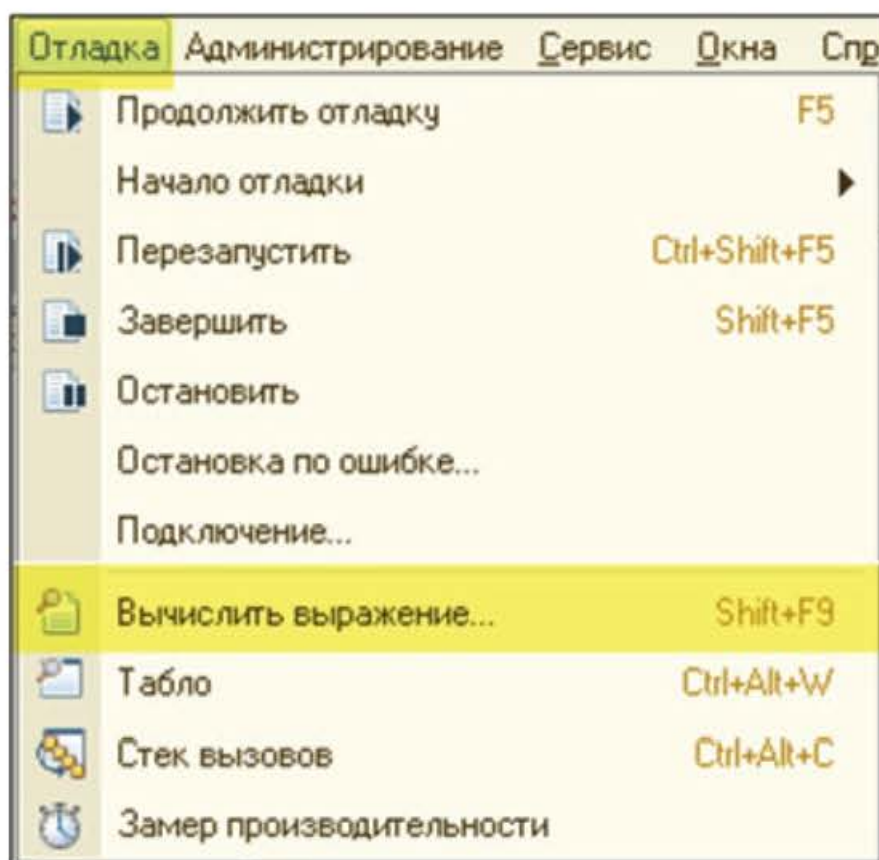
И увидим, что компьютер дошел до пятой строки и замер в ожидании наших дальнейших действий. Об этом свидетельствует стрелка, появившаяся слева от пятой строки. Эта стрелка указывает нам, на какой команде находится компьютер в данный момент исполнения программы.

```
A = 3;  
B = 5;  
Сумма = A + B;  
Сообщить (Сумма) ;  
  
Сумма = Сумма * 2;  
Сообщить (Сумма) ;
```

Предположим, что мы хотим увидеть, чему равно значение переменной Сумма в данный момент. Для этого выделим мышкой эту переменную:

```
Сообщить (Сумма) ;
```

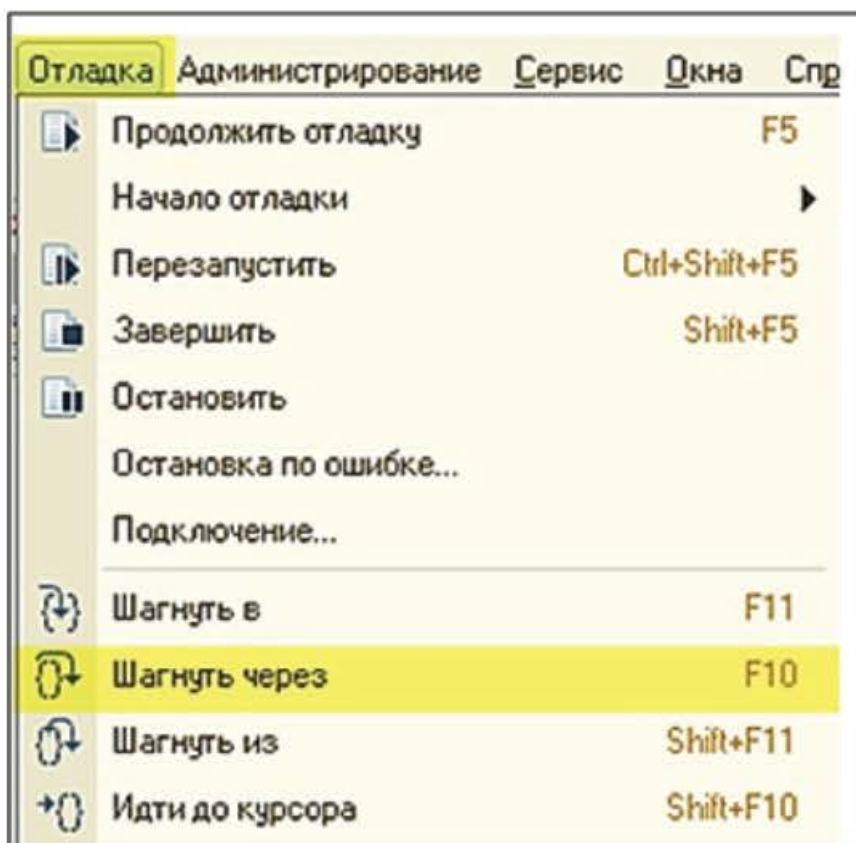
И выполним через главное меню команду "Отладка"- "Вычислить выражение...":



Откроется следующее окно, в котором мы видим нашу переменную и ее значение в данный момент. Оно действительно равно восьми. Закройте это окно.

Выражение		
Выражение:		
Сумма		
Результат:		
Свойство	Значение	Тип
Сумма	8	Число

Но давайте заставим компьютер выполнить текущую команду и перейти к следующей. Для этого выполним "Отладка" - "Шагнуть через":



И мы увидим, что стрелка выполнения переместилась на следующую команду, которая удваивает значение переменной Сумма:



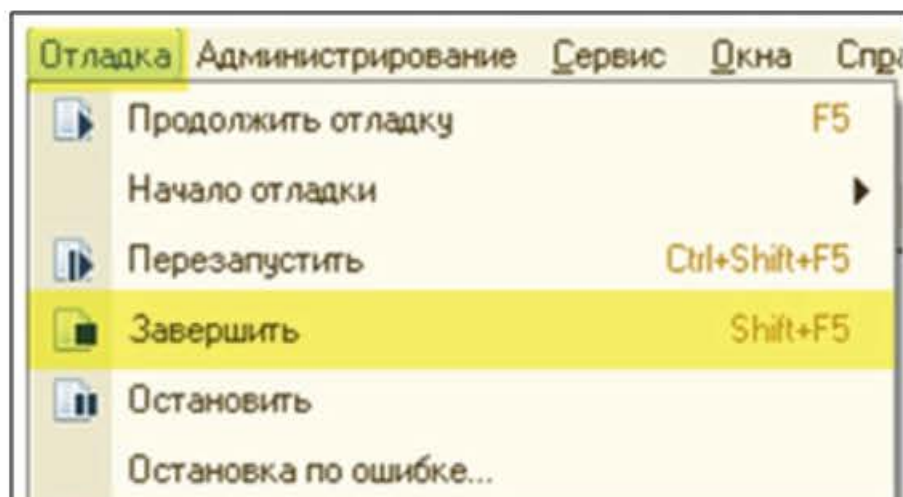
И еще раз выполним "Отладка"-"Шагнуть через", чтобы стрелочка оказалась на последней строке:



А теперь снова выделим переменную Сумма и выполним "Отладка"—"Вычислить выражение...". Как видите, значение переменной Сумма теперь стало равно 16:



Чтобы прекратить отладку программы, выполним команду "Отладка"—"Завершить".



Когда отладка закончена и мы не хотим, чтобы компьютер останавливался во время выполнения программы, просто выполним "Отладка"—"Убрать все точки останова", и все красные точки из программы исчезнут.

Выполните отладку следующей программы:

```
МассивЧисел = Новый Массив(5); // объявим массив на
                                // 5 ящиков
МассивЧисел[0] = 10;
МассивЧисел[1] = 5;
МассивЧисел[2] = 450;
МассивЧисел[3] = 654;
МассивЧисел[4] = 34;

// изначально делаем наименьшее число заведомо очень
//большим
НаименьшееЧисло = 1000000;
Для Шаг = 0 По 4 Цикл // пробегаемся по всем
                        //значениям массива
    // если в массиве есть число меньшее,
    //чем НаименьшееЧисло
    Если МассивЧисел[Шаг] < НаименьшееЧисло Тогда
        // тогда его и делаем новым наименьшим числом
        НаименьшееЧисло = МассивЧисел[Шаг];
    КонецЕсли;
КонецЦикла;
```

Сообщить("Наименьшее из введенных чисел равно " + НаименьшееЧисло);

В процессе отладки установите в качестве точки останова самую первую строку и последовательно выполняйте команды одну за одной. На каждом шаге вычисляйте значения переменных. Экспериментируйте до тех пор, пока не освоите этот замечательный и столь необходимый в работе инструмент.

ЮНИТ 16. СИНТАКС-ПОМОЩНИК В ЯЗЫКЕ 1С

Мы подошли к такому этапу изучения языка 1С, на котором пора научиться пользоваться встроенным синтакс-помощником. Синтакс-помощник — это справочная система по языку 1С 8.3, встроенная в конфигуратор.

Синтакс-помощник описывает сам язык и правила его использования. В нем перечислены все команды и их параметры, даны примеры использования.

Научитесь использовать и понимать синтакс-помощник, и это существенно сэкономит время, которое вы потратите на освоение языка. Умение пользоваться документацией по языку программирования отличает профессионала от любителя.

Использование синтакс-помощника языка 1С

Предположим, что мы пишем или просматриваем код с использованием массива и забыли, что такое массив, какие у него есть команды и как им пользоваться.

Можно, конечно, полезть в учебники и интернет, но в таких ситуациях нет ничего быстрее и проще, чем прибегнуть к встроенному синтакс-помощнику.

Для этого установите курсор на интересующее слово (сейчас нас интересует тип данных *Массив*, поэтому установим курсор на него):

Данные = Новый Массив();

Нажмите на клавиатуре комбинацию клавиш **Ctrl + F1**.

Клавиши нажимаются так:

1. Нажимаете Ctrl.
2. Удерживая Ctrl, жмете F1.
3. Отпускаете все клавиши.

Тогда в правой части окна откроется окно синтакс-помощника с описанием типа данных *Массив*:



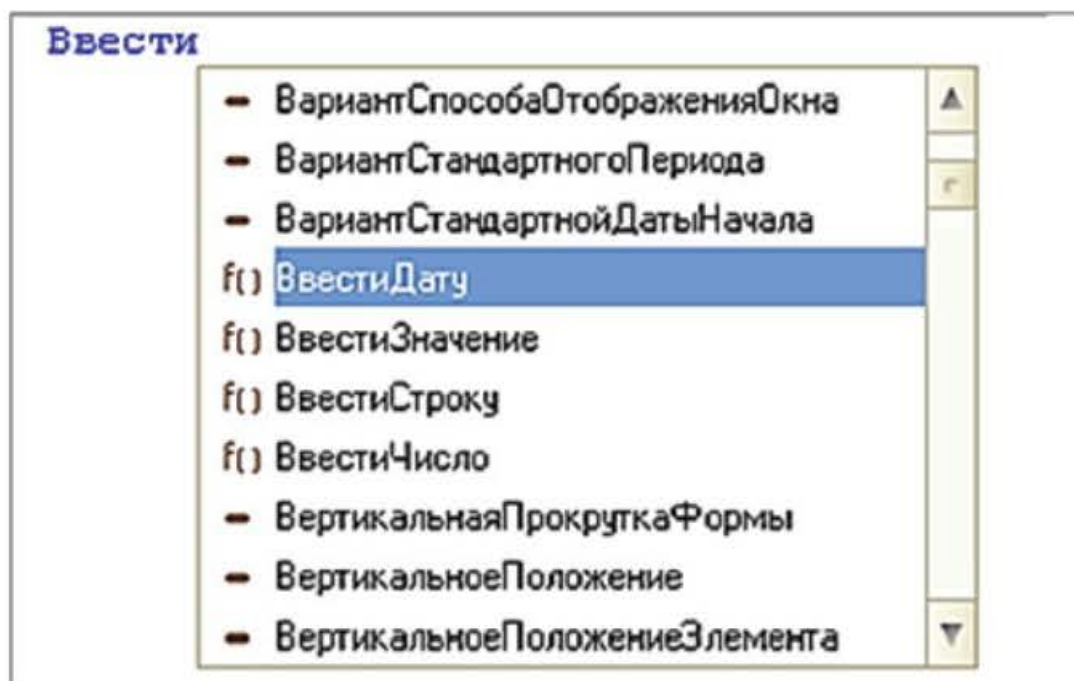
Обратите внимание на то, что в этом окне дана исчерпывающая информация по массивам: полное описание, команды и примеры использования.

Такую информацию вы можете получить по любому из объектов встроенного языка 1С 8.3. Достаточно установить курсор на интересующую вас конструкцию и нажать **Ctrl + F1**.

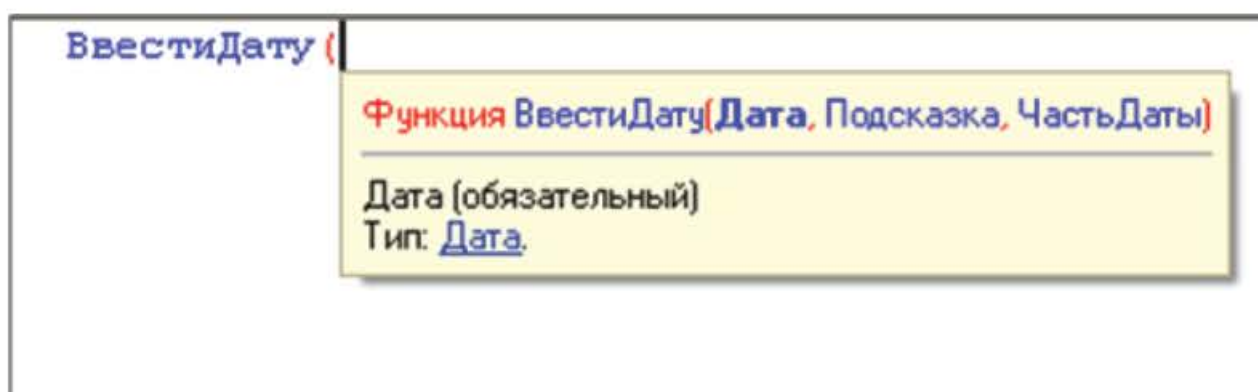
Рассмотрим следующую ситуацию.

Давайте представим, что нам нужно написать команду по вводу даты. Мы не знаем ее полного названия, но знаем, что она начинается со слова "Ввести".

Давайте напишем слово "Ввести", а затем нажмем на клавиатуре комбинацию клавиш **Ctrl + Пробел**. Как видите, перед нами открылось окно возможных вариантов команд, которые начинаются со слова "Ввести".



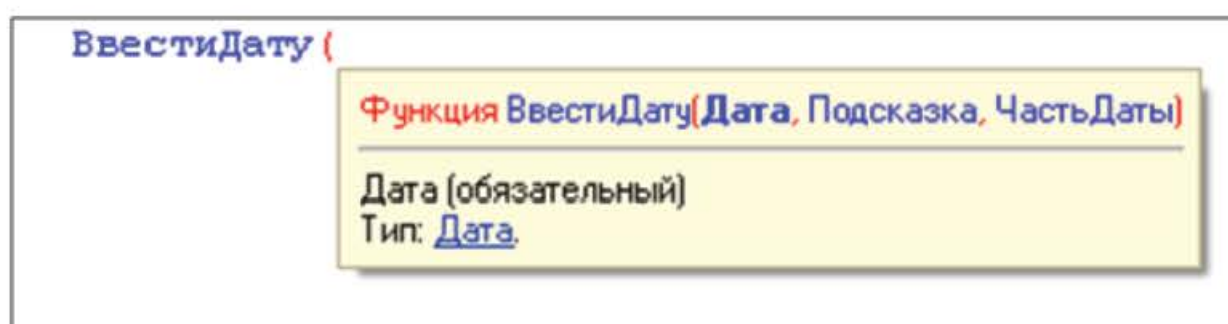
Выберем подходящую нам *ВвестиДату* и нажмем **Enter**. И команда вставится в редактор автоматически. Причем компьютер сам откроет круглую скобку, подсказывая, что мы ввели команду и нужно указать параметры ее вызова. И тут же откроется окно подсказки параметров вызова.



А давайте нажмем Esc, чтобы подсказка параметров вызова закрылась:

ВвестиДату (

Как теперь открыть ее снова? Очень просто. Для этого поместите курсор после круглой скобки и нажмите **Ctrl + Shift + Пробел**. Подсказка появится вновь:

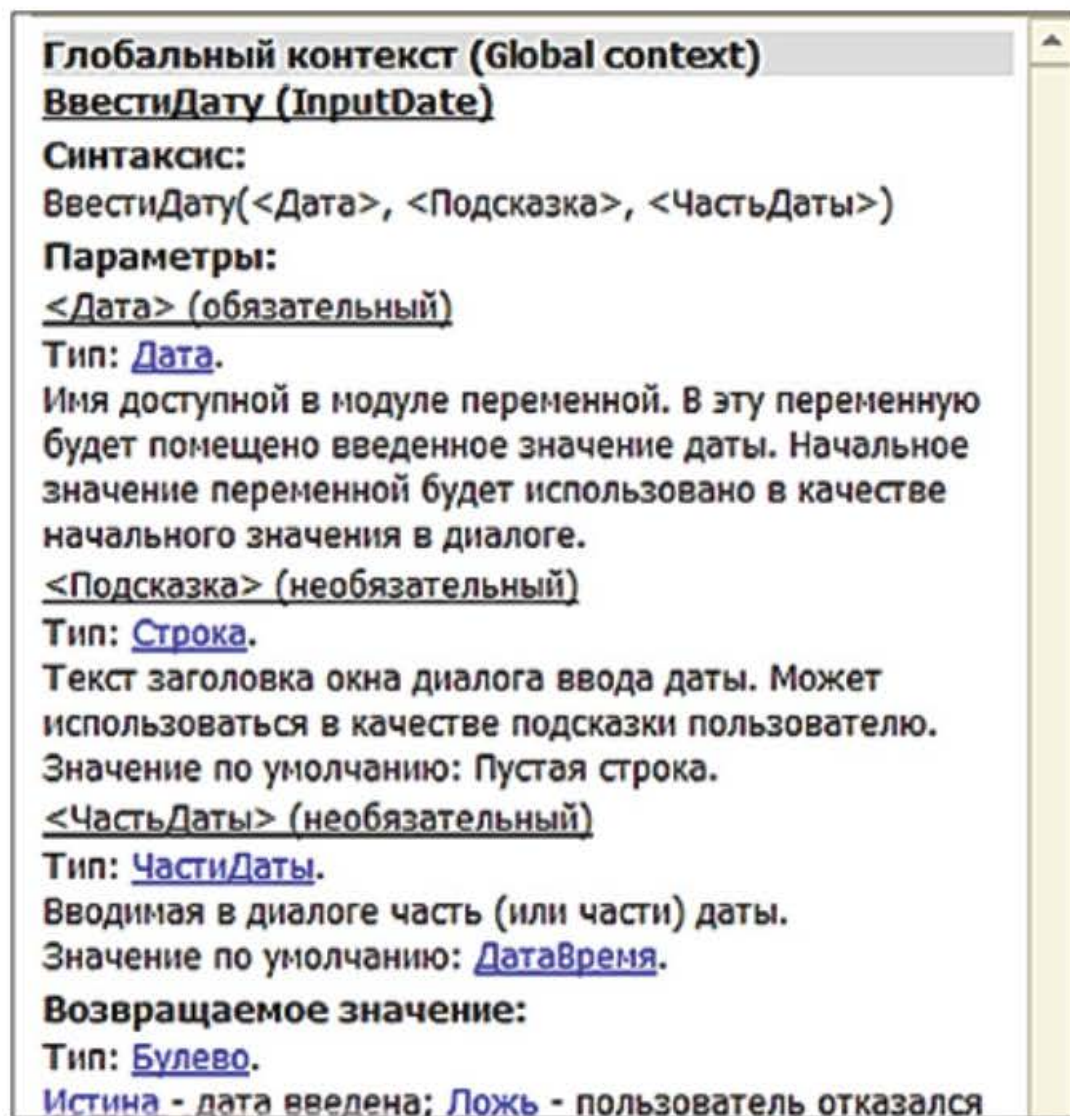


Из подсказки следует, что команда ввода даты принимает три параметра, но все ли они являются обязательными? Как это выяснить?

А давайте посмотрим более полное описание команды в синтаксис-помощнике. Для этого установим курсор на название команды:

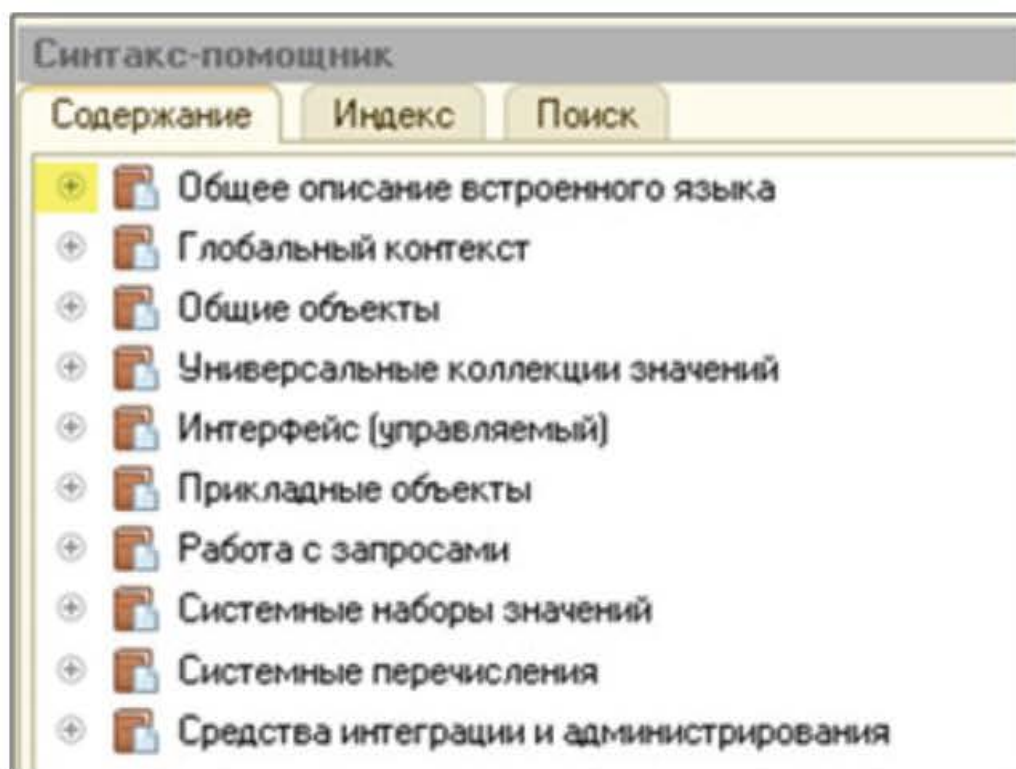
ВвестиДату (

И нажмем **Ctrl + F1**. Откроется окно синтаксис-помощника с полным описанием команды *ВвестиДату*:

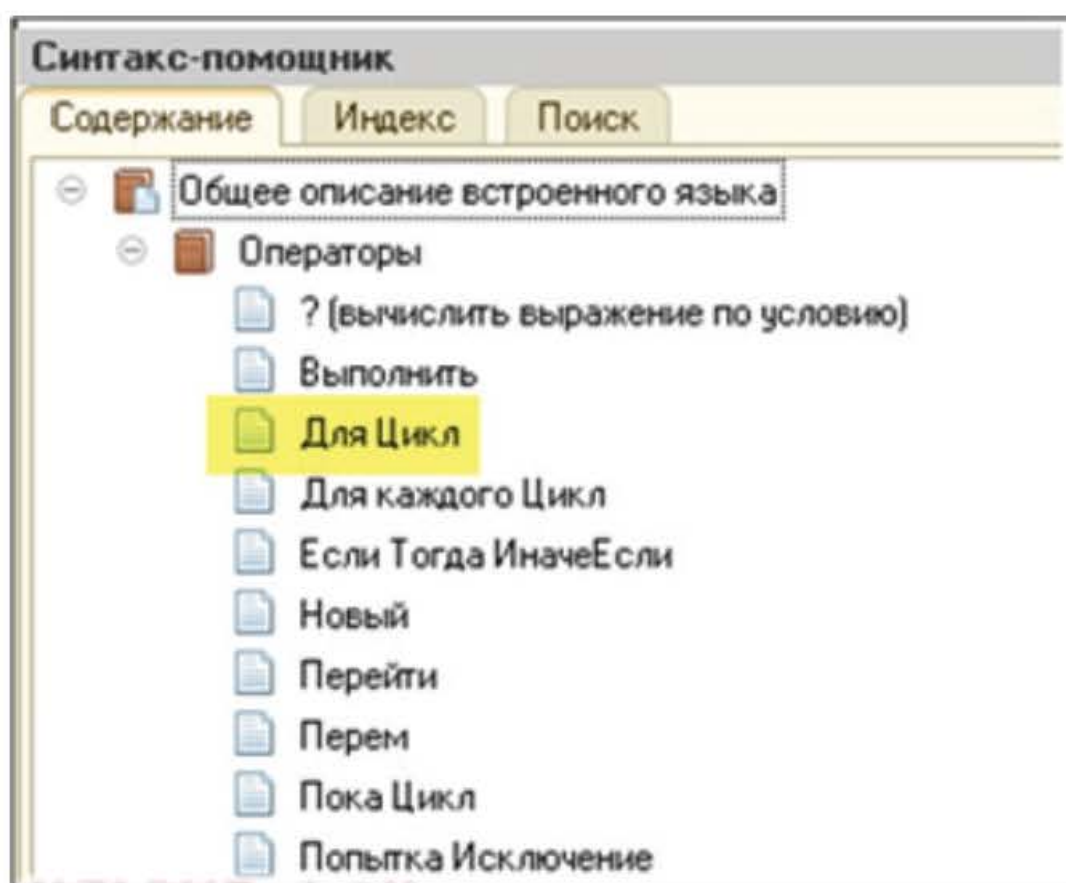


Из описание видно, что обязательным является только первый параметр: переменная, в которую будет возвращено значение, введенное пользователем. Кроме того, мы видим, что сама команда вернет значение *Истина* в том случае, если пользователь ввел дату, и *Ложь*, если пользователь отказался от ввода даты.

Но и это еще не все. В верхней части синтаксис-помощника (который открывается по **Ctrl + F1**) на закладке *Содержание* можно увидеть оглавление, в котором собраны все разделы помощника.



Нас сейчас в первую очередь интересует общее описание встроенного языка. Раскройте его:



Здесь можно еще раз прочитать про уже изученные нами конструкции и узнать много нового.

Многим программистам не нравится синтаксис-помощник языка 1С. Они находят его неудобным, но поверьте — к нему нужно просто привыкнуть и тогда любую информацию по языку 1С вы сможете находить в два счета.

Найдите и изучите в синтакс-помощнике информацию по следующим уже знакомым нам объектам:

Цикл Для

Цикл Пока

Условный оператор Если Тогда

Структура

Постарайтесь понять как можно больше из описанного в синтакс-помощнике по указанным темам.

ЮНИТ 17. РАБОТА С ДАТАМИ В ЯЗЫКЕ 1С

Тип дата в языке 1С 8.3 уже знаком нам по прошлым юнитам. Сегодня мы рассмотрим более подробно операции, которые можно совершать над датами.

Итак, откуда изначально появляется тип дата в нашей программе?

Во-первых, мы можем задать значение даты прямо в коде:

```
ПервыйДеньЛета = '20140601';
```

```
Сообщить(ПервыйДеньЛета);
```

В этом случае значение даты вводится в ординарных кавычках. Первые четыре числа означают год (2014), следующие два числа — месяц (06, июнь) и, наконец, последние два — день месяца (01). Таким образом мы задали 1 июня 2014 года.

Во-вторых, мы можем использовать специальную команду, формирующую значение даты:

```
ПервыйДеньЛета = Дата(2014, 06, 01);
```

```
Сообщить(ПервыйДеньЛета);
```

Результат будет тем же самым. Но в этом случае мы вызываем команду (или, правильнее сказать, конструктор) *Дата* и в качестве параметров указываем числа, обозначающие год, месяц и число нашей даты.

В-третьих, мы можем ввести значение даты от пользователя:

```
ДатаОтПользователя = '00010101';
```

```
ВвестиДату(ДатаОтПользователя);
```

```
ОткрытьЗначение(ДатаОтПользователя);
```

Тут мы сначала заводим переменную для хранения значения даты и присваиваем начальное пустое значение ('00010101'), а затем вызываем команду *ВвестиДату*, указывая в качестве параметра переменную, в которую нужно вернуть значение, введенное пользователем.

И, наконец, мы можем получить значение даты как результат работы другой команды 1С. Например, так:

```
Сегодня = ТекущаяДата();  
Сообщить(Сегодня);
```

Давайте продолжим работу с последним примером (будем дописывать в него код).

Вот как мы можем из уже готовой даты вытащить только год, месяц или число:

```
Сообщить(Год(Сегодня));  
Сообщить(Месяц(Сегодня));  
Сообщить(День(Сегодня));
```

А что если нужно прибавить к дате один день? Это можно сделать вот так:

```
Сегодня = ТекущаяДата();  
Завтра = Сегодня + 24 * 60 * 60;  
Сообщить(Завтра);
```

Почему мы прибавили именно такое число ($24 * 60 * 60$)? Потому что напрямую к дате можно прибавлять только секунды. А нам нужно прибавить сутки, в которых 24 часа, в часе 60 минут, в минуте 60 секунд.

Секунды можно не только прибавлять, но и отнимать от даты.

А что если нам нужно прибавить целый месяц? Конечно, и в этом случае можно посчитать количество секунд в месяце и прибавить, но гораздо проще использовать специальную команду 1С, которая как раз для этого предназначена:

```
Сегодня = ТекущаяДата();  
ЧерезМесяц = ДобавитьМесяц(Сегодня, 1);
```

Команда *ДобавитьМесяц* в качестве параметров принимает дату, к которой нужно прибавить месяц и количество месяцев. Если указать отрицательное количество, то произойдет не прибавление, а вычитание месяцев из даты.

Даты также можно отнимать друг от друга:

```
РазностьДат = '20140101' - '20100101';  
Сообщить(РазностьДат);
```

Результатом вычитания дат будет количество секунд между ними. Ну, а как перевести секунды в минуты, часы или даже дни, думаю, вы догадаетесь сами.

Какие еще есть интересные функции для работы с датой в 1С?
Мы можем узнать день недели:

Сообщить(ДеньНедели(Сегодня));

Команда *ДеньНедели* возвращает число от 1 до 7. Единица обозначает понедельник, двойка — вторник, и так до воскресенья.

Таким же образом мы можем узнать порядковый номер дня с начала года:

Сообщить(ДеньГода(Сегодня));

А также порядковый номер недели (к которой относится дата) в году:

Сообщить(НеделяГода(Сегодня));

Но и это еще не все функции для работы с датой, которые есть в 1С. С более полным перечнем на примерах вы можете ознакомиться в разделе 2.

Для отработки и закрепления материала данного юнита рекомендуется выполнить два нижеследующих задания:

а) Напишите функцию, которая принимает дату и возвращает день недели, на который приходится дата в строковом виде (например, понедельник). Теперь, используя эту функцию, напишите программу, которая выводит пользователю, какой сегодня день недели.

Решение

```
/** Модуль уроки **/
```

```
Функция ДеньНеделиСтрокой(Дата) Экспорт
```

```
Если ДеньНедели(Дата) = 1 Тогда
```

```
    Возврат "Понедельник";
```

```
ИначеЕсли ДеньНедели(Дата) = 2 Тогда
```

```
    Возврат "Вторник";
```

```
ИначеЕсли ДеньНедели(Дата) = 3 Тогда
```

```
    Возврат "Среда";
```

```
ИначеЕсли ДеньНедели(Дата) = 4 Тогда
```

```
    Возврат "Четверг";
```

```
ИначеЕсли ДеньНедели(Дата) = 5 Тогда
```

```
    Возврат "Пятница";
```

```
ИначеЕсли ДеньНедели(Дата) = 6 Тогда
```

```
    Возврат "Суббота";
```

```
ИначеЕсли ДеньНедели(Дата) = 7 Тогда
```

```
    Возврат "Воскресенье";
```



```
КонецЕсли;  
Возврат "Невозможное возможно";  
КонецФункции
```

```
/** Модуль управляемого приложения */  
Сообщить("Сегодня " +  
Уроки.ДеньНеделиСтрокой(ТекущаяДата()));
```

б) Напишите программу, которая вводит от пользователя дату его рождения и, если день его рождения в этом году уже прошел, то выводит "Ваш день рождения в этом году уже прошел!", иначе выводит количество секунд, оставшееся до дня рождения.

Решение

```
ДатаРождения = '00010101';  
ВвестиДату(ДатаРождения, "Когда вы родились?");  
Сегодня = ТекущаяДата();  
ДеньРожденияВЭтомГоду = Дата(Год(Сегодня),  
Месяц(ДатаРождения),  
День(ДатаРождения));  
Если Сегодня > ДеньРожденияВЭтомГоду Тогда  
ОткрытьЗначение("Ваш день рождения в этом году уже  
прошел!");  
Иначе  
ОсталосьСекунд = ДеньРожденияВЭтомГоду - Сегодня;  
ОткрытьЗначение(  
"До вашего дня рождения в этом году осталось " +  
ОсталосьСекунд + " секунд!");  
КонецЕсли;
```

ЮНИТ 18. МАТЕМАТИКА В ЯЗЫКЕ 1С

В одном из первых юнитов мы уже разбирали использование арифметических операций в 1С 8.3. Конечно, это не все математические операции, которые доступны разработчику, и в этом юните мы восполним этот пробел.

Ваша задача воспроизводить все примеры у себя на компьютере в модуле управляемого приложения.

Представим, что у нас есть число с дробной частью:

Цена = 100.45;

И мы хотим округлить его дробную часть до первого знака после запятой:

```
ОкругленнаяЦена = Окр(Цена, 1);  
// вернет 100.5
```

Мы воспользовались командой *Окр*, которая принимает число и количество знаков дробной части, после которых нужно округлить значение.

В качестве результата округления мы получим 100.5. Но почему не 100.4?

По умолчанию, команда *Окр* делает округление в большую сторону, поэтому мы получим 100.5. Но если потребуется округление в меньшую сторону, нужно просто передать третьим параметром ноль:

```
ОкругленнаяЦена = Окр(Цена, 1, 0);
```

```
// вернет 100.4
```

А теперь давайте отбросим дробную часть числа, чтобы осталась только целая часть:

```
ЦелаяЧасть = Цел(Цена);
```

```
// вернет 100
```

Мы воспользовались командой *Цел*, которая принимает число с дробной частью и возвращает только его целую часть.

Иногда при расчетах требуется возвести число в степень:

```
ДесятьВКубе = Pow(10, 3);
```

```
// вернет 1000
```

Команда *Pow* принимает число и степень, в которую его нужно возвести. А порою нам требуется, наоборот, к примеру, извлечь квадратный корень из числа:

```
КвадратныйКореньИз25 = Sqrt(25);
```

```
// вернет 5
```

Еще задача. Пусть у нас имеется конечный набор чисел и нужно найти максимальное значение:

```
МаксимальноеЗначениеИзТрехЧисел = Макс(10, 20, 30);
```

```
// вернет 30
```

```
МаксимальноеЗначениеИзДвухЧисел = Макс(0, -5);
```

```
// вернет 0
```

Количество параметров, которые можно передавать в функцию *Макс* для поиска максимального элемента неограниченно.

Функция *Мин* решает обратную задачу:

```
МинимальноеЗначениеИзТрехЧисел = Мин(10, 20, 30);
```

```
// вернет 10
```

```
МинимальноеЗначениеИзДвухЧисел = Мин(0, -5);
```

```
// вернет -5
```

Еще очень популярной задачей является генерация случайных чисел. В 1С это делается таким образом:

```
ГСЧ = Новый ГенераторСлучайныхЧисел();
```

```
Сообщить(ГСЧ.СлучайноеЧисло(0, 2));
```

Первой строкой мы создаем объект *ГенераторСлучайныхЧисел* и связываем с ним нашу переменную *ГСЧ*.

Во второй строке мы вызываем команду этого генератора *СлучайноеЧисло*, которая в качестве параметров принимает левую и правую границы (включительно) диапазона, в котором требуется генерировать случайные числа.

В приведенном примере вы каждый раз будете запускать программу, и она каждый раз будет вам выдавать новое число от нуля (включительно) до двух (включительно). Это и есть генерация случайных чисел.

С более полным перечнем математических функций в 1С вы можете ознакомиться во втором разделе пособия.

Для отработки и закрепления материала данного юнита рекомендуется выполнить нижеследующее задание.

Напишите программу, которая генерирует три случайных числа в диапазоне от 1 до 1000. Генератор случайных чисел инициализируете числом 2014 (об этом ниже). Затем каждое число возводится в 5 степень. Затем из каждого числа извлекается квадратный корень. Затем каждое число делится на 3. Затем каждое число округляется до второго знака в меньшую сторону. Затем каждое число умножается на 0.25. Затем от каждого числа отбрасывается дробная часть. И, наконец, находится сумма всех получившихся чисел, которая и выводится пользователю. Получившееся значение будет равно 849442.

Подсказка:

Чтобы инициализировать генератор определенным числом (например, 2014) напомним:

```
ГСЧ = Новый ГенераторСлучайныхЧисел(2014);
```

Инициализация определенным числом позволит вам получить на своем компьютере ту же самую последовательность случайных чисел, что получил я. Это нужно, чтобы вы в конце выполнения программы получили тот же самый ответ, что и я - 849442.

Эталонное решение. Сначала пишем код сами, проверяем на компьютере (как и все примеры программ из уроков), только потом сверяемся

```
ГСЧ = Новый ГенераторСлучайныхЧисел(2014);
```

```
// объявили генератор случайных чисел
```

```
А = ГСЧ.СлучайноеЧисло(1, 1000);
```

```
// сгенерировали первое случайное число от 1 до 1000
```

```
Б = ГСЧ.СлучайноеЧисло(1, 1000);
```

```
В = ГСЧ.СлучайноеЧисло(1, 1000);
```

```
А = Pow(А, 5); // возвели число в 5 степень
```

```
Б = Pow(Б, 5);
```

```
В = Pow(В, 5);
```

```
А = Sqrt(А); // вычислили квадратный корень из числа
```

```
Б = Sqrt(Б);
```

```
В = Sqrt(В);
```

```
A = A / 3;      // разделили на три
B = B / 3;
V = V / 3;
```

```
A = Окp(A, 2, 0); // округлили до второго знака
B = Окp(B, 2, 0);
V = Окp(V, 2, 0);
```

```
A = A * 0.25;    // умножили на 0.25
B = B * 0.25;
V = V * 0.25;
```

```
A = Цел(A);      // отбросили дробную часть
B = Цел(B);
V = Цел(V);
```

```
Сообщить(A + B + V); // сложили все три числа и вывели
                      // пользователю
```

ЮНИТ 19. ФОРМАТ В ЯЗЫКЕ 1С

При программировании в 1С часто приходится выводить (в те же отчеты) значения различных типов (строки, даты, числа ...). У каждого из значений есть различные представления.

К примеру, одну и ту же дату '01.01.2005' можно представить в виде строки как:

"01.01.2005"

"1 января 2005"

"01.01.05"

Все это строковые представления одного и того же значения, для формирования которых в 1С используется специальная функция *Формат*.

Отключаем группировку разрядов

Пусть нам требуется вывести число 10000.

Если мы напишем:

Сообщить(10000); // выводит 10 000

то число 10000 отобразится с пробелами 10 000. Так получается потому, что 1С по умолчанию группирует цифры по 3 штуки (начиная справа) и выводит пробелы между ними.

Чтобы отменить это правило, воспользуемся функцией *Формат*:

Сообщить(Формат(10000, "ЧГ=0")); // выводит 10000

Первый параметр — это само значение, а второй — форматная строка. Ее не нужно пытаться запомнить — просто нужно знать, где

посмотреть пример или справку по этой функции (во втором разделе пособия или в синтакс-помощнике).

Форматная строка в общем случае состоит из двух частей, разделенных знаком равно. Слева от равно стоит имя задаваемого параметра (смотрим в справке или на примерах), а справа — значение этого параметра.

В приведенном выше примере форматная строка "ЧГ=0" имеет параметр ЧГ и значение 0. Такая комбинация отменяет группировку разрядов числа. И, как вы можете убедиться, теперь выводится 10000.

Выводим лидирующие нули

Также частой задачей является вывод лидирующих нулей перед цифрой. К примеру, пусть требуется вывести число 5 с лидирующим впереди нулем, то есть в виде "05":

```
Сообщить(Формат(5, "ЧЦ=2; ЧВН=")); // выводит 05
```

Здесь составная форматная строка "ЧЦ=2; ЧВН=" состоит из двух форматных строк, разделенных точкой с запятой. Разберем каждую из них по отдельности.

Строка "ЧЦ=2" задает общее число отображаемых десятичных разрядов целой и дробной частей. Таким образом общее число позиций, которое будет занимать число при выводе, будет равно 2.

Строка "ЧВН=", как следует из справки, указывает функции формат, что, если число не дотягивает по длине до заявленного (как в нашем случае, ведь мы указали 2 позиции, а 5 занимает только одну), то следует использовать лидирующие нули. Особенность данной форматной строки в том, что она имеет только имя параметра и знак равно, но не имеет значения.

Комбинация двух форматных строк и дает нужный нам результат "05", вместо "5".

Отлично. Идем дальше.

Меняем разделитель целой и дробной части

Предположим, нам требуется выводить дробные числа с разделителем звездочка вместо точки. То есть, чтобы 25.46 выводилось как "25*46":

```
Сообщить(Формат(25.46, "ЧРД=*")); // выведет 25*46
```

Разберем форматную строку "ЧРД=*". ЧРД — это имя параметра, которое можно узнать в синтакс-помощнике, * — значение этого параметра. Оно и будет являться новым разделителем целой и дробной части вместо точки.

Общее представления даты

А что, если мы хотим вывести дату '20050101' в виде "01.01.2005":

```
Сообщить(Формат('20050101', "ДЛФ=Д")); // выведет 01.01.2005
```

А что, если мы хотим вывести дату как "1 января 2005 г.":

Сообщить(Формат('20050101', "ДФ=ДД")); // выведет 1 января 2005 г.

Обе используемые форматные строки "ДФ=Д" и "ДФ=ДД" имеют один и тот же параметр ДФ, но вот значения у него различные. В первом случае (сокращенный вариант даты) — Д, а во втором (длинный вариант) ДД.

Представление дня недели даты

Если посмотреть календарь, то выяснится, что 1 января 2005 года было субботой. Вот как можно выводить описание дня недели по его дате:

Сообщить(Формат('20050101', "ДФ=ддд"));

// выведет суббота

Форматная строка представляет из себя параметр ДФ и значение ддд, что и указывает функции *Формат* вывести длинное представление дня недели (обратите внимание, как много букв "д" она содержит).

Представление месяца даты

Описание месяца по дате выводится так:

Сообщить(Формат('20050101', "ДФ=ММММ"));

// выведет Январь

Форматная строка имеет тот же самый параметр ДФ, как и в предыдущем случае. Но вот значение отличается. Теперь оно равно ММММ.

Для отработки и закрепления материала данного юнита рекомендуется выполнить нижеследующее задание.

Напишите программу, которая:

Выводит 123.45 как "123\$45".

Выводит 9 как "009".

Выводит строковое представление текущего дня недели.

Выводит строковое представление текущего месяца.

Решение

Сообщить(Формат(123.45, "ЧРД = \$"));

Сообщить(Формат(9, "ЧЦ = 3; ЧВН ="));

Сообщить(Формат(ТекущаяДата(), "ДФ = ддд"));

Сообщить(Формат(ТекущаяДата(), "ДФ = ММММ"));

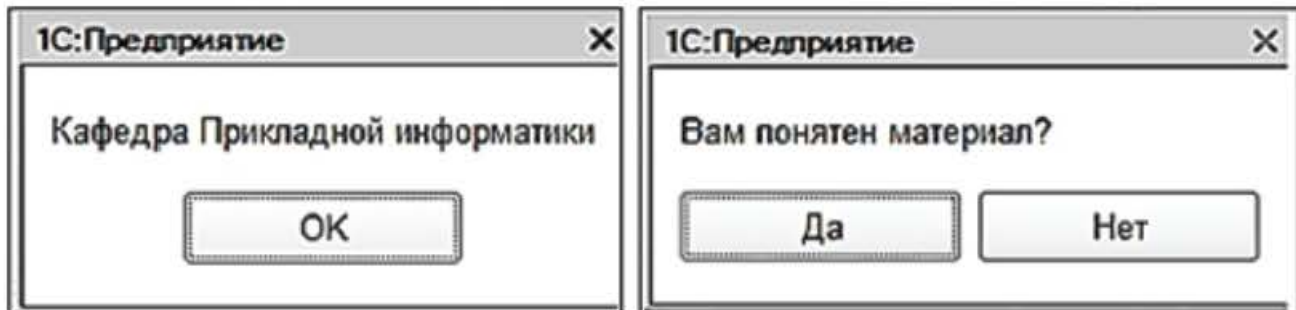
ЮНИТ 20. ИСПОЛЬЗОВАНИЕ ДИАЛОГОВ В ЯЗЫКЕ 1С

Давайте определимся, что же такое диалог? Вызов команды *Сообщить* является диалогом? А вызов команды *ВвестиЧисло*? А вызов *ОткрытьЗначение*?

Итак, диалог — это прежде всего элемент графического интерфейса, окно. А, следовательно, вызов *Сообщить* диалогом не является, так как у него нет своего отдельного окна.

Диалог — это не просто окно. Это окно, которое открывается пользователю с целью вывода информации и (или) получения ответа.

Примеры диалогов:



Для того, чтобы показать какой-либо диалог, необходимо вызвать команду, соответствующую этому диалогу, и передать ей необходимые параметры. Эта команда представляет собой обычную процедуру или функцию, только написанную не нами, а программистами фирмы 1С. Мы не можем менять эти команды, мы можем их только вызывать. Рассмотрим некоторые из них.

Предупреждение

Отличным примером диалога является вывод предупреждения пользователю:

`Предупреждение("Осторожно!");`

Что мы видим? Вызов команды *Предупреждение*, в которую в качестве параметра передается строка "Осторожно".

Когда компьютер дойдет до строчки, в которой вызывается эта команда, пользователю отобразится диалог предупреждения. И до тех пор, пока пользователь не закроет этот диалог, компьютер будет ожидать на этой же строчке. Только после закрытия диалога выполнение программы продолжится и начнется выполнение строки, следующей за вызовом диалога.

К примеру, пусть у нас есть такой код:

`Предупреждение("1");`

`Предупреждение("2");`

`Предупреждение("3");`

Как он будет выполняться?

Вначале компьютер дойдет до строчки №1 и выполнит команду *Предупреждение* с параметром "1". В этот момент пользователю покажется диалоговое окно с текстом "1", а компьютер замрет на этой строчке и будет ожидать, пока пользователь закроет диалог.

После закрытия диалога компьютер перейдет к выполнению строчки №2 и выполнит команду *Предупреждение* уже с другим па-

раметром "2". Пользователю снова покажется диалоговое окно, но уже с текстом "2", а компьютер замрет уже на второй строчке и будет ожидать, пока пользователь закроет диалог.

Затем компьютер перейдет к третьей строчке. Ну, и так далее.

У команды *Предупреждение* параметр, отвечающий за текст, который выводится — не единственный. Есть еще два дополнительных параметра:

Предупреждение(<ТекстПредупреждения>, <Таймаут>, <Заголовок>)

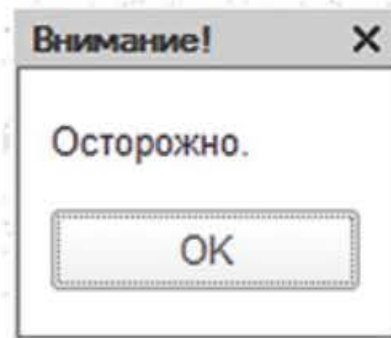
Таймаут — означает количество секунд, через которое диалог закроется сам, если этого не сделает пользователь.

Заголовок — подменяет стандартный заголовок "1С:Предприятие" на строку пользователя.

К примеру, такой код:

Предупреждение("Осторожно.", 5, "Внимание!");

покажет диалог с текстом "Осторожно." и заголовком "Внимание!", а также закроется через 5 секунд, если пользователь не сделает этого раньше:



А что если мы хотим задать только текст и заголовок окна, а таймаут не задавать?

Во-первых, мы можем передать в качестве таймаута ноль:

Предупреждение("Осторожно.", 0, "Внимание!");

А во-вторых, мы можем просто оставить пустое место вместо параметра, и система сама поймет, что нужно оставить его значение по умолчанию:

Предупреждение("Осторожно.", , "Внимание!");

Для того, чтобы узнавать обо всех возможных параметрах встроенных команд 1С, пользуйтесь синтакс-помощником так, как это описано в одном из предыдущих юнитов.

Ввод числа

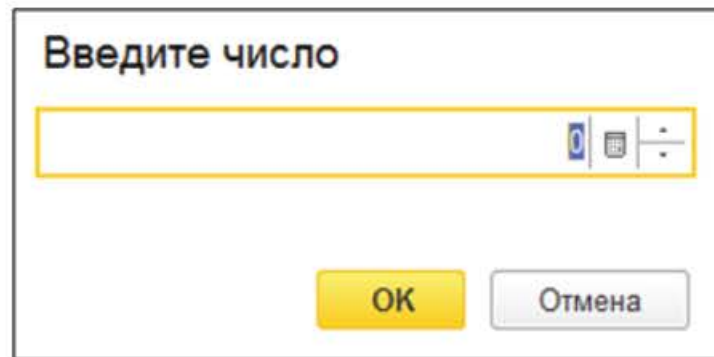
Некоторые диалоги не только выводят какую-то информацию от нас пользователю, но и, наоборот, возвращают некий результат вза-

и взаимодействия с пользователем. Примером такого диалога является ввод числа.

К примеру, напишем:

Число = 0;

ВвестиЧисло(Число);



Зачем мы передаем в команду параметр *Число*? Во-первых, для того, чтобы отобразить начальное значение ввода в диалоге. А, во-вторых, чтобы вернуть в эту переменную значение, которое ввел пользователь.

Но что если пользователь отказался от ввода и нажал кнопку "Отмена"? Как об этом узнать? Оказывается, функция *ВвестиЧисло* не только показывает диалог ввода числа, но еще и возвращает нам *Истина* в том случае, если пользователь нажал кнопку "ОК", и *Ложь*, если он отменил ввод, нажав кнопку "Отмена".

Таким образом, более правильным будет такой код:

Число = 0;

Результат = ВвестиЧисло(Число);

Если Результат = Истина Тогда

ОткрытьЗначение("Вы ввели " + Строка(Число));

Иначе

ОткрытьЗначение("Вы отменили ввод");

КонецЕсли;

В переменную *Результат* (имя могло быть любым) возвращается *Истина* или *Ложь* в зависимости от того, какую кнопку нажал пользователь ("ОК" или "Отмена").

В переменную *Число* возвращается число, введенное пользователем.

Вопрос

Заключительным диалогом, который мы рассмотрим, будет диалог вопроса пользователю.

Имя команды, которая вызывает этот диалог, нам известно — *Вопрос*.

Теперь давайте посмотрим в синтакс-помощнике, какие параметры принимает эта команда:

Вопрос(*<ТекстВопроса>*, *<Кнопки>*, *<Таймаут>*, *<КнопкаПоУмолчанию>*, *<Заголовок>*, *<КнопкаТаймаута>*)

ТекстВопроса — это сам вопрос в текстовом виде, который увидит пользователь.

Кнопки — это одно из predefined значений, которые также можно посмотреть в синтакс-помощнике. Например, если передать в этот параметр:

- *РежимДиалогаВопрос.ДаНет* — в диалоге будет всего две кнопки Да и Нет.
- *РежимДиалогаВопрос.ДаНетОтмена* — три кнопки Да, Нет и Отмена.
- *РежимДиалогаВопрос.ОкОтмена* — две кнопки Ок и Отмена.
- И другие варианты кнопок.

Таймаут — этот параметр нам уже знаком.

КнопкаПоУмолчанию — это кнопка, на которой будет фокус при показе окна. Ее значения могут быть такими:

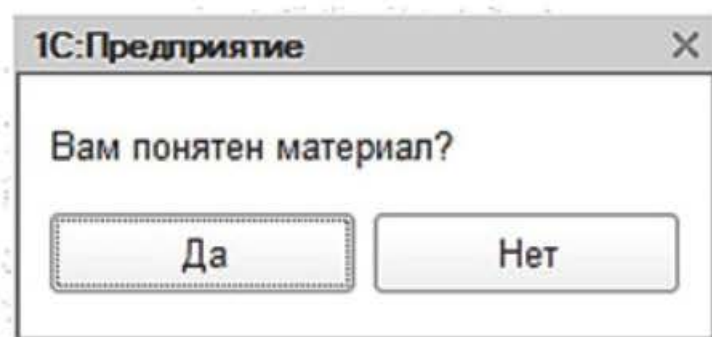
- *КодВозвратаДиалога.Нет*
- *КодВозвратаДиалога.Да*
- *КодВозвратаДиалога.Отмена*
- И другие кнопки.

Заголовок — этот параметр нам уже знаком.

КнопкаТаймаута — если задать значение этой кнопки и параметр *Таймаут*, то именно на ней будет отображаться количество оставшихся секунд до закрытия диалога. У нее могут быть такие же значения, как и у параметра *КнопкаПоУмолчанию*.

Как видите, параметров много. Но необязательно использовать их все. Как следует из синтакс-помощника, обязательными к заполнению являются только первые два:

Вопрос("Вам понятен материал?",
РежимДиалогаВопрос.ДаНет);



Вопрос-то мы задали. Хорошо бы теперь получить ответ — понятен материал или нет. Из того же синтакс-помощника мы узнаем, что кнопка, которая была нажата, возвращается в качестве результата вызова команды *Вопрос*:

Результат = Вопрос("Вам понятен материал?", РежимДиалогаВопрос.ДаНет);

Если Результат = КодВозвратаДиалога.Да Тогда

ОткрытьЗначение("Какой вы молодец!");

Иначе

ОткрытьЗначение("Выполните задания еще раз!");

КонецЕсли;

Модальность

Все примеры диалогов, которые мы рассмотрели, являются модалными, потому что компьютер, показывая диалог, замирает на той же строчке и не идет дальше, пока диалог не закроется.

Еще можно сказать, что, когда мы показываем модалный диалог, пользователь может работать только с этим диалогом. Работа со всей остальной частью системы блокируется, пока не закроется диалог.

Таким недостатком не обладают немодальные версии диалогов, но к ним переходить нам еще рано.

Для отработки и закрепления материала данного юнита рекомендуется выполнить нижеследующее задание.

Напишите программу, которая спрашивает пользователя "Нажмите Да, чтобы вывести числа от 1 до 10, Нет — чтобы вывести числа от 10 до 100, ни в коем случае не нажимайте кнопку Отмена". В диалоге должны быть три кнопки Да, Нет и Отмена.

Если пользователь нажал Да — выводим числа от 1 до 10, Нет — от 10 до 100, Отмена — выводим диалог предупреждения с текстом "Мы же вас предупреждали, что нельзя нажимать отмену!".

Решение

Результат = Вопрос(

"Нажмите Да, чтобы вывести числа от 1 до 10" +

", Нет — чтобы вывести числа 10 до 100, ни в коем" +

"случае не нажимайте кнопку Отмена", РежимДиалогаВопрос.ДаНетОтмена);

Если Результат = КодВозвратаДиалога.Да Тогда

Для Шаг = 1 По 10 Цикл

Сообщить(Шаг);

КонецЦикла;

ИначеЕсли Результат = КодВозвратаДиалога.Нет Тогда

Для Шаг = 10 По 100 Цикл

Сообщить(Шаг);

КонецЦикла;

```
ИначеЕсли Результат = КодВозвратаДиалога.Отмена Тогда  
    ОткрытьЗначение("Мы же вас предупреждали, что нельзя на-  
жимать отмену!");  
КонецЕсли;
```

ЮНИТ 21. СПИСОК ЗНАЧЕНИЙ В ЯЗЫКЕ 1С

В данном юните мы познакомимся с новым типом данных языка 1С, который называется *СписокЗначений*.

Список значений — это массив, но с большими возможностями. А раз так, то он тоже является коллекцией данных и в него можно поместить набор чисел:

```
Список = Новый СписокЗначений;  
// создали список значений  
Список.Добавить(10); // добавили в список число 10  
Список.Добавить(0); // добавили в список число 0  
Список.Добавить(-5);  
Список.Добавить(23);
```

Пока все очень похоже на массив, не правда ли? Но идем дальше. Чтобы теперь вывести эти числа пользователю, напишем:

```
// пробежимся циклом по каждому элементу списка  
Для Каждого Элемент Из Список Цикл  
    Сообщить(Элемент.Значение);  
    // выводим значение элемента  
КонецЦикла;
```

А вот и первое отличие. Элементами обычного массива являются сами числа. А доступ к текущему значению элемента *Списка* мы получаем с помощью опции *Значение*, используя конструкцию *Элемент.Значение*.

Рассмотрим некоторые дополнительные возможности *Списка Значений*.

Умеет сортировать

К, примеру, мы легко можем отсортировать список значений. *Список* это умеет делать сам:

```
Список.СортироватьПоЗначению(НаправлениеСортировки.Возр);
```

В случае с обычным массивом нам пришлось бы реализовывать один из алгоритмов сортировки (например, "пузырек").

Умеет искать

Список значений умеет сам искать свои элементы. Пусть мы хотим выяснить, присутствует ли введенное пользователем число в нашем списке:


```

Число = 0;
ВвестиЧисло(Число); // ввели число от пользователя
НайденныйЭлемент = Список.НайтиПоЗначению(Число);
Если НайденныйЭлемент = Неопределено Тогда
    ОткрытьЗначение("Число " + Число + " в списке не найдено!");
Иначе
    ОткрытьЗначение("Число " + Число + " присутствует в списке.
Вот оно: " + НайденныйЭлемент.Значение);
КонецЕсли; СписокЗначений

```

Поддерживает вставку и удаление

Список значений, также как и массив, поддерживает вставку и удаление элементов:

```

Список.Вставить(0, 1000);
// вставляем тысячу в самое начало списка
Список.Удалить(0); // и тут же удаляем ее из списка
Список.Вставить(Список.Количество(), 13);
// вставляем 13 в конец списка
Сообщить(Список[Список.Количество() — 1]);
// выводим последний элемент списка (13)

```

Поддерживает квадратные скобки

Как вы уже поняли, нумерация элементов списка также идет с нуля, и мы можем обратиться к любому элементу по его индексу, используя квадратные скобки:

```
Сообщить(Список[1].Значение); // вывели второй элемент списка
```

Таким образом, мы можем пробежаться по всем элементам списка еще и так:

```

Для А = 0 По Список.Количество() — 1 Цикл
    Сообщить(Список[А].Значение);
КонецЦикла;

```

Умеет создавать свою копию

У списка значений есть просто шикарная возможность — он умеет сам создавать свою копию:

```

КопияСписка = Список.Скопировать();
// сделали копию списка
// теперь можем смело менять КопияСписка
// при этом исходный список меняться не будет
КопияСписка[0].Значение = 123456;

```

Умеет превращаться в массив

Ну и, наконец, вы можете в любой момент легко выгрузить все элементы списка в массив:

```
МассивЧисел = Список.ВыгрузитьЗначения();  
// выгрузили в массив
```

```
Для А = 0 По МассивЧисел.Количество() - 1 Цикл  
    Сообщить(МассивЧисел[А]); // не нужно указывать // опцию  
Значение  
КонецЦикла;
```

Для отработки и закрепления материала данного юнита рекомендуется выполнить нижеследующее задание.

Необходимо ввести от пользователя 5 чисел, найти наибольшее и наименьшее из них и вывести их пользователю. В решении должен использоваться список значений.

Решение

```
Список = Новый СписокЗначений;  
// создали список значений  
Для А = 1 По 5 Цикл  
    Число = 0;  
    ВвестиЧисло(Число); // введем число от пользователя  
    Список.Добавить(Число); // и добавим его в список  
КонецЦикла; // и так 5 раз  
// сделаем сортировку списка чисел  
Список.СортироватьПоЗначению(НаправлениеСортировки.  
Возр);  
// после сортировки первый элемент списка — это и есть  
// наименьший элемент, а последний — наибольший  
ОткрытьЗначение("Самый маленький элемент " + Список[0] +  
", а самый большой " + Список[Список.Количество() — 1]);
```

Раздел 2 ОСНОВЫ ЯЗЫКА ПРОГРАММИРОВАНИЯ 1С 8.3 ДЛЯ ПРОДВИНУТЫХ ПОЛЬЗОВАТЕЛЕЙ (В ПРИМЕРАХ)

ЮНИТ 1. РАБОТА С ТИПОМ ДАТА В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Описание:

Значения данного типа содержит дату григорианского календаря (с 01 января 0001 года) и время с точностью до 0,1 миллисекунды.

Литералы:

Строка цифр, заключенная в одинарные кавычки вида: 'ГГГ-ГММДДччммсс', где:

- ГГГГ — четыре цифры года (включая тысячелетие и век);
- ММ — две цифры месяца;
- ДД — две цифры даты;
- чч — две цифры часа (в 24-х часовом формате);
- мм — две цифры минут;
- сс — две цифры секунд;

Во встроенном языке в литерале типа *Дата* обязательно должно задаваться значение года, месяца и дня. Для задания даты соответствующей началу отсчета достаточно указать '00010101'. Допускается при указании литералов типа *Дата* опускать последние символы (секунды, минуты, часы и т.д.). Это означает, что данные параметры будут равны нулю (для времени) или единице (для даты). В литерале даты допускается использование различных разделителей.

Процедура КнопкаВыполнитьНажатие(Кнопка)

/// Как инициализировать дату константой

Дата = '20130724'; // 24.07.2013

/// Как преобразовать строку в дату

Дата = Дата("20130724"); // 24.07.2013

/// Как инициализировать дату отдельными компонентами

Дата = Дата(2013, 07, 24); // 24.07.2013

/// Как указать в дате часы, минуты и секунды

Дата = '20130724132506'; // 24 июля 2013 г. 13 ч. 25
// мин. 6 сек.

/// Как узнать год (месяц, день, час, минута, секунда) у даты

Г = Год(Дата); // 2013

М = Месяц(Дата); // 7

```

Д = День(Дата); // 24
Ч = Час(Дата); // 13
Ми = Минута(Дата); // 25
С = Секунда(Дата); // 6
/// Как получить текущую дату
Сообщить(ТекущаяДата());
/// Операции с датой
Сообщить(ТекущаяДата() + 1); // прибавили секунду
Сообщить(ДобавитьМесяц(ТекущаяДата(), 1));
// прибавили месяц
Сообщить(ДобавитьМесяц(ТекущаяДата(), -1));
// отняли месяц
/// Вспомогательные функции для работы с датой
Дата = '20130110125905';
// 10 января 2013 года 12:59:05
Сообщить(ДеньГода(Дата)); // 10
Сообщить(ДеньНедели(Дата));
// 4, т.е. четверг (нумерация с понедельника)
Сообщить(НеделяГода(Дата)); // 2
Сообщить(НачалоГода(Дата)); // 01.01.2013 0:00:00
Сообщить(КонецГода(Дата)); // 31.12.2013 23:59:59
Сообщить(НачалоКвартала(Дата)); // 01.01.2013 0:00:00
Сообщить(КонецКвартала(Дата)); // 31.03.2013 23:59:59
Сообщить(НачалоМесяца(Дата)); // 01.01.2013 0:00:00
Сообщить(КонецМесяца(Дата)); // 31.01.2013 23:59:59
Сообщить(НачалоНедели(Дата)); // 07.01.2013 0:00:00
Сообщить(КонецНедели(Дата)); // 13.01.2013 23:59:59
Сообщить(НачалоДня(Дата)); // 10.01.2013 0:00:00
Сообщить(КонецДня(Дата)); // 10.01.2013 23:59:59
Сообщить(НачалоЧаса(Дата)); // 10.01.2013 12:00:00
Сообщить(КонецЧаса(Дата)); // 10.01.2013 12:59:59
Сообщить(НачалоМинуты(Дата)); // 10.01.2013 12:59:00
Сообщить(КонецМинуты(Дата)); // 10.01.2013 12:59:59
КонецПроцедуры

```

ЮНИТ 2. МАССИВЫ В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Описание:

Предназначен для доступа к элементам массива, его методам и конструктору.

Элементы коллекции: Произвольный

Для объекта доступен обход коллекции посредством оператора *Для каждого ... Из ... Цикл*. При обходе выбираются значения элементов массива.

Возможно обращение к значению элемента посредством оператора [...]. В качестве аргумента передается индекс значения (**нумерация с 0**).

Методы:

ВГраница()

Получает наибольший индекс элемента массива.

Вставить(<Индекс>, <Значение>)

Вставляет значение в массив по указанному индексу.

Добавить(<Значение>)

Добавляет элемент в конец массива.

Количество()

Получает количество элементов в массиве.

Найти(<Значение>)

Выполняет поиск элемента в массиве. Если элемент найден, возвращается его индекс. Если элемент не найден, возвращается *Неопределено*.

Очистить()

Удаляет все значения из массива.

Получить(<Индекс>)

Получает значение по индексу. Работает аналогично оператору [].

Удалить(<Индекс>)

Удаляет значение из массива по указанному индексу.

Установить(<Индекс>, <Значение>)

Устанавливает значение по индексу. Работает аналогично оператору [].

Конструкторы:

Новый Массив(<ФиксированныйМассив>)

На основании фиксированного массива.

Новый Массив(<КоличествоЭлементов1>, ..., <КоличествоЭлементовN>)

По количеству элементов. Создает массив из указанного количества элементов. Если задано несколько параметров, то будет создан массив, элементами которого являются массивы (и так далее в зависимости от количества параметров). Фактически конструктор позволяет создать массивы массивов, которые могут являться аналогом многомерного массива.

/// Как создать массив (сразу нужного размера)

Процедура Пример1()

// в массиве сразу 3 элемента

Массив = Новый Массив(3);

// инициализируем их значениями

Массив[0] = 5;

```

Массив[1] = Массив[0] * 2; // 10
Массив[2] = Массив[1] * 2; // 20
// выводим на печать
Для Каждого ЭлементМассива из Массив Цикл
    Сообщить(ЭлементМассива); // 5 10 20
КонецЦикла;
КонецПроцедуры

/// Как создать пустой массив (с последующим добавлением
/// элементов)
Процедура Пример2()
    // в массиве 0 элементов
    Массив = Новый Массив;
    // добавляем последовательно три элемента
    Массив.Добавить(100); // (100)
    Массив.Добавить(300); // (100, 300)
    Массив.Добавить(500); // (100, 300, 500)
    // выводим на печать
    Для Каждого ЭлементМассива из Массив Цикл
        Сообщить(ЭлементМассива); // 100 300 500
    КонецЦикла;
КонецПроцедуры

/// Как обойти все элементы массива по индексу

Процедура Пример3()
    // инициализируем массив: (100, 300, 500)
    Массив = Новый Массив;
    Массив.Добавить(100);
    Массив.Добавить(300);
    Массив.Добавить(500);
    // пробегаемся от первого (с индексом 0) до последнего
    // элемента (с индексом Количество - 1).
    Для Индекс = 0 По Массив.Количество() - 1 Цикл
        Сообщить(Массив[Индекс]);
    КонецЦикла;
КонецПроцедуры

/// Массив может содержать элементы различных типов:
/// числа, строки и т.д.

Процедура Пример4()
    Массив = Новый Массив(4);
    Массив[0] = "HELP"; // ("HELP")

```

```
Массив[1] = "ME"; // ("HELP", "ME")
Массив[2] = "1С"; // ("HELP", "ME", "1С")
Массив[3] = Формат(2013, "ЧГ=0"); // ("HELP", "ME", "1С", 2016)
```

```
    Для Каждого ЭлементМассива из Массив Цикл
        Сообщить(ЭлементМассива); // HELP ME 1С 2016
    КонецЦикла;
КонецПроцедуры
```

/// Как передать массив в качестве параметра функции

```
Функция Метод5(массивПереданныйПоСсылке)
    массивПереданныйПоСсылке[0] = 10;
    Возврат массивПереданныйПоСсылке[0] * 2;
КонецФункции
```

```
Процедура Пример5()
    // инициализация массива из одного элемента: (13)
    Массив = Новый Массив(1);
    Массив[0] = 13;
    // значение первого элемента до вызова функции
    Сообщить(Массив[0]); // 13
    // массив ВСЕГДА передается в функцию
    // ПО ССЫЛКЕ, значит, изменяя массив внутри
    // функции, мы меняем его снаружи
    Сообщить(Метод5(Массив)); // 20
    // значение первого элемента после вызова
    // функции изменилось
    Сообщить(Массив[0]); // 10
КонецПроцедуры
```

**/// Как вернуть массив в качестве результата работы
/// функции**

```
Функция Метод6()
    // инициализация массива (СПАСИБО, ВАМ)
    массив = Новый Массив(2);
    массив[0] = "СПАСИБО";
    массив[1] = "ВАМ";
    Возврат массив;
КонецФункции
```

```
Процедура Пример6()
    // инициализация переменной массивом, который
    // возвращает и формирует функция Метод6
```

```

Массив = Метод6();
// вывод на печать
Для Каждого ЭлементМассива из Массив Цикл
    Сообщить(ЭлементМассива); // СПАСИБО ВАМ
КонецЦикла;
КонецПроцедуры

```

**/// Еще пример на передачу массива в качестве параметра
/// процедуры**

```

Процедура Метод7(Массив)
    Если Массив <> Неопределено И Массив.Количество() > 0
Тогда
    Сообщить(Массив[0]);
    КонецЕсли;
КонецПроцедуры

```

```

Процедура Пример7()
    // инициализация массива: (10, 20)
    Массив = Новый Массив(2);
    Массив[0] = 10;
    Массив[1] = 20;
    // печать первого элемента, переданного массива
    Метод7(Массив); // 10
    Метод7(Неопределено); // ничего
    Метод7(Новый Массив); // ничего
КонецПроцедуры

```

/// Как получить последний элемент массива

```

Процедура Пример8()
    // инициализация массива: ("Кошка", "Собака",
    // "Пантера", "Тигр")
    Массив = Новый Массив;
    Массив.Добавить("Кошка");
    Массив.Добавить("Собака");
    Массив.Добавить("Пантера");
    Массив.Добавить("Тигр");
    Сообщить(Массив[Массив.Количество() - 1]); // Тигр
КонецПроцедуры

```

/// Использование многомерных массивов

```

Процедура Пример9()
    // ((0, 0), (0, 0), (0, 0))

```



```

    Массив = Новый Массив(3, 2);
    // ( ( "Юрий", "Гагарин" ), ( (), () ), ( (), () ) )
    Массив[0][0] = "Юрий";
    Массив[0][1] = "Гагарин";
    // ( ( "Юрий", "Гагарин" ), ( "Герман", "Титов" ), ( (), () ) )
    Массив[1][0] = "Герман";
    Массив[1][1] = "Титов";
    // ( ( "Юрий", "Гагарин" ), ( "Герман", "Титов" ),
    // ( "Валентина", "Терешкова" ) )
    Массив[2][0] = "Валентина";
    Массив[2][1] = "Терешкова";
    // обход всех элементов через конструкцию Для Каждого
    Для Каждого Строка Из Массив Цикл
        Для Каждого Элемент Из Строка Цикл
            Сообщить(Элемент);
        КонецЦикла;
        Сообщить(" ");
    КонецЦикла;
    Сообщить(" ");
    // обход всех элементов через индекс
    Для ИндексСтрока = 0 По Массив.Количество() - 1 Цикл
        Для ИндексСтолбец = 0 По Массив[ИндексСтрока].Количество()
        - 1 Цикл
            Сообщить(Массив[ИндексСтрока][ИндексСтолбец]);
            КонецЦикла;
            Сообщить(" ");
        КонецЦикла;
    КонецПроцедуры

```

/// Вставка, удаление и очистка массива

Процедура Пример10()

```

    // инициализация пустого массива
    Массив = Новый Массив;

```

```

    // вставка трех элементов; каждый элемент
    // вставляется в начало

```

```

    Массив.Вставить(0, 10); // (10)

```

```

    Массив.Вставить(0, 100); // (100, 10)

```

```

    Массив.Вставить(0, 1000); // (1000, 100, 10)

```

```

    // определение последнего индекса

```

```

    Сообщить(Массив.ВГраница()); // 2

```

```

    // вывод на печать

```

```

    Для Индекс = 0 по Массив.ВГраница() Цикл

```

```

        Сообщить(Массив[Индекс]); // 1000 100 10
    КонецЦикла

```

```

КонецЦикла;
// удаление элемента со значением 100
// для этого сначала находим индекс элемента
// если не находим возвращаем Неопределено
Индекс = Массив.Найти(100); // 1
// и удаляем элемент по найденному индексу
Массив.Удалить(Индекс); // (100, 10)
Сообщить(Массив.ВГраница()); // 1
// удаление всех элементов из массива
Массив.Очистить();
КонецПроцедуры
/// Как из обычного получить фиксированный
/// (неизменяемый) массив

Процедура Пример11()
    Числа = Новый Массив;
    Числа.Добавить(1);
    Числа.Добавить(2);
    Числа.Добавить(3);
    ФиксированныеЧисла = Новый ФиксированныйМассив(Числа);
    // Получился константный массив на основе обычного:
    // нельзя менять значения имеющихся элементов
    // нельзя добавлять новые элементы
    // нельзя удалять имеющиеся элементы
КонецПроцедуры

```

ЮНИТ 3. МАТЕМАТИКА В ЯЗЫКЕ 1С

Выполнение многочисленных, часто применяемых простых математических операций рассмотрены на конкретных нижеследующих примерах.

```

Процедура Пример()

    /// Как округлить число
    // до второго знака после запятой
    Сообщить(Окр(321.123, 2)); // 321.12
    // до второго знака перед запятой
    Сообщить(Окр(321.123, -2)); // 300
    // по умолчанию округление работает
    // в большую сторону
    Сообщить(Окр(1.5, 0)); // 2
    // а если указать третьим параметром 0,
    // то в меньшую
    Сообщить(Окр(1.5, 0, 0)); // 1

```

```

/// Как получить целую часть числа
Сообщить (Цел(12.654)); // 12

/// Как возвести число в степень
Сообщить (Pow(10, 3)); //  $10^3 = 1000$ 

/// Как извлечь квадратный корень из числа
Сообщить (Sqrt(25)); // 5

/// Как вычислить максимальное из
/// нескольких значений
Сообщить (Макс(1, 2, 3)); // 3
Сообщить (Макс("aaa", "bbb", "ccc")); // ccc
Сообщить (Макс(ТекущаяДата(),
ДобавитьМесяц(ТекущаяДата(), 1)));
Сообщить (Макс(Истина, Ложь)); // Истина

/// Как вычислить минимальное из
/// нескольких значений
Сообщить (Мин(1, 2, 3)); // 1
Сообщить (Мин("aaa", "bbb", "ccc")); // aaa
Сообщить (Мин(ТекущаяДата(),
ДобавитьМесяц(ТекущаяДата(), 1)));
Сообщить (Мин(Истина, Ложь)); // Ложь

/// Как вычислить выражение из строки
Сообщить (Вычислить("1+2")); // 3
Сообщить (Вычислить("ИСТИНА И ЛОЖЬ")); // Нет
Сообщить (Вычислить("Sin(3.14)")); // ~0
Сообщить (Вычислить("Pow(Sin(3.14), 2) + Pow
(Cos(3.14), 2)")); // ~1

/// Тригонометрия
// углы выражаются в радианах
Пи = 3.14;
Е = 2.718; // Число Эйлера
Сообщить (Cos(Пи / 2)); // ~0
Сообщить (ACos(0)); // ~1,57
Сообщить (Sin(Пи / 2)); // ~1
Сообщить (ASin(1)); // ~1,57
Сообщить (Sin(Пи / 2) * Sin(Пи / 2) +
Cos(Пи / 2) * Cos(Пи / 2)); // ~1
Сообщить (Sin(Пи / 3) / Cos(Пи / 3)); // 1.73
Сообщить (Tan(Пи / 3)); // 1.73

```

```

Сообщить (Tan(ATan(1.73))); // 1.73
// экспонента – возведение числа Эйлера
// в степень X
// http://ru.wikipedia.org/wiki/Экспонента
Сообщить (Pow(E, 2)); // 7,389
Сообщить (Exp(2)); // 7,389
// натуральный логарифм – степень в которую надо
// возвести число Эйлера e, чтобы получить X
Сообщить (Log(9)); // 2,197
Сообщить (Pow(E, 2.197)); // ~9
// десятичный логарифм – степень в которую надо
// возвести число 10, чтобы получить X
Сообщить (Log10(1000)); // 3
Сообщить (Pow(10, 3)); // 1000

/// Как вывести число без пробелов
Сообщить (Формат(1000000, "ЧГ=0")); // 1000000

КонецПроцедуры

```

ЮНИТ 4. ПРОЦЕДУРЫ В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Описание:

Ключевое слово *Процедура* начинает секцию исходного текста, выполнение которого можно инициировать из любой точки программного модуля, просто указав Имя_процедуры со списком параметров (если параметры не передаются, то круглые скобки, тем не менее, обязательны). Если в модуле обычного приложения, модуле управляемого приложения или общем программном модуле в теле описания процедуры использовано ключевое слово *Экспорт*, то это означает, что данная процедура является доступной из всех других программных модулей конфигурации.

При выполнении оператора *Возврат* процедура заканчивается и возвращает управление в точку вызова. Если в тексте процедуры не встретился оператор *Возврат*, то после выполнения последнего исполняемого оператора происходит выполнение неявного оператора *Возврат*. Конец программной секции процедуры определяется по оператору *КонецПроцедуры*.

Переменные, объявленные в теле процедуры в разделе *Объявления локальных переменных*, являются локальными переменными данной процедуры, поэтому доступны только в этой процедуре (за исключением случая передачи их как параметров при вызове других процедур, функций или методов).

/// Как передать в процедуру параметр по ссылке

Процедура П1(ПараметрПереданныйПоСсылке)

// изменяем значение и внутри и снаружи
ПараметрПереданныйПоСсылке = 25;

КонецПроцедуры

/// Как передать в процедуру параметр по значению в языке

/// 1С

// возможно только для простых типов (число, строка, дата, ...)

Процедура П2(Знач ПараметрПереданныйПоЗначению)

// изменяем значение только внутри процедуры
ПараметрПереданныйПоЗначению = 25;

КонецПроцедуры

/// Как передать в процедуру массив по значению

// это можно сделать только через создание копии массива

Процедура П3(Параметр)

КонецПроцедуры

Процедура П4()

Числа = Новый Массив();
КопияЧисел = Новый СписокЗначений;
КопияЧисел.ЗагрузитьЗначения(Числа);
П3(КопияЧисел.ВыгрузитьЗначения());

КонецПроцедуры

/// Как для параметра процедуры установить значение

/// по умолчанию

Процедура П5(Имя = "Сергей", Фамилия, Отчество = "не задано")

КонецПроцедуры

Процедура П6()

П5("Вадим", "Иванович", "Сидоров"); // Вадим
// Иванович Сидоров

```
П5("Вадим", "Иванович"); // Вадим Иванович не задано
П5(, "Иванович"); // Сергей Иванович не задано
П5(,,); // Сергей Неопределено не задано
КонецПроцедуры
```

```
/// Как в процедуре объявить локальные переменные
///
Процедура П7()
    Перемен пр1, пр2; // объявляем в самом начале
КонецПроцедуры
```

```
/// Как сделать процедуру доступной из других модулей
Процедура П8() Экспорт
КонецПроцедуры
```

```
/// Как вернуть значение из функции
```

```
Функция Ф1()
    Возврат 1;
КонецФункции
```

ЮНИТ 5. СТРОКИ В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Описание:

Значения данного типа содержат строку в формате Unicode произвольной длины.

Литералы:

Литералы строкового типа представляют собой набор символов заключенных в кавычки. Для задания в строке символа " (кавычка) необходимо записать две кавычки подряд. Кроме того, допускаются "многострочные" строковые константы. В исходном тексте многострочные константы могут задаваться двумя способами:

- Между фрагментами, представляющими отдельные строки многострочной строки, не должно встречаться никаких символов, за исключением пробелов, переводов строки и строк комментариев.
- Каждая отдельная составляющая не замыкается кавычками, а на каждой последующей строке помещен символ переноса строки | (вертикальная черта). В этом варианте комментарии допускаются, если строка начинается с символа комментария //.

Процедура Пример ()

/// Как найти длину строки

```
Сообщить(СтрДлина("123456789")); // 9
```

/// Как взять часть строки

```
// как взять N символов слева
```

```
Строка = "Иван Грозный";
```

```
Сообщить(Лев(Строка, 4)); // Иван
```

```
// как взять N символов справа
```

```
Строка = "Иван Грозный";
```

```
Сообщить(Прав(Строка, 7)); // Грозный
```

```
// как взять строку из середины
```

```
Строка = "Мама, папа, я.";
```

```
// возьмем, начиная с 7 индекса 4 символа
```

```
Сообщить(Сред(Строка, 7, 4)); // папа
```

/// Как найти подстроку

```
Строка = "Привет, мир!";
```

```
Индекс = Найти(Строка, "Привет"); // 1, строки
```

```
// индексируются с единицы
```

/// Как узнать число вхождений подстроки в строку

```
Сообщить(СтрЧислоВхождений("я, я и еще раз я.", "я")); // 3
```

/// Как заменить подстроку в строке

```
Сообщить(СтрЗаменить("Идет сильный дождь.", "дождь",  
"снег"));
```

/// Как проверить на пустую строку

```
Если ПустаяСтрока(" ") Тогда
```

```
Сообщить("Строка пустая, то есть не содержит  
| значащих символов.");
```

```
КонецЕсли;
```

/// Как удалить незначащие символы из строки

```
// слева
```

```
Сообщить(СокрЛ(" из лесу елочку "));
```

```
// "из лесу елочку "
```

```
// справа
```

```
Сообщить(СокрП(" из лесу елочку "));
```

```
// " из лесу елочку"
```

```
// с двух сторон
```

```
Сообщить(СокрЛП(" из лесу елочку "));
```

```
// "из лесу елочку"
```

/// Как вставить в строку перенос

```
Строка =  
"Первая строка" + Символы.ПС +  
"Вторая строка" + Символы.ПС +  
"Третья строка";
```

/// Как работать с многострочными строками (нумерация с /// единицы)

```
Сообщить(СтрЧислоСтрок(Строка)); // 3  
Сообщить(СтрПолучитьСтроку(Строка, 2));  
// "Вторая строка"
```

/// Верхний, нижний и титульный регистры

```
Сообщить(ВРег("россия - родина моя!"));  
// РОССИЯ - РОДИНА МОЯ!  
Сообщить(НРег("МИР, ТРУД, МАЙ!"));  
// мир, труд, май  
Сообщить(ТРег("мама мыла раму"));  
// Мама Мыла Раму
```

/// Как получить код символа строки

```
// код отдельного символа:  
// туда  
Сообщить(КодСимвола("А")); // 65  
// и обратно  
Сообщить(Символ(65)); // "А"  
// код третьего символа в строке  
Сообщить(КодСимвола("АВС", 3)); // 67  
// код первого символа в строке  
КодПервогоСимволаФамилии = КодСимвола("Грозный");  
Сообщить(КодПервогоСимволаФамилии);  
// 1043, потому что Unicode.
```

/// Как преобразовать строку в число

```
Сообщить(Число("125.36")); // 125,36  
Сообщить(Число("125,36")); // 125,36
```

/// Как преобразовать число в строку

```
Сообщить(25 + "10"); // 35  
Сообщить(Строка(25) + "10"); // 2510
```

/// Как разбить строку с разделителями

```
Разделитель = ",";  
Строки = СтрЗаменить("январь, февраль, март", Разделитель,  
Символы.ПС);
```



```
Для Индекс = 1 По СтрЧислоСтрок(Строки) Цикл
    Сообщить(СтрПолучитьСтроку(Строки, Индекс));
КонецЦикла;
```

КонецПроцедуры

ЮНИТ 6. ЦИКЛЫ (В ПРИМЕРАХ)

В примерах ниже представлены возможные варианты практического решения различных вариантов организации циклов.

Процедура Пример()

/// Как организовать цикл

```
// Для Цикл
Для Счетчик = 1 По 5 Цикл
    Сообщить(Счетчик); // 1 2 3 4 5
КонецЦикла;
// Для Каждого Цикл
Дни = Новый Массив();
Дни.Добавить("Понедельник");
Дни.Добавить("Вторник");
Дни.Добавить("Среда");
Для Каждого Элемент Из Дни Цикл
    Сообщить(Элемент);
// Понедельник Вторник Среда
КонецЦикла;
// Пока Цикл
Счетчик = 0;
Пока Счетчик < Дни.Количество() Цикл
    Сообщить(Дни[Счетчик]);
// Понедельник Вторник Среда
Счетчик = Счетчик + 1;
КонецЦикла;
```

/// Как организовать обратный цикл

```
Счетчик = Дни.Количество() - 1;
Пока Счетчик >= 0 Цикл
    Сообщить(Дни[Счетчик]);
// Среда Вторник Понедельник
Счетчик = Счетчик - 1;
КонецЦикла;
```

/// Как прервать цикл в 1с

```
Для Счетчик = 1 По 5 Цикл
    Если Счетчик > 2 Тогда
```

```
    Прервать;  
    КонецЕсли;  
    Сообщить(Счетчик); // 1 2  
КонецЦикла;
```

/// Как принудительно продолжить цикл

```
Для Счетчик = 1 По 5 Цикл  
    Если Счетчик <> 3 Тогда  
        Продолжить;  
    КонецЕсли;  
    Сообщить(Счетчик); // 3  
КонецЦикла;
```

КонецПроцедуры

ЮНИТ 7. ОПИСАНИЕ ТИПОВ В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Описание:

Представляет собой объект для управления допустимыми типами значений. В основном он предназначен для присвоения в качестве значений соответствующим свойствам других объектов. Набор допустимых типов и квалификаторы примитивных типов задаются при создании объекта.

Свойства:

КвалификаторыДаты

Содержит квалификаторы даты — объект, используемый для описания допустимых значений типа *Дата*.

КвалификаторыДвоичныхДанных

Содержит квалификаторы двоичных данных.

КвалификаторыСтроки

Содержит квалификаторы строки, используемые для описания допустимых значений строкового типа.

КвалификаторыЧисла

Содержит квалификаторы числа, используемые для описания допустимых значений числового типа.

Методы:

ПривестиЗначение(<Значение>)

Формирует значение на основании описания типа и переданного значения. Если значение передано, то выполняется проверка на соответствие значения описанию типа. Если значение соответствует, то метод возвращает переданное значение. Если значение не соответствует, то оно приводится к типу, доступному в описании типа.

Если значение не передано, то выдается значение по умолчанию для данного описания типа.

СодержитТип(<Тип>)

Определяет, входит ли указанный тип в используемые данным объектом типы.

Типы()

Получает массив типов, используемых в объекте.

Конструкторы:

Новый ОписаниеТипов(<ИсходноеОписаниеТипов>, <ДобавляемыеТипы>, <ВычитаемыеТипы>, <КвалификаторыЧисла>, <КвалификаторыСтроки>, <КвалификаторыДаты>, <КвалификаторыДвоичныхДанных>)

Создает описание типов на основании другого описания типов путем добавления одних и исключения других типов. Если новый квалификатор не указан, будут оставлены квалификаторы исходного описания типов.

Новый ОписаниеТипов(<Типы>, <КвалификаторыЧисла>, <КвалификаторыСтроки>, <КвалификаторыДаты>, <КвалификаторыДвоичныхДанных>)

Создает описание типов на основании заданных типов и квалификаторов для типов Число, Строка, Дата.

Процедура Пример()

/// Как сделать описание типа строки (числа, даты, булево) в 1С

```
ОписаниеЧисла = Новый ОписаниеТипов("Число");  
ОписаниеСтроки = Новый ОписаниеТипов("Строка");  
ОписаниеДаты = Новый ОписаниеТипов("Дата");  
ОписаниеБулева = Новый ОписаниеТипов("Булево");
```

/// Как сделать описание типа структуры (соответствия, массива) в 1С

```
ОписаниеСтруктуры =  
Новый ОписаниеТипов("Структура");  
ОписаниеСоответствия =  
Новый ОписаниеТипов("Соответствие");
```

```
ОписаниеМассива =  
Новый ОписаниеТипов("Массив");
```

/// Как сделать описание типа справочника в 1С

```
ОписаниеКонтрагента =  
Новый ОписаниеТипов("СправочникСсылка.Контрагенты");
```

/// Как сделать описание составного типа в 1С

```
СписокТипов = Новый Массив;  
СписокТипов.Добавить("Строка");  
СписокТипов.Добавить("Структура");  
СписокТипов.Добавить("СправочникСсылка.Контрагенты");  
ОписаниеСоставногоТипа =  
Новый ОписаниеТипов(СписокТипов);
```

**/// Как уточнить описание числа, строки, даты при помощи
/// квалификаторов в 1С**

```
// общее число разрядов - 10, число дробной части - 2  
КвалификаторыЧисла =  
Новый КвалификаторыЧисла(10, 2, ДопустимыйЗнак.Любой);  
ОписаниеЧисла =  
Новый ОписаниеТипов("Число", КвалификаторыЧисла);  
// максимальная длина строки - 20 символов  
КвалификаторыСтроки =  
Новый КвалификаторыСтроки(20);  
ОписаниеСтроки =  
Новый ОписаниеТипов("Строка", КвалификаторыСтроки);  
// храниться только дата, без времени  
КвалификаторыДаты =  
Новый КвалификаторыДаты(ЧастиДаты.Дата);  
ОписаниеДаты =  
Новый ОписаниеТипов("Дата", КвалификаторыДаты);
```

КонецПроцедуры

ЮНИТ 8. РАБОТА С ОПЕРАЦИОННОЙ СИСТЕМОЙ В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Рассмотрим выполнение некоторых часто применяемых операций с операционной системой, на конкретных примерах.

Процедура Пример()

/// Как запустить внешнюю программу из приложения в 1С

```
Перем КодВозврата;  
ЗапуститьПриложение(  
    "calc.exe",  
    "", // текущий каталог для запускаемого приложения  
    Истина, // дождаться завершения работы запущенного приложения  
);
```



```

    КодВозврата // сюда попадет код, возвращаемый программой
);
// или так
ЗапуститьПриложение("http://www.fa.ru");

/// Как выполнить команду операционной системы из
/// приложения в 1С
КомандаСистемы(
    "winver", // выполняется через командный
               //интерпретатор windows
    ""        // текущий каталог
);

/// Как получить список пользователей операционной
/// системы в 1С
ТаблицаПользователей = ПользователиОС();
Сообщить(ЗначениеВСтрокуВнутри(ТаблицаПользователей));

/// Как получить имя компьютера в 1С
Сообщить(ИмяКомпьютера());

/// Как узнать текущий язык, используемый в приложении
/// в 1С
Сообщить(ТекущийЯзыкСистемы()); // ru

КонецПроцедуры

```

ЮНИТ 9. ТИП *СООТВЕТСТВИЕ* В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Описание:

Представляет доступ к соответствию. Не используется в данных формы.

Элементы коллекции: *КлючИЗначение*

Для объекта доступен обход коллекции посредством оператора *Для каждого ... Из ... Цикл*. При обходе выбираются элементы соответствия. Возможно обращение к значению элемента посредством оператора [...]. В качестве аргумента передается значение ключа элемента.

Методы:

Вставить(<Ключ>, <Значение>)

Устанавливает значение элемента соответствия по ключу. Если элемент с переданным значением ключа существует, то его значение заменяется, в противном случае добавляется новый элемент.

Количество()

Получает количество элементов соответствия.

Очистить()

Удаляет все элементы соответствия.

Получить(<Ключ>)

Получает значение по ключу. Работает аналогично оператору [].

Удалить(<Ключ>)

Удаляет элемент соответствия с заданным ключом.

Конструкторы:

Новый Соответствие(<ФиксированноеСоответствие>)

Создаваемое соответствие заполняется элементами из исходного фиксированного соответствия.

Новый Соответствие()

Создает пустое соответствие (без элементов).

Процедура Пример()

/// Как создать соответствие в 1С

СобытияДаты = Новый Соответствие;

// пара: ключ (произвольного типа) -

// значение (произвольного типа)

СобытияДаты.Вставить(1980, "Олимпиада в Москве");

СобытияДаты.Вставить(1961, "Первый полет человека в космос.");

/// Как получить значение соответствия по ключу в 1С

Сообщить(СобытияДаты.Получить(1980));

// "Олимпиада в Москве"

/// Как узнать, есть ли соответствие значения ключу в 1С

Если СобытияДаты.Получить(2014) = Неопределено

Тогда

Сообщить("В 2014 ничего не произошло.");

КонецЕсли;

/// Как изменить значение ключа созданного соответствия

/// в 1С

СобытияДаты.Вставить(1961, "Первый полет Гагарина в космос.");

/// Как узнать количество элементов соответствия в 1С

Сообщить(СобытияДаты.Количество());

/// Как обойти все элементы соответствия в 1С

Для Каждого Элемент Из СобытияДаты Цикл
Сообщить(Строка(Элемент.Ключ) + " " + Элемент.Значение);
КонецЦикла;

/// Как удалить элемент соответствия в 1С

СобытияДаты.Удалить(1980);

/// Как очистить все элементы соответствия в 1С

СобытияДаты.Очистить();

КонецПроцедуры

ЮНИТ 10. СПИСОК ЗНАЧЕНИЙ В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Описание:

Список значений — это не сохраняемый в базе данных объект, который позволяет строить для решения интерфейсных задач динамические наборы значений и манипулировать ими (добавлять, редактировать, удалять элементы, сортировать). Он может быть наполнен значениями любого типа, т.е. в одном списке типы хранимых значений могут быть разными. Одним из примеров использования данного объекта может служить организация выбора конкретного документа из списка возможных документов, сформированного по сложному алгоритму.

Элементы коллекции: *ЭлементСпискаЗначений*

Для объекта доступен обход коллекции посредством оператора *Для каждого ... Из ... Цикл*. При обходе выбираются элементы коллекции.

Возможно обращение к элементу коллекции посредством оператора [...]. В качестве аргумента передается индекс элемента (**нумерация с 0**).

Свойства:

ДоступныеЗначения

Применяется в случаях, когда требуется ограничить возможность задания значений для элементов списка некоторым подмножеством значений допустимого типа (если для списка установлено описание типа). *Неопределено* — ограничения отсутствуют.

Установка этого свойства влияет на работу диалога редактирования списка значений, элементов управления "Таблица" и "Поле вво-

да", связанных со списком значений, а также на программное добавление-вставку нового элемента списка значений и установку значений элементам списков.

ТипЗначения

Определяет тип для значений, которые могут храниться в элементах данного списка значений.

Методы:

Вставить(<Индекс>, <Значение>, <Представление>, <Пометка>, <Картинка>)

Вставляет новый элемент в список значений на позицию с указанным индексом.

ВыбратьЭлемент(<Заголовок>, <Элемент>)

Вызывает окно для интерактивного выбора одного из элементов, входящих в список значений.

Элемент — элемент, на который следует изначально спозиционировать список значений при интерактивном выборе. Если значением параметра не является элемент списка значений, входящий в данный список, позиционирования не произойдет.

ВыгрузитьЗначения()

Создает массив и копирует в него значения элементов списка значений.

Добавить(<Значение>, <Представление>, <Пометка>, <Картинка>)
Добавляет новый элемент в конец списка значений.

ЗагрузитьЗначения(<МассивЗначений>)

Загружает список значений значениями из переданного массива. При этом все прежние элементы списка удаляются.

ЗаполнитьПометки(<Пометка>)

Устанавливает пометку у всех элементов списка значений.

Индекс(<Элемент>)

Получает индекс элемента в списке значений. Если не найдено, то возвращается -1.

Количество()

Получает количество элементов списка значений.

НайтиПоЗначению(<ИскомоеЗначение>)

Осуществляет поиск значения элемента списка значений. Если ни один элемент не хранит значения, равного искомому, то возвращается значение *Неопределено*.

НайтиПоИдентификатору(<Идентификатор>)

Получает элемент списка значений по идентификатору. Если элемент не найден, то возвращается *Неопределено*.

ОтметитьЭлементы(<Заголовок>)

Устанавливает или снимает (интерактивно) пометки у элементов списка значений. Возвращает *Истина*, если в диалоге нажата кнопка "ОК", *Ложь* — в противном случае.

Очистить()

Очищает список значений, удаляя из него все элементы.

ПоказатьВыборЭлемента(<ОписаниеОповещенияОЗакрытии>, <Заголовок>, <Элемент>)

Вызывает окно для интерактивного выбора одного из элементов, входящих в список значений.

ПоказатьОтметкуЭлементов(<ОписаниеОповещенияОЗакрытии>, <Заголовок>)

Предназначен для интерактивной установки состояний пометок у элементов списка значений.

Получить(<Индекс>)

Получает значение по индексу. Работает аналогично оператору [].

Сдвинуть(<Элемент>, <Смещение>)

Сдвигает элемент списка значений на заданное количество позиций вперед или назад.

Скопировать()

Создает копию списка значений.

СортироватьПоЗначению(<Направление>)

Сортирует список значений в порядке возрастания или убывания хранимых элементами значений.

СортироватьПоПредставлению(<Направление>)

Сортирует список значений по возрастанию или по убыванию в алфавитном порядке представлений входящих в состав списка значений элементов.

Удалить(<Элемент>)

Удаляет элемент из списка значений, где *Элемент* — индекс удаляемого элемента либо сам элемент.

Конструкторы:

Новый СписокЗначений

Процедура Пример()

/// Как создать список значений в 1С

Список = Новый СписокЗначений();

/// Как добавить элемент в список значений в 1С

// параметры: значение, представление, (необязательно)

// пометка, (необязательно) картинка

Список.Добавить(1980, "Год рождения Васи");

Список.Добавить(1985, "Год рождения Юли");

// значения могут быть разных типов

Список.Добавить("Полина", "Имя ребенка");

```

/// Как вставить элемент в список значений в 1С
// на вторую позицию
Список.Вставить(2, 2010, "Год рождения их совместной дочери");

/// Как обойти элементы списка значений в 1С
Для Каждого Элемент Из Список Цикл
    Сообщить(Элемент.Представление + ": " +
        Строка(Элемент.Значение));
КонецЦикла;

/// Как очистить список значений в 1С
Список.Очистить();
Список.Добавить("Понедельник");
Список.Добавить("Вторник");
Список.Добавить("Среда");

/// Как узнать количество элементов списка значений,
/// а также получить элемент списка по его индексу в 1С
Для Индекс = 0 По Список.Количество() - 1 Цикл
// нумерация с нуля
    Сообщить(Список[Индекс]);
КонецЦикла;

/// Как найти элемент списка по его значению в 1С
ЗначВторник = Список.НайтиПоЗначению("Вторник");

/// Как узнать индекс элемента в списке в 1С
Сообщить(Список.Индекс(ЗначВторник));
// 1, так как нумерация с нуля

/// Как отсортировать список по его значениям в 1С
Список.СортироватьПоЗначению
(НаправлениеСортировки.Убыв);

/// Как удалить элемент из списка значений в 1С
// удаляем первый элемент
Список.Удалить(Список[0]);
// параметр: элемент списка

/// Как сдвинуть элемент списка значений в 1С
// сдвигаем первый элемент на одну позицию вперед
Список.Сдвинуть(0, 1);

/// Как сделать копию списка в 1С
КопияСписка = Список.Скопировать();

```

```

Цвета = Новый СписокЗначений();
Цвета.Добавить("Красный");
Цвета.Добавить("Зеленый");
Цвета.Добавить("Синий");

///Как выгрузить значения списка в массив в 1С
МассивЦветов = Цвета.ВыгрузитьЗначения();
///Как загрузить значения списка из массива в 1С
Цвета.ЗагрузитьЗначения(МассивЦветов);

///Как сделать модальный выбор значения из списка в 1С
ВыбЦвет =
Цвета.ВыбратьЭлемент("Выберите любимый цвет");
Если ВыбЦвет <> Неопределено Тогда
    Сообщить(ВыбЦвет.Значение);
КонецЕсли;

///Как сделать модальную отметку значений из списка в 1С
Если Цвета.ОтметитьЭлементы("Выберите любимые цвета")
Тогда
    Для Каждого Цвет Из Цвета Цикл
        Если Цвет.Пометка Тогда
            Сообщить(Цвет.Значение);
        КонецЕсли;
    КонецЦикла;
КонецЕсли;
// можно разом установить все отметки в определенное значение
Цвета.ЗаполнитьПометки(Истина);

КонецПроцедуры

```

ЮНИТ 11. ТИП СТРУКТУРА В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Описание:

Представляет собой коллекцию пар КлючИЗначение. При этом ключ может быть только строковым и должен удовлетворять требованиям, предъявляемым к именованию переменных встроенного языка.

К значениям структуры можно обращаться как к свойствам объекта. При этом ключ используется как имя свойства.

Структура используется обычно для хранения небольшого количества значений, каждое из которых имеет некоторое имя.

Элементы коллекции: *КлючИЗначение*

Для объекта доступен обход коллекции посредством оператора *Для каждого ... Из ... Цикл*. При обходе выбираются элементы коллекции.

Свойства:

<Имя ключа>

В качестве свойств структура предоставляет элементы. Имя свойства определяется ключом, а значение свойства определяется значением элемента.

Методы:

Вставить(<Ключ>, <Значение>)

Устанавливает значение элемента структуры по ключу. Если элемент с переданным значением ключа существует, то его значение заменяется, в противном случае добавляется новый элемент.

Количество()

Получает количество элементов структуры.

Очистить()

Удаляет все элементы структуры.

Свойство(<Ключ>, <НайденноеЗначение>)

Получает значение элемента по указанному имени, а также проверяет, имеется ли указанное свойство. Возвращает *Истина* — ключ найден, *Ложь* — в противном случае.

Удалить(<Ключ>)

Удаляет элемент структуры с заданным ключом.

Конструкторы:

Новый Структура(<ФиксированнаяСтруктура>)

Создаваемая структура заполняется свойствами из исходной фиксированной структуры.

Новый Структура(<Ключ>, <Значения>)

Создает структуру с ключами, значения которых передаются в параметре конструктора. Если в первом параметре заданы ключи элементов структуры, то в следующих параметрах могут быть указаны значения этих элементов в том порядке, в котором они расположены в строке в первом параметре.

Процедура Пример()

/// Как создать структуру (способ 1) ЛичныеДанные = Новый Структура;

 // пара ключ (имя переменной) - значение (произв. типа)

 ЛичныеДанные.Вставить("Фамилия", "Иванов");


```
ЛичныеДанные.Вставить("Имя", "Иван");  
ЛичныеДанные.Вставить("ДатаРождения", '19800802');  
Сообщить(ЛичныеДанные.Имя + " " +  
ЛичныеДанные.Фамилия +  
" родился " + ЛичныеДанные.ДатаРождения);
```

/// Как создать структуру (способ 2)

```
Время = Новый Структура("Часы, Минуты, Секунды", 12, 45, 33);  
Сообщить("Время " + Время.Часы + ":" +  
Время.Минуты + ":" + Время.Секунды);
```

/// Как изменить значение поля созданной структуры

/// (способ 1)

```
ЛичныеДанные.Вставить("Фамилия", "Сидоров");
```

/// Как изменить значение поля созданной структуры

/// (способ 2)

```
Время.Часы = 13;
```

/// Как узнать количество элементов структуры

```
Сообщить(Время.Количество());
```

/// Как обойти все элементы структуры

```
Для Каждого Элемент Из ЛичныеДанные Цикл  
Сообщить(Элемент.Ключ + " " + Элемент.Значение);  
КонецЦикла;
```

/// Как удалить элемент структуры

```
ЛичныеДанные.Удалить("ДатаРождения");
```

/// Как очистить все элементы структуры

```
ЛичныеДанные.Очистить();
```

КонецПроцедуры

ЮНИТ 12. ТАБЛИЦА ЗНАЧЕНИЙ В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Описание:

Таблица значений — это не сохраняемый в базе данных объект, который позволяет строить динамические наборы значений и манипулировать ими (добавлять, редактировать, удалять элементы, сортировать). Он может быть наполнен значениями любого типа, т.е. в одной таблице типы хранимых значений могут быть разными.

Одним из примеров использования данного объекта может служить организация представления в форме списка элементов справочника, отобранных по сложному алгоритму.

Необходимо отметить, что таблица значений, связанная с элементом управления ТабличноеПоле, будет иметь все колонки, имеющиеся в табличном поле, с соответствующими именами. Кроме того, таблица значений может иметь также и собственные колонки, не отображаемые в табличном поле, при этом их имена должны отличаться от имен колонок табличного поля.

Элементы коллекции: СтрокаТаблицыЗначений

Для объекта доступен обход коллекции посредством оператора *Для каждого ... Из ... Цикл*. При обходе выбираются элементы коллекции.

Возможно обращение к элементу коллекции посредством оператора [...]. В качестве аргумента передается индекс строки (**нумерация с 0**).

Свойства:

Индексы

Содержит коллекцию индексов таблицы значений.

Колонки

Содержит коллекцию колонок таблицы значений.

Методы:

Вставить(<Индекс>)

Вставляет строку на позицию в таблице значений, соответствующую указанному индексу. Возвращает вставленную строку.

ВыбратьСтроку(<Заголовок>, <НачальнаяСтрока>)

Вызывает модальное диалоговое окно для интерактивного выбора строки из таблицы значений. Возвращает выбранную строку или *Неопределено*, если пользователь отказался от выбора.

ВыгрузитьКолонку(<Колонка>)

Создает массив и копирует в него значения, содержащиеся в колонке таблицы значений. Возвращает этот массив.

Добавить()

Добавляет строку в конец таблицы значений. Возвращает добавленную строку.

ЗагрузитьКолонку(<Массив>, <Колонка>)

Загружает колонку таблицы значений из массива значений.

ЗаполнитьЗначения(<Значение>, <Колонки>)

Заполняет все строки таблицы значений указанным значением.

Индекс(<Строка>)

Получает индекс строки в коллекции строк таблицы значений. Если не найдено, то возвращается —1.

Итог(<Колонка>)

Суммирует значения всех строк в указанной колонке.

Если в колонке установлен тип и он единственный, то при суммировании будет предприниматься попытка преобразования значения к типу *Число*.

Если колонке не присвоены типы, то в процессе суммирования будут принимать участие только значения, имеющие тип *Число*, значения других типов будут игнорироваться.

Если в колонке несколько типов и среди них есть тип *Число*, то в процессе суммирования будут принимать участие только значения, имеющие тип *Число*, значения других типов будут игнорироваться.

Если в колонке несколько типов и среди них нет типа *Число*, то результатом будет значение *Неопределено*.

Количество()

Получает количество строк таблицы значений.

Найти(<Значение>, <Колонки>)

Осуществляет поиск значения в указанных колонках таблицы значений. Возвращает строку, в которой содержится искомое значение. Если значение не найдено, то возвращается значение *Неопределено*.

НайтиСтроки(<ПараметрыОтбора>)

Осуществляет поиск строк таблицы значений, отвечающих заданным условиям поиска. Возвращает массив строк таблицы значений, соответствующих условиям поиска. Замечание! Массив хранит ссылки на строки таблицы значений, то есть при изменении строки в таблице значение в массиве тоже будет измененным.

Очистить()

Удаляет все строки таблицы значений. Структура колонок остается неизменной.

Получить(<Индекс>)

Получает значение по индексу. Работает аналогично оператору [].

Свернуть(<КолонкиГруппировок>, <КолонкиСуммирования>)

Осуществляет свертку таблицы значений по указанным колонкам группировки. Строки, у которых совпадают значения в колонках, указанных в первом параметре, сворачиваются в одну строку. Значения этих строк, хранящиеся в колонках, указанных во втором параметре, накапливаются. Важно! Списки колонок не должны пересекаться. Колонки, не вошедшие ни в один из списков колонок, после выполнения метода удаляются из таблицы значений.

Сдвинуть(<Строка>, <Смещение>)

Сдвигает строку на указанное количество позиций.

Скопировать(<Строки>, <Колонки>)

Если указаны строки и колонки, то только они будут скопированы. Если не указаны, то будет создана полная копия таблицы значений.

СкопироватьКолонки(<Колонки>)

Создает таблицу значений с заданным списком колонок.

Сортировать(<Колонки>, <ОбъектСравнения>)
Сортирует таблицу значений в соответствии с указанными правилами сортировки.

Удалить(<Строка>)

Удаляет строку таблицы значений, где *Строка* — индекс или сама строка.

Конструктор:

Новый ТаблицаЗначений

Процедура КнопкаВыполнитьНажатие (Кнопка)

/// Как создать таблицу значений

Таб = Новый ТаблицаЗначений;

/// Как добавить колонки в таблицу значений

Таб.Колонки.Добавить("Фрукт");

Таб.Колонки.Добавить("Цвет");

Таб.Колонки.Добавить("Вкус");

/// Как добавить строки в таблицу значений

Стр = Таб.Добавить();

Стр.Фрукт = "Яблоко";

Стр.Цвет = "Зеленый";

Стр.Вкус = "Кислый";

Стр = Таб.Добавить();

Стр.Фрукт = "Банан";

Стр.Цвет = "Желтый";

Стр.Вкус = "Сладкий";

Стр = Таб.Добавить();

Стр.Фрукт = "Слива";

Стр.Цвет = "Синий";

Стр.Вкус = "Терпкий";

/// Как связать таблицу значений на форме

ЭлементыФормы.Добавить(Тип("ТабличноеПоле"), "Таб", Истина);

ЭлементыФормы.Таб.Лево = 50;

ЭлементыФормы.Таб.Верх = 50;

ЭлементыФормы.Таб.Ширина = 800;

ЭлементыФормы.Таб.Высота = 600;


```
ЭлементыФормы.Таб.Значение = Таб;  
ЭлементыФормы.Таб.СоздатьКолонки();  
ЭлементыФормы.Таб.Колонки[0].Ширина = 50;  
ЭлементыФормы.Таб.ЧередованиеЦветовСтрок =  
Истина;
```

```
/// Как добавить в таблицу значений колонку с описанием  
/// типа и заголовка
```

```
ОписаниеВеса = Новый ОписаниеТипов("Число",  
    Новый КвалификаторыЧисла(12, 3));  
Таб.Колонки.Добавить("ВесФрукта", ОписаниеВеса, "Вес фрук-  
та");  
Таб[0].ВесФрукта = 100.005;  
Таб[1].ВесФрукта = 60.010;  
Таб[2].ВесФрукта = 25.000;  
ЭлементыФормы.Таб.СоздатьКолонки();
```

```
/// Как перечислить все строки в таблице значений в 1с
```

```
Для Каждого Стр Из Таб Цикл
```

```
    Сообщить(Стр.Фрукт + " " + Стр.Цвет + " " + Стр.Вкус);  
КонецЦикла;
```

```
/// Как перечислить все колонки в таблице значений
```

```
Для Каждого Колонка Из Таб.Колонки Цикл
```

```
    Сообщить(Колонка.Имя);  
КонецЦикла;
```

```
/// Как интерактивно выбрать строку из таблицы значений
```

```
// Таб.ВыбратьСтроку("Выберите фрукт", 1);
```

```
/// Как выгрузить колонку таблицы значений в массив
```

```
// выгрузка названий фруктов в массив
```

```
МассивФруктов = Таб.ВыгрузитьКолонку("Фрукт");
```

```
// ничего не измениться, так как массив не менялся
```

```
Таб.ЗагрузитьКолонку(МассивФруктов, "Фрукт");
```

```
/// Как заполнить значения в колонке для всех строк таблицы
```

```
/// значений
```

```
// обнулим вес во всех строках
```

```
Таб.ЗаполнитьЗначения(0, "ВесФрукта");
```

```
// выставим новые веса
```

```

Таб[0].ВесФрукта = 100;
Таб[1].ВесФрукта = 60;
Таб[2].ВесФрукта = 25;

/// Как в таблице значений посчитать итог по колонке
Сообщить(Таб.Итог("ВесФрукта")); // 185

/// Как найти строку в таблице значений по значению
// найдем строчку с яблоком и покажем его цвет
Сообщить(Таб.Найти( "Яблоко" ).Цвет); // Зеленый

/// Как найти строку в таблице значений поиском значения
/// в определенных колонках
// будем искать по слову "Кислый" в колонках вкус и цвет
Сообщить(Таб.Найти("Кислый", "Вкус, Цвет").Фрукт);
// Яблоко
// добавим еще один зеленый фрукт
Стр = Таб.Добавить();
Стр.Фрукт = "Киви";
Стр.Цвет = "Зеленый";
Стр.Вкус = "Сладкий";
Стр.ВесФрукта = 30;

/// Как искать в таблице значений неуникальные значения
Отбор = Новый Структура;
Отбор.Вставить("Цвет", "Зеленый");
Строки = Таб.НайтиСтроки(Отбор);
Для Каждого Стр Из Строки Цикл
    Сообщить(Стр.Фрукт); // Яблоко Киви
КонецЦикла;

/// Как сдвинуть строку таблицы значений
// сдвигаем первую строку на одну позицию вперед
Таб.Сдвинуть(0, 1);
// и возвращаем обратно
Таб.Сдвинуть(1, -1);

/// Как создать копию таблицы значений в 1с 8.3, 8.2
// полная копия
КопияТаб = Таб.Скопировать();

/// Как создать копию таблицы значений по нужным
/// колонкам

```

```

// только две колонки будет Фрукт и Цвет
КопияТаб = Таб.СкопироватьКолонки("Фрукт, Цвет");

/// Как создать копию таблицы значений по отбору
Отбор = Новый Структура;
Отбор.Вставить("Фрукт", "Яблоко");
// таблица будет содержать только строчку с яблоком
КопияТаб = Таб.Скопировать(Отбор);
// таблица будет содержать цвет и вес яблока
КопияТаб = Таб.Скопировать(Отбор, "Цвет, ВесФрукта");

/// Как свернуть таблицу значений по колонке
// свернем по вкусу, просуммируем по весу
Таб.Свернуть("Вкус", "ВесФрукта");

/// Как отсортировать таблицу значений по колонке
Таб.Сортировать("ВесФрукта Возр");

/// Как отсортировать таблицу значений по нескольким
/// колонкам
Таб.Сортировать("ВесФрукта Возр, Вкус Убыв");

/// Как удалить строку из таблицы значений
// по индексу или по значению
Таб.Удалить(0);

КонецПроцедуры

```

ЮНИТ 13. РАБОТА С ФАЙЛОВОЙ СИСТЕМОЙ В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Рассмотрим выполнение некоторых часто применяемых операций с файловой системой на конкретных примерах.

Процедура КнопкаВыполнитьНажатие(Кнопка)

```

/// Как скопировать файл (ftp, http, https)
ФайлИсточник = "c:\\file_src.txt";
ФайлПриемник = "c:\\file_dest.txt";
КопироватьФайл(ФайлИсточник, ФайлПриемник);

/// Как найти файлы в каталоге (ftp)
КаталоГдеИскать = "c:\\Documents";

```

```

// возвращает значения типа Файл; поиск нерекурсивный
МассивНайденныхФайлов =
НайтиФайлы(КаталоГдеИскать, "*.txt");
Для Каждого Файл Из МассивНайденныхФайлов Цикл
Сообщить(Файл.Имя);
КонецЦикла;

/// Как переместить файл (ftp, http, https)
ФайлИсточник = "c:\\file_src.txt";
ФайлПриемник = "c:\\file_dest.txt";
ПереместитьФайл(ФайлИсточник, ФайлПриемник);

/// Как удалить файл (ftp, http, https)
// если файл readonly - ошибка
ФайлДляУдаления = "c:\\file_for_delete.txt";
УдалитьФайлы(ФайлДляУдаления);
КаталогДляУдаленияФайловИзНего = "C:\\Documents";
// поиск нерекурсивный
УдалитьФайлы(КаталогДляУдаленияФайловИзНего, "*.txt");

/// Как создать каталог (ftp)
СоздатьКаталог("c:\\НовыйКаталог");
// каталог может уже существовать
СоздатьКаталог("c:\\НовыйКаталог\\2\\3");
// создадутся все каталоги

/// Как получить каталог временных файлов
Сообщить(КаталогВременныхФайлов());

/// Как получить имя временного файла
Сообщить(ПолучитьИмяВременногоФайла());
// с расширением tmp
Сообщить(ПолучитьИмяВременногоФайла(".txt"));
// с расширением txt

/// Как получить каталог программы
Сообщить(КаталогПрограммы());

/// Как записать значение в файл
Массив = Новый Массив(1);
Массив[0] = 10;
ВременныйФайл = ПолучитьИмяВременногоФайла();
ЗначениеВФайл(ВременныйФайл, Массив);

```



```
/// Как восстановить значение из файла
КопияМассива = ЗначениеИзФайла(ВременныйФайл);
Сообщить(КопияМассива[0]); // 10
```

```
/// Как проверить существование файла
КаталогНаДиске = Новый Файл("c:\windows");
Если КаталогНаДиске.Существует() Тогда
    КонецЕсли;
```

КонецПроцедуры

ЮНИТ 14. ФУНКЦИЯ ФОРМАТ В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Описание:

Формирует удобное для чтения представление значений. Полезно использование в отчетах и при прочем визуальном отображении значений.

Параметры:

<Значение>

Форматируемое значение.

<ФорматнаяСтрока>

Форматная строка представляет собой строковое значение, включающее параметры форматирования.

Параметры форматирования перечисляются через символ ";". Наличие параметра означает отличие форматирования от стандартного.

Если параметр не задан, то применяется стандартное форматирование, которое в основном соответствует преобразованию значения к строке. Однако для значений типа Число и Дата по умолчанию (0 и 01.01.0001 00:00:00 соответственно) будет выдаваться пустая строка.

Каждый параметр задается именем параметра, символом "=" и значением параметра. Значение параметра может указываться в одинарных или двойных кавычках. Это необходимо, если значение параметра содержит символы, используемые в синтаксисе форматной строки.

Внутри значения параметра могут указываться произвольные символы, которые при формировании результирующей строки будут отображаться как есть. Таким образом, например, можно указывать разделители часов, минут и секунд при форматировании времени. Если эти символы совпадают со значениями форматной строки, их необходимо заключать в одинарные кавычки.

Имена и значения параметров форматной строки:

- Л — имя языка, страны, для которых будет выполняться стандартное форматирование.
- ЧЦ — общее число отображаемых десятичных разрядов целой и дробной частей. Исходное число округляется при этом в соответствии с правилами округления. Если указан этот параметр, то для отображения дробной части числа обязательно указание параметра ЧДЦ, иначе дробная часть отображаться не будет.
- ЧДЦ — число десятичных разрядов в дробной части. Исходное число округляется при этом в соответствии с правилами округления.
- ЧС — сдвиг разрядов: положительный — деление, отрицательный — умножение. Другими словами, это означает, что исходное число будет умножено или поделено на 10^C , где C — значение параметра по модулю.
- ЧРД — символ-разделитель целой и дробной части.
- ЧРГ — символ-разделитель групп целой части числа. Если в качестве разделителя использовать пустую строку, то в этом случае разделителем будет символ неразрывного пробела.
- ЧН — строка, представляющая нулевое значение числа. Если не задано, то представление в виде пустой строки. Если задано "ЧН=", то в виде "0". Не используется для числовых полей ввода.
- ЧВН — нужно ли выводить лидирующие нули. Значение данного параметра не задается, собственно наличие параметра определяет вывод лидирующих нулей.
- ЧГ — порядок группировки разрядов числа. В качестве значения указываются числа, через запятую, обозначающие количество группируемых разрядов справа налево. Имеют смысл только два первых числа. Первое из них указывает первичную группировку, то есть ту, которая будет использована для наименее значимых разрядов целой части числа. Если второе число не указано, то будут сгруппированы только наименее значимые разряды. Если в качестве второго числа задан 0, то для всех разрядов целой части числа будет применено значение, указанное для первичной группировки. Если в качестве второго числа используется значение, отличное от 0, то это значение будет использовано для группировки всех разрядов, кроме уже сгруппированных наименее значимых.
- ЧО — представление отрицательных чисел.
 - 0 (0) — строка вида "(1,1)";
 - 1 (1) — строка вида "-1,1";
 - 2 (2) — строка вида "- 1,1";
 - 3 (3) — строка вида "1,1-";
 - 4 (4) — строка вида "1,1 -".

- **ДФ** — формат даты.
 - **д** — день месяца (цифрами) без лидирующего нуля;
 - **дд** — день месяца (цифрами) с лидирующим нулем;
 - **ддд** — краткое название дня недели;
 - **дддд** — полное название дня недели;
 - **М** — номер месяца (цифрами) без лидирующего нуля;
 - **ММ** — номер месяца (цифрами) с лидирующим нулем;
 - **МММ** — краткое название месяца;
 - **ММММ** — полное название месяца;
 - **к** — номер квартала в году;
 - **г** — номер года без века и лидирующего нуля;
 - **гг** — номер года без века с лидирующим нулем;
 - **гггг** — номер года с веком;
 - **ч** — час в 12 часовом варианте без лидирующих нулей;
 - **чч** — час в 12 часовом варианте с лидирующим нулем;
 - **Ч** — час в 24 часовом варианте без лидирующих нулей;
 - **ЧЧ (НН)** — час в 24 часовом варианте с лидирующим нулем;
 - **м** — минута без лидирующего нуля;
 - **мм** — минута с лидирующим нулем;
 - **с** — секунда без лидирующего нуля;
 - **сс** — секунда с лидирующим нулем;
 - **вв** — отображение половины дня АМ/РМ (действительно только для языков конфигурирования, поддерживающих 12 часовой вариант представления времени).
- **ДЛФ** — локальный формат даты. Указывается вариант отображения частей даты.
 - **Д** — дата (цифрами);
 - **ДД** — длинная дата (месяц прописью);
 - **В** — полное время, дата может объединяться со временем;
 - **ДВ** — дата время.
- **ДП** — строка, представляющая пустую дату (например, `Формат('00010101000000', "ДП=""пустая дата"")`) вернет строку "пустая дата").
- **БЛ** — строка, представляющая логическое значение *Ложь*.
- **БИ** — строка, представляющая логическое значение *Истина*.

Процедура `КнопкаВыполнитьНажатие(Кнопка)`

/// Как указать общее число цифр

`Сообщить(Формат(125, "ЧЦ=2")); // 99`

`Сообщить(Формат(125, "ЧЦ=1")); // 9`

/// Как указать число цифр после запятой

`Сообщить(Формат(125.368, "ЧДЦ=2")); // 125.37`

`Сообщить(Формат(125.368, "ЧДЦ=1")); // 125.4`

/// Как сдвинуть разряды числа

Сообщить(Формат(2, "ЧС=1")); // 0,2

Сообщить(Формат(2, "ЧС=-2")); // 200

/// Как задать разделитель целой и дробной части

Сообщить(Формат(125.36, "ЧРД=///")); // 125///36

/// Как задать разделитель групп

Сообщить(Формат(10000000, "ЧРГ=*")); // 10*000*000

/// Как задать нулевое представление числа

Сообщить(Формат(0, "ЧН=ноль")); // ноль

/// Как заставить выводить лидирующие нули

Сообщить(Формат(125, "ЧЦ=10; ЧВН=")); // 0 000 000 125

Сообщить(Формат(5, "ЧЦ=2; ЧВН=")); // 05

/// Как отключить группировку разрядов при выводе числа

Сообщить(Формат(1000000, "ЧГ=0")); // 1000000

/// Как получить отдельные компоненты даты при выводе

Дата = '20050101200103'; // 01 января 2005 года 20:01:03

// год

Сообщить(Формат(Дата, "ДФ=г")); // 5

Сообщить(Формат(Дата, "ДФ=гг")); // 05

Сообщить(Формат(Дата, "ДФ=гггг")); // 2005

// номер квартала

Сообщить(Формат(Дата, "ДФ=к")); // 1

// месяц

Сообщить(Формат(Дата, "ДФ=М")); // 1

Сообщить(Формат(Дата, "ДФ=ММ")); // 01

Сообщить(Формат(Дата, "ДФ=МММ")); // янв.

Сообщить(Формат(Дата, "ДФ=ММММ")); // Январь

// день

Сообщить(Формат(Дата, "ДФ=д")); // 1

Сообщить(Формат(Дата, "ДФ=дд")); // 01

Сообщить(Формат(Дата, "ДФ=ддд")); // Сб

Сообщить(Формат(Дата, "ДФ=дддд")); // суббота

// час

Сообщить(Формат(Дата, "ДФ=ч")); // 8

Сообщить(Формат(Дата, "ДФ=чч")); // 08

Сообщить(Формат(Дата, "ДФ=Ч")); // 20

Сообщить(Формат(Дата, "ДФ=ЧЧ")); // 20

// минута

Сообщить(Формат(Дата, "ДФ=м")); // 1


```

Сообщить(Формат(Дата, "ДФ=мм")); // 01
// секунда
Сообщить(Формат(Дата, "ДФ=с")); // 3
Сообщить(Формат(Дата, "ДФ=сс")); // 03

/// Как задать локальный формат даты при выводе
Сообщить(Формат(Дата, "ДФ=Д")); // 01.01.2005
Сообщить(Формат(Дата, "ДФ=ДД")); // 1 января 2005 г.
Сообщить(Формат(Дата, "ДФ=В")); // 20:01:03
Сообщить(Формат(Дата, "ДФ=ДДВ")); // 1 января
//2005 г. 20:01:03
Сообщить(Формат(Дата, "ДФ=ДВ")); // 01.01.2005
// 20:01:03

/// Как задать нулевое представление даты при выводе
Сообщить(Формат('00010101000000', "ДП=""пустая дата"""));
// пустая дата

/// Как задать представление значения типа Булево
/// при выводе
Сообщить(Формат(Ложь, "БЛ=""врешь!""")); // врешь!
Сообщить(Формат(Истина, "БИ=""правду говоришь""")); //
правду говоришь

/// Как задать представление периода при выводе
Сообщить(
    ПредставлениеПериода(
        НачалоКвартала(ТекущаяДата()),
        КонецКвартала(ТекущаяДата()),
        "ФП=Истина")); // 3 кв. 2013 г., влияет, если 6 или
// 9 месяцев

/// Как задать число прописью при выводе
Сообщить(
    ЧислоПрописью(123.65)
); // Сто двадцать три 65
Сообщить(
    ЧислоПрописью(
        123.65,,
        "рубль, рубля, рублей, м, копейка, копейки, копеек, ж, 2")
); // Сто двадцать три рубля 65 копеек

КонецПроцедуры

```

ЮНИТ 15. ОБЪЕКТ ЗАПРОС В ЯЗЫКЕ 1С (В ПРИМЕРАХ)

Объект Запрос встроенного языка системы предоставляет эффективный механизм обработки данных. Для этого объект формирует

временную таблицу, структура и содержимое которой полностью определяются *текстом запроса*.

Текст запроса, по сути дела, определяет структуру этой временной таблицы, которая будет построена в результате работы объекта. Эта таблица имеет тип данных — *РезультатЗапроса*, который обладает достаточным набором методов для организации выборки, извлечения информации из временной таблицы. В этом юните мы рассмотрим применение объекта и написание текста запроса на конкретных примерах работы с условным справочником: "Номенклатура".

/// Пример 1. Как запросить все строки со значениями всех столбцов из справочника "Номенклатура"

Процедура

ЗапросВсехСтрокСоЗначениямиВсехСтолбцовТаблицы()

Запрос = Новый Запрос(

"

| ВЫБРАТЬ

| *

| ИЗ

| Справочник.Номенклатура

|"

);

Результат = Запрос.Выполнить();

ОткрытьЗначение

(Результат.Выгрузить(ОбходРезультатаЗапроса.Прямой));

КонецПроцедуры

/// Пример 2. Как запросить все строки со значениями определенных столбцов

Процедура

ЗапросВсехСтрокСоЗначениямиОпределенныхСтолбцов()

Запрос = Новый Запрос(

"

| ВЫБРАТЬ

| Код, Наименование, Родитель

| ИЗ

| Справочник.Номенклатура

|"

);

Результат = Запрос.Выполнить();

ОткрытьЗначение

(Результат.Выгрузить(ОбходРезультатаЗапроса.Прямой));

КонецПроцедуры

**/// Пример 3. Как использовать в запросах псевдонимы
/// для столбцов и таблиц**

Процедура ИспользованиеПсевдонимов()

```
    Запрос = Новый Запрос(
        "
        | ВЫБРАТЬ
        | Товары.Наименование КАК Имя, Товары.Родитель КАК Группа
        | ИЗ
        | Справочник.Номенклатура КАК Товары
        | "
    );
    Результат = Запрос.Выполнить();
    ОткрытьЗначение
(Результат.Выгрузить(ОбходРезультатаЗапроса.Прямой));
КонецПроцедуры
```

**/// Пример 4. Как добавить в результат запроса столбец с
/// определенным значением**

Процедура

ДобавлениеВРезультатЗапросаСтолбцаСОпределеннымЗначением()

```
    Запрос = Новый Запрос(
        "
        | ВЫБРАТЬ
        | Наименование, 10000 КАК Цена, ""Оранжевый"" КАК Цвет
        | ИЗ
        | Справочник.Номенклатура
        | "
    );
    Результат = Запрос.Выполнить();
    ОткрытьЗначение
(Результат.Выгрузить(ОбходРезультатаЗапроса.Прямой));
КонецПроцедуры
```

**/// Пример 5. Как запросить неповторяющиеся (различные)
/// строки из таблицы**

Процедура ЗапросРазличныхСтрокИзТаблицы()

```
    Запрос = Новый Запрос(
        "
        | ВЫБРАТЬ РАЗЛИЧНЫЕ
        | Родитель
        | ИЗ
```

```

| Справочник.Номенклатура
|
);
Результат = Запрос.Выполнить();
ОткрытьЗначение
(Результат.Выгрузить(ОбходРезультатаЗапроса.Прямой));
КонецПроцедуры

```

/// Пример 6. Как запросить первые N строк из таблицы

Процедура ЗапросПервыхNСтрокИзТаблицы()

Запрос = Новый Запрос(

"

| ВЫБРАТЬ ПЕРВЫЕ 10

| Наименование

| ИЗ

| Справочник.Номенклатура

"

);

Результат = Запрос.Выполнить();

ОткрытьЗначение

(Результат.Выгрузить(ОбходРезультатаЗапроса.Прямой));

КонецПроцедуры

/// Пример 7. Как выбрать строки из таблицы в соответствии

/// с правами пользователя

Процедура ВыборкаРазрешенныхСтрокИзТаблицы()

Запрос = Новый Запрос(

"

| ВЫБРАТЬ РАЗРЕШЕННЫЕ

| *

| ИЗ

| Справочник.Номенклатура

"

);

Результат = Запрос.Выполнить();

ОткрытьЗначение

(Результат.Выгрузить(ОбходРезультатаЗапроса.Прямой));

КонецПроцедуры

ЗАКЛЮЧЕНИЕ

Учебное пособие обращено в большей степени к начинающим пользователям, делающим первые шаги в разработке прикладных решений. Пояснения, приведенные в книге, подробны и доступны даже для тех, кто лишь отдаленно знаком с азами программирования.

Если вы только начинаете работу с "1С" или даже совсем не знакомы с этой системой, но очень хотите научиться, то эта книга — для вас. Цель книги — "провести вас за руку" по основным этапам разработки простого прикладного решения в системе 1С:Предприятие 8 и показать, что нет ничего недоступного для человека с интеллектом.

Более опытным пользователям эта книга также будет полезна и позволит вспомнить или подробнее изучить отдельные моменты программирования на встроенном языке 1С 8.2 и 8.3.

ЛИТЕРАТУРА

1. *Дадян Э.Г.* Проектирование бизнес-приложений в системе «1С: Предприятие 8» [Текст]: учеб. пособие / Э.Г. Дадян. — М.: Вузовский учебник: ИНФРА-М, 2014. — 283 с.
2. *Дадян Э.Г.* 1С: Предприятие. Проектирование приложений [Текст]: учеб. пособие / Э.Г. Дадян. — М.: Вузовский учебник: ИНФРА-М, 2015. — 288 с.
3. *Радченко М.Г.* 1С: Предприятие 8.2. Практическое пособие разработчика. Примеры и типовые приемы [Текст] / М.Г. Радченко, Е.Ю. Хрусталева. — М.: ООО «1С-Паблишинг», 2013. — 964 с.
4. *Дадян Э.Г.* Повышение эффективности обучения на основе применения облачных технологий в учебном процессе [Текст] / Э.Г. Дадян // Информатика и образование. — 2016. — № 2 (271). — С. 54–56.
5. *Дадян Э.Г.* Выбор программных продуктов формирования электронного документооборота для учебного центра в сборнике: Наука в современном информационном обществе. Материалы V международной научно-практической конференции [Текст] / Э.Г. Дадян, М.А. Шестовец. — М., 2015. — С. 110.
6. *Дадян Э.Г.* Инновационные тенденции преподавания дисциплин практической направленности в высшей школе. Тенденции науки и образования в современном мире [Текст] / Э.Г. Дадян. — М., 2015. — № 4. — С. 9, 10.
7. *Дадян Э.Г.* Повышение эффективности образовательного процесса с использованием технологий «1С» в сборнике: Новые информационные технологии в образовании: применение технологий «1С» для повышения эффективности деятельности организаций образования [Текст]: Сборник научных трудов Четырнадцатой Международной научно-практической конференции / Э.Г. Дадян. — М., 2014. — С. 195–197.
8. *Дадян Э.Г.* Информационная система автоматизации рутинных процессов вуза [Текст] / Э.Г. Дадян // Информатика и образование. — 2014. — № 8 (257). — С. 28, 29.
9. *Дадян Э.Г.* Применение инновационных технологий в системе профессионального образования [Текст] / Э.Г. Дадян // Информатика и образование. — 2014. — № 9. — С. 37.
10. *Дадян Э.Г.* Информационная система расчета оценки государственного экзамена и выпускной квалификационной работы бакалавра. Гуманитарные, социально-экономические и общественные науки [Текст] / Э.Г. Дадян. — М., 2014. — № 11-2. — С. 54–56.
11. *Дадян Э.Г.* Привлечение инновационных методов в учебном процессе на примере дисциплины «Проектирование бизнес приложений в системе 1С: предприятие» в сборнике: Информационные технологии в фи-

нансово-экономической сфере: прошлое, настоящее, будущее. Международная научная конференция [Текст] / Э.Г. Дадян. — М., 2013. — С. 195–199.

12. Дадян Э.Г. Проектирование современных баз данных [Текст]: учеб. пособие для студентов, обучающихся по специальностям «Бухгалтерский учет, анализ и аудит», «Финансы и кредит», «Мировая экономика», «Налоги и налогообложение» / Э.Г. Дадян // Финансовая акад. при Правительстве Российской Федерации. — М., 2007. — С. 195–199.

13. Дадян Э.Г. Методы, модели, средства хранения и обработки данных [Текст]: учебник / Э.Г. Дадян, Ю.А. Зеленков. — М.: Вузовский учебник: ИНФРА-М. — М., 2016. — 168 с.

СОДЕРЖАНИЕ

Введение	3
 Раздел 1 ОСНОВЫ ЯЗЫКА ПРОГРАММИРОВАНИЯ 1С 8.3 ДЛЯ НАЧИНАЮЩИХ ПОЛЬЗОВАТЕЛЕЙ	
Юнит 1. Установка 1с и наша первая программа.....	4
Юнит 2. Основные правила языка.....	11
Юнит 3. Типы данных в языке 1С	15
Юнит 4. Имена переменных в языке 1С.....	18
Юнит 5. Простые логические выражения в языке 1С	22
Юнит 6. Сложные логические выражения в языке 1С.....	25
Юнит 7. Условная команда в языке 1С	29
Юнит 8. Цикл "Для" в языке 1С.....	32
Юнит 9. Цикл "Пока" в языке 1С.....	35
Юнит 10. Комбинирование простых конструкций в языке 1С.....	37
Юнит 11. Массивы в языке 1С	40
Юнит 12. Структуры в языке 1С.....	44
Юнит 13. Функции и процедуры в языке 1С	47
Юнит 14. Функции и процедуры в языке 1С (Продолжение)	55
Юнит 15. Отладка в языке 1С	60
Юнит 16. Синтакс-помощник в языке 1С	65
Юнит 17. Работа с датами в языке 1С	70
Юнит 18. Математика в языке 1С	73
Юнит 19. Формат в языке 1С	76
Юнит 20. Использование диалогов в языке 1С	78
Юнит 21. Список значений в языке 1С	84
 Раздел 2 ОСНОВЫ ЯЗЫКА ПРОГРАММИРОВАНИЯ 1С 8.3 ДЛЯ ПРОДВИНУТЫХ ПОЛЬЗОВАТЕЛЕЙ (В ПРИМЕРАХ)	
Юнит 1. Работа с типом Дата в языке 1С (в примерах)	87
Юнит 2. Массивы в языке 1С (в примерах)	88
Юнит 3. Математика в языке 1С	94
Юнит 4. Процедуры в языке 1С (в примерах)	96
Юнит 5. Строки в языке 1С (в примерах).....	98
Юнит 6. Циклы (в примерах)	101
Юнит 7. Описание типов в языке 1С (в примерах).....	102

Юнит 8. Работа с операционной системой в языке 1С (в примерах)	104
Юнит 9. Тип <i>Соответствие</i> в языке 1С (в примерах)	105
Юнит 10. Список значений в языке 1С (в примерах)	107
Юнит 11. Тип <i>Структура</i> в языке 1С (в примерах)	111
Юнит 12. Таблица значений в языке 1С (в примерах)	113
Юнит 13. Работа с файловой системой в языке 1С (в примерах)	119
Юнит 14. Функция <i>Формат</i> в языке 1С (в примерах)	121
Юнит 15. Объект <i>Запрос</i> в языке 1С (в примерах)	125
Заключение	129
Литература	130